

# 长广溪智能制造（无锡）有限公司

Changuangxi Intelligent Manufacturing (Wuxi) Co., Ltd.

## 协作机器人二次开发接口文档 (C/C++)

文档版本: v2.20e

发布日期: 2025 年 10 月 27 日

文档编号: CGXi-QM-CR-0012



长广溪智能制造（无锡）有限公司

Changuangxi Intelligent Manufacturing(Wuxi) Co., Ltd.



长广溪智能制造（无锡）有限公司（简称长广溪智造）保留对此文件修改之权利且不另行通知。长广溪智能制造（无锡）有限公司所提供之信息相信为正确且可靠之信息，但并不保证本文件中绝无错误。请于向长广溪智能制造（无锡）有限公司提出订单前，自行确定所使用之相关技术文件及规格为最新之版本。若因贵公司使用本公司之文件或产品，而涉及第三人之专利或著作权等智能财产权之应用及配合时，则应由贵公司负责取得同意及授权，本公司仅单纯贩售产品，上述关于同意及授权，非属本公司应为保证之责任。又未经长广溪智能制造（无锡）有限公司之正式书面许可，本公司之所有产品不得使用于医疗器材，维持生命系统及飞航等相关设备。

## 注意事项

本手册仅适用于长广溪智能制造（无锡）有限公司 G 系列协作机器人。

# 目录

|                         |    |
|-------------------------|----|
| 一、SDK 介绍 .....          | 12 |
| 二、数据结构 .....            | 12 |
| 2.1 机械臂句柄 .....         | 12 |
| 2.2 接口调用返回值 .....       | 12 |
| 2.3 坐标系选择 .....         | 13 |
| 2.4 点转换类型 .....         | 13 |
| 2.5 位姿变换 .....          | 14 |
| 2.6 运动触发模式 .....        | 14 |
| 2.7 TCP 偏移量 .....       | 14 |
| 2.8 运动参数 .....          | 14 |
| 2.9 简化版运动参数 .....       | 15 |
| 2.10 圆弧运动参数 .....       | 16 |
| 2.11 运动模式 .....         | 16 |
| 2.12 关节状态 .....         | 16 |
| 2.13 机械臂状态 .....        | 16 |
| 2.14 安全模式 .....         | 17 |
| 2.15 程序状态 .....         | 18 |
| 2.16 弹窗信息 .....         | 18 |
| 2.17 版本信息 .....         | 18 |
| 2.18 Lua 变量 .....       | 19 |
| 2.19 变量元素 .....         | 19 |
| 2.20 变量信息 .....         | 19 |
| 2.21 消息来源 .....         | 20 |
| 2.22 警告等级 .....         | 20 |
| 2.23 世界时间 .....         | 20 |
| 2.24 日志信息 .....         | 21 |
| 2.25 版本信息 .....         | 21 |
| 2.26 负载 .....           | 21 |
| 2.27 校验码检查 .....        | 21 |
| 2.28 串口通信 .....         | 22 |
| 2.29 波特率 .....          | 22 |
| 2.30 串口配置 .....         | 23 |
| 2.31 IO 类型 .....        | 23 |
| 2.32 安装角度 .....         | 23 |
| 2.33 系统功能输入 .....       | 23 |
| 2.34 系统功能输出 .....       | 24 |
| 2.35 IO 选项卡输出 .....     | 24 |
| 2.36 输出信号状态（自动复位） ..... | 25 |
| 2.37 输入信号 .....         | 25 |

|                          |    |
|--------------------------|----|
| 2.38 输出信号 .....          | 25 |
| 2.39 工具 IO 类型 .....      | 25 |
| 2.40 模拟量类型 .....         | 25 |
| 2.41 模拟量输入 .....         | 26 |
| 2.42 模拟量输出 .....         | 26 |
| 2.43 工具电压 .....          | 26 |
| 2.44 编码器配置 .....         | 26 |
| 2.45 BOOL 寄存器 .....      | 26 |
| 2.46 寄存器 .....           | 27 |
| 2.47 工具端信息 .....         | 27 |
| 2.48 点坐标系 .....          | 27 |
| 2.49 线坐标系 .....          | 27 |
| 2.50 面坐标系 .....          | 28 |
| 2.51 机器人限值等级 .....       | 28 |
| 2.52 碰撞后处理模式 .....       | 28 |
| 2.53 关节限值模式 .....        | 29 |
| 2.54 安全限值模式 .....        | 29 |
| 2.55 安全限值 .....          | 29 |
| 2.56 关节限值 .....          | 29 |
| 2.57 安全平面 .....          | 30 |
| 2.58 安全 IO 输入信号 .....    | 30 |
| 2.59 安全 IO 输出信号 .....    | 30 |
| 2.60 控制柜网络配置 .....       | 30 |
| 2.61 MODBUS 地址映射功能 ..... | 31 |
| 2.62 MODBUS 地址映射 .....   | 31 |
| 2.63 MODBUS 连接状态 .....   | 31 |
| 2.64 MODBUS 参数配置 .....   | 32 |
| 2.65 通讯变量选择 .....        | 32 |
| 2.66 脚本程序状态 .....        | 32 |
| 2.67 串口类型 .....          | 32 |
| 2.68 轨迹点数据 .....         | 33 |
| 2.69 轨迹参数 .....          | 33 |
| 2.70 轨迹数据 .....          | 33 |
| 2.71 下载轨迹数据 .....        | 33 |
| 2.72 轨迹记录参数 .....        | 33 |
| 2.73 轨迹记录状态 .....        | 34 |
| 2.74 轨迹运行状态 .....        | 34 |
| 2.75 控制模式 .....          | 34 |
| 2.76 力控设置 .....          | 34 |
| 2.77 力控传感器 tcp .....     | 35 |
| 2.78 力控传感器负载 .....       | 35 |

|                           |    |
|---------------------------|----|
| 2.79 力控传感器信息 .....        | 35 |
| 2.80 通讯配置 .....           | 35 |
| 2.81 力控传感器工具设置 .....      | 35 |
| 2.82 力控配置 .....           | 36 |
| 2.83 力控控制参数 .....         | 36 |
| 2.84 力控数据 .....           | 36 |
| 2.85 机器人状态数据 .....        | 37 |
| 2.86 机械臂和扩展轴数据 .....      | 37 |
| 2.87 扩展轴关节移动参数 .....      | 37 |
| 2.88 扩展轴点动运动参数 .....      | 38 |
| 2.89 扩展轴运动参数 .....        | 38 |
| 2.90 扩展轴类型 .....          | 38 |
| 2.91 扩展轴配置 .....          | 38 |
| 2.92 常用宏定义 .....          | 39 |
| 三、接口函数 .....              | 40 |
| 3.1 基础 .....              | 40 |
| 3.1.1 基础接口 .....          | 40 |
| 3.1.1.1 机器人创建连接 .....     | 40 |
| 3.1.1.2 机器人断开连接 .....     | 41 |
| 3.1.1.3 机器人关机 .....       | 41 |
| 3.1.1.4 机械臂上电 .....       | 42 |
| 3.1.1.5 机械臂断电 .....       | 43 |
| 3.1.1.6 机械臂使能 .....       | 43 |
| 3.1.1.7 机械臂关使能 .....      | 44 |
| 3.1.1.8 机械臂故障复位 .....     | 45 |
| 3.1.1.9 获取控制柜状态 .....     | 45 |
| 3.1.1.10 写机械臂速度百分比 .....  | 46 |
| 3.1.1.11 读机械臂速度百分比 .....  | 47 |
| 3.1.1.12 获取 SDK 版本号 ..... | 47 |
| 3.1.1.13 获取机械臂状态数据 .....  | 48 |
| 3.1.2 程序控制 .....          | 49 |
| 3.1.2.1 下载脚本程序 .....      | 49 |
| 3.1.2.2 下载脚本程序（加密） .....  | 50 |
| 3.1.2.3 上传脚本程序 .....      | 52 |
| 3.1.2.4 运行程序 .....        | 52 |
| 3.1.2.5 停止程序运行 .....      | 53 |
| 3.1.2.6 暂停程序运行 .....      | 54 |
| 3.1.2.7 读取脚本当前运行行号 .....  | 54 |
| 3.1.2.8 读脚本运行状态 .....     | 55 |
| 3.1.2.9 上传工程文件 .....      | 56 |
| 3.1.2.10 下载工程文件 .....     | 56 |

|                                           |    |
|-------------------------------------------|----|
| 3.1.2.11 是否存在弹窗 .....                     | 57 |
| 3.1.2.12 读取弹窗信息 .....                     | 58 |
| 3.1.2.13 设置输入弹窗信息 .....                   | 59 |
| 3.1.2.14 关闭弹窗 .....                       | 60 |
| 3.2 运动模块 .....                            | 60 |
| 3.2.1 运动接口 .....                          | 60 |
| 3.2.1.1 轴空间运动 moveJ .....                 | 60 |
| 3.2.1.2 轴空间运动 movejoint .....             | 61 |
| 3.2.1.3 直线运动 moveL .....                  | 63 |
| 3.2.1.4 直线运动 moveline .....               | 64 |
| 3.2.1.5 点动运动 .....                        | 65 |
| 3.2.1.6 圆弧运动 .....                        | 66 |
| 3.2.1.7 运动控制 .....                        | 68 |
| 3.2.1.8 设置阻塞移动中参数阈值 .....                 | 69 |
| 3.2.1.9 PointControlParaSimple 数据转换 ..... | 70 |
| 3.2.1.10 设置软件自由驱动启动 .....                 | 71 |
| 3.2.1.11 读机械臂运动状态 .....                   | 71 |
| 3.2.1.12 读取移动缓存区中移动指令的数量 .....            | 72 |
| 3.2.2 运动状态信息 .....                        | 73 |
| 3.2.2.1 读实际 TCP 位置 .....                  | 73 |
| 3.2.2.2 读目标 TCP 位置 .....                  | 74 |
| 3.2.2.3 读实际 TCP 速度 .....                  | 74 |
| 3.2.2.4 读目标 TCP 速度 .....                  | 75 |
| 3.2.2.5 读实际 TCP 加速度 .....                 | 76 |
| 3.2.2.6 读当前使用的 TCP 偏移信息 .....             | 77 |
| 3.2.2.7 读取当前使用负载信息 .....                  | 78 |
| 3.2.3 运动配置 .....                          | 79 |
| 3.2.3.1 读所有 TCP 偏移信息列表 .....              | 79 |
| 3.2.3.2 读取所有负载信息 .....                    | 80 |
| 3.2.3.3 获取指定 TCPoffset 的位姿 .....          | 81 |
| 3.2.4 关节状态 .....                          | 82 |
| 3.2.4.1 读实际关节位置 .....                     | 82 |
| 3.2.4.2 读目标关节位置 .....                     | 82 |
| 3.2.4.3 读实际关节速度 .....                     | 83 |
| 3.2.4.4 读目标关节速度 .....                     | 84 |
| 3.2.4.5 读实际关节加速度 .....                    | 85 |
| 3.2.4.6 读目标关节加速度 .....                    | 86 |
| 3.2.4.7 读实际关节电机电流 .....                   | 86 |
| 3.2.4.8 读目标关节电机电流 .....                   | 87 |
| 3.2.4.9 读目标关节转矩 .....                     | 88 |
| 3.2.4.10 读实际关节采集的母线电压 .....               | 89 |

|                              |     |
|------------------------------|-----|
| 3.2.4.11 读关节温度 .....         | 90  |
| 3.2.4.12 读关节模式 .....         | 90  |
| 3.2.5 轨迹复现 .....             | 91  |
| 3.2.5.1 读取控制柜当前存在的轨迹索引 ..... | 91  |
| 3.2.5.2 上传轨迹 .....           | 92  |
| 3.2.5.3 下载轨迹 .....           | 93  |
| 3.2.5.4 轨迹控制 .....           | 98  |
| 3.2.5.5 轨迹数据转换为轨迹文件 .....    | 100 |
| 3.2.5.6 轨迹文件转换为轨迹数据 .....    | 101 |
| 3.2.5.7 设置轨迹记录参数 .....       | 101 |
| 3.2.5.8 获取当前轨迹记录状态 .....     | 102 |
| 3.2.5.9 获取当前轨迹运行状态 .....     | 103 |
| 3.2.6 扩展轴运动 .....            | 104 |
| 3.2.6.1 扩展轴关节移动 .....        | 104 |
| 3.2.6.2 扩展轴直线移动 .....        | 105 |
| 3.2.6.3 扩展轴点动运动 .....        | 106 |
| 3.2.6.4 扩展轴运动 .....          | 107 |
| 3.3 系统属性 .....               | 108 |
| 3.3.1 硬件信息 .....             | 108 |
| 3.3.1.1 读取控制柜硬件序列号 .....     | 108 |
| 3.3.1.2 读取整臂序列号 .....        | 109 |
| 3.3.1.3 获取版本信息 .....         | 109 |
| 3.3.2 本体参数 .....             | 110 |
| 3.3.2.1 读机械臂 DH 参数 .....     | 110 |
| 3.3.2.1 读机械臂标准 DH 参数 .....   | 111 |
| 3.3.2.2 读机械臂动力学参数 .....      | 112 |
| 3.3.2.3 读机械臂零位补偿值 .....      | 112 |
| 3.3.2.4 读机械臂减速比 .....        | 113 |
| 3.3.3 系统信息 .....             | 114 |
| 3.3.3.1 读机械臂当前状态 .....       | 114 |
| 3.3.3.2 获取日志数量 .....         | 115 |
| 3.3.3.3 获取机械臂实时日志 .....      | 115 |
| 3.3.3.4 获取机械臂历史所有日志 .....    | 116 |
| 3.3.3.5 日志功能是否开启 .....       | 117 |
| 3.3.3.6 日志大小设置 .....         | 118 |
| 3.3.3.7 读系统时间 .....          | 119 |
| 3.3.3.8 读取机型型号 .....         | 119 |
| 3.3.3.9 读取控制器型号 .....        | 120 |
| 3.3.3.10 触发黑匣子数据 .....       | 121 |
| 3.3.3.11 获取配置改变标志 .....      | 122 |
| 3.3.3.12 触发系统更新 .....        | 122 |



|                                       |     |
|---------------------------------------|-----|
| 3.4 变量 .....                          | 123 |
| 3.4.1 数字量 .....                       | 123 |
| 3.4.1.1 读控制柜标准数字输出 .....              | 123 |
| 3.4.1.2 写控制柜标准数字输出 .....              | 124 |
| 3.4.1.3 读控制柜可配置数字输出 .....             | 125 |
| 3.4.1.4 读控制柜所有可配置数字输出 .....           | 126 |
| 3.4.1.5 写控制柜可配置数字输出 .....             | 126 |
| 3.4.1.6 读工具端数字输出 .....                | 127 |
| 3.4.1.7 写工具端数字输出 .....                | 128 |
| 3.4.1.8 读控制柜标准数字输入 .....              | 129 |
| 3.4.1.9 读控制柜可配置数字输入 .....             | 130 |
| 3.4.1.10 读控制柜所有可配置数字输入 .....          | 130 |
| 3.4.1.11 读工具端数字输入 .....               | 131 |
| 3.4.2 模拟量 .....                       | 132 |
| 3.4.2.1 读控制柜模拟量输出 .....               | 132 |
| 3.4.2.2 写控制柜模拟量输出 .....               | 133 |
| 3.4.2.3 读工具端模拟量输入 .....               | 134 |
| 3.4.2.4 读控制柜模拟量输入 .....               | 134 |
| 3.4.2.5 读机械臂控制柜和工具端上所有输入数字量和模拟量 ..... | 135 |
| 3.4.2.6 读机械臂控制柜和工具端上所有输出数字量和模拟量 ..... | 136 |
| 3.4.3 通讯变量 .....                      | 138 |
| 3.4.3.1 读 bool 寄存器 .....              | 138 |
| 3.4.3.2 写 bool 寄存器 .....              | 139 |
| 3.4.3.3 读 int16 寄存器 .....             | 139 |
| 3.4.3.4 写 int16 寄存器 .....             | 140 |
| 3.4.3.5 读 int32 寄存器 .....             | 141 |
| 3.4.3.6 写 int32 寄存器 .....             | 142 |
| 3.4.3.7 读 float 寄存器 .....             | 143 |
| 3.4.3.8 写 float 寄存器 .....             | 144 |
| 3.4.4 脚本变量 .....                      | 145 |
| 3.4.4.1 读安装变量 .....                   | 145 |
| 3.4.4.2 修改安装变量 .....                  | 145 |
| 3.4.4.3 读取程序变量数据 .....                | 146 |
| 3.4.4.4 修改程序变量数据 .....                | 147 |
| 3.4.5 其他 IO .....                     | 148 |
| 3.4.5.1 获取编码器计数数量 .....               | 148 |
| 3.4.5.2 设置监控界面的工具输出电压 .....           | 149 |
| 3.4.5.3 读取监控界面的工具输出电压 .....           | 149 |
| 3.5 数学模块 .....                        | 150 |
| 3.5.1 位姿变换 .....                      | 150 |
| 3.5.1.1 位姿转换 .....                    | 150 |

|                                |     |
|--------------------------------|-----|
| 3.5.1.2 位姿求逆 .....             | 151 |
| 3.5.1.3 轴角转欧拉角 .....           | 152 |
| 3.5.1.4 欧拉角转轴角 .....           | 153 |
| 3.5.1.5 TCP 位姿转法兰位姿 .....      | 153 |
| 3.5.1.6 基坐标系下位姿转移到用户坐标系下 ..... | 154 |
| 3.5.1.7 目标点旋转变换 .....          | 155 |
| 3.5.2 运动学 .....                | 156 |
| 3.5.2.1 正解 .....               | 156 |
| 3.5.2.2 逆解 .....               | 157 |
| 3.6 配置 .....                   | 158 |
| 3.6.1 安装 .....                 | 158 |
| 3.6.1.1 设置安装角度 .....           | 158 |
| 3.6.1.2 读取安装角度 .....           | 159 |
| 3.6.2 TCP .....                | 159 |
| 3.6.2.1 读取 TCP 个数 .....        | 159 |
| 3.6.2.2 读取 TCP 数据 .....        | 160 |
| 3.6.2.3 读取 TCP 数据(通过名称) .....  | 161 |
| 3.6.2.4 增加 TCP .....           | 162 |
| 3.6.2.5 删除 TCP .....           | 163 |
| 3.6.2.6 编辑 TCP 数据 .....        | 163 |
| 3.6.2.7 读取当前激活的 TCP 索引 .....   | 164 |
| 3.6.2.8 激活 TCP .....           | 165 |
| 3.6.2.9 计算 TCP 位置 .....        | 166 |
| 3.6.2.10 计算 TCP 角度 .....       | 167 |
| 3.6.3 负载 .....                 | 168 |
| 3.6.3.1 读取负载个数 .....           | 168 |
| 3.6.3.2 读取负载数据 .....           | 168 |
| 3.6.3.3 读取负载数据(通过名称) .....     | 169 |
| 3.6.3.4 增加负载 .....             | 170 |
| 3.6.3.5 删除负载 .....             | 171 |
| 3.6.3.6 编辑负载 .....             | 171 |
| 3.6.3.7 读取当前激活的负载索引 .....      | 172 |
| 3.6.3.8 激活负载 .....             | 173 |
| 3.6.4 初始位/包装位 .....            | 174 |
| 3.6.4.1 设置初始位 .....            | 174 |
| 3.6.4.2 读取初始位 .....            | 174 |
| 3.6.4.3 删除初始位 .....            | 175 |
| 3.6.4.4 读取包装位 .....            | 176 |
| 3.6.5 启动 .....                 | 177 |
| 3.6.5.1 设置机器人是否自动上电使能 .....    | 177 |
| 3.6.5.2 读取机器人自动上电状态 .....      | 177 |

|                            |     |
|----------------------------|-----|
| 3.6.6 控制参数 .....           | 178 |
| 3.6.6.1 设置机器人拖动阻尼 .....    | 178 |
| 3.6.6.2 读取机器人拖动阻尼 .....    | 179 |
| 3.6.6.3 设置转矩异常门限倍率 .....   | 180 |
| 3.6.6.4 读转矩异常门限倍率 .....    | 181 |
| 3.6.6.5 设置扩展轴步长及速度倍率 ..... | 181 |
| 3.6.6.6 读扩展轴步长及速度倍率 .....  | 182 |
| 3.6.7 机器人限值 .....          | 183 |
| 3.6.7.1 设置限制模式 .....       | 183 |
| 3.6.7.2 读取限制模式 .....       | 184 |
| 3.6.7.3 设置自定义模式下参数 .....   | 184 |
| 3.6.7.4 读取自定义模式下参数 .....   | 185 |
| 3.6.7.5 设置碰撞后处理方式 .....    | 186 |
| 3.6.7.6 读取碰撞后处理方式 .....    | 187 |
| 3.6.8 关节限值 .....           | 188 |
| 3.6.8.1 设置关节限值 .....       | 188 |
| 3.6.8.2 读取关节限值 .....       | 189 |
| 3.6.9 安全平面 .....           | 189 |
| 3.6.9.1 读取安全平面个数 .....     | 189 |
| 3.6.9.2 读取安全平面信息 .....     | 190 |
| 3.6.9.3 增加安全平面 .....       | 191 |
| 3.6.9.4 删除安全平面 .....       | 192 |
| 3.6.9.5 修改安全平面数据 .....     | 192 |
| 3.6.10 安全 I/O .....        | 193 |
| 3.6.10.1 设置可配置输入信号 .....   | 193 |
| 3.6.10.2 读取可配置输入信号 .....   | 194 |
| 3.6.10.3 设置可配置输出信号 .....   | 195 |
| 3.6.10.4 读取可配置输出信号 .....   | 196 |
| 3.6.11 示教器控制 .....         | 197 |
| 3.6.11.1 设置是否启用示教器 .....   | 197 |
| 3.6.11.2 读取当前是否启用示教器 ..... | 197 |
| 3.6.12 IO 变量 .....         | 198 |
| 3.6.12.1 设置数字输入信号配置 .....  | 198 |
| 3.6.12.2 读取数字输入信号配置 .....  | 199 |
| 3.6.12.3 设置数字输出信号配置 .....  | 200 |
| 3.6.12.4 读取数字输出信号配置 .....  | 201 |
| 3.6.12.5 设置工具 IO 类型 .....  | 202 |
| 3.6.12.6 读取工具 IO 类型 .....  | 203 |
| 3.6.12.7 设置模拟量输入信号类型 ..... | 203 |
| 3.6.12.8 读取模拟量输入信号类型 ..... | 204 |
| 3.6.12.9 设置模拟量输出信号类型 ..... | 205 |

|                                  |     |
|----------------------------------|-----|
| 3.6.12.10 读取模拟量输出信号类型 .....      | 206 |
| 3.6.12.11 设置工具电压输出值 .....        | 207 |
| 3.6.12.12 读取工具电压输出值 .....        | 208 |
| 3.6.12.13 设置编码器配置 .....          | 208 |
| 3.6.12.14 读取编码器配置 .....          | 209 |
| 3.6.13 通讯变量 .....                | 210 |
| 3.6.13.1 设置位寄存器配置 .....          | 210 |
| 3.6.13.2 读取位寄存器配置 .....          | 211 |
| 3.6.13.3 读取所有位寄存器配置 .....        | 212 |
| 3.6.13.4 设置 16 位整数寄存器变量名 .....   | 213 |
| 3.6.13.5 读取 16 位整数寄存器变量名 .....   | 214 |
| 3.6.13.6 读取所有 16 位整数寄存器变量名 ..... | 214 |
| 3.6.13.7 设置 32 位整数寄存器变量名 .....   | 215 |
| 3.6.13.8 读取 32 位整数寄存器变量名 .....   | 216 |
| 3.6.13.9 读取所有 32 位整数寄存器变量名 ..... | 217 |
| 3.6.13.10 设置浮点寄存器变量名 .....       | 218 |
| 3.6.13.11 读取浮点寄存器变量名 .....       | 218 |
| 3.6.13.12 读取所有浮点寄存器变量名 .....     | 219 |
| 3.6.14 安装变量 .....                | 220 |
| 3.6.14.1 读取安装变量个数 .....          | 220 |
| 3.6.14.2 读取安装变量数据 .....          | 221 |
| 3.6.14.3 增加安装变量 .....            | 222 |
| 3.6.14.4 删除安装变量 .....            | 222 |
| 3.6.14.5 修改安装变量 .....            | 223 |
| 3.6.15 坐标系 .....                 | 224 |
| 3.6.15.1 读取点坐标系个数 .....          | 224 |
| 3.6.15.2 读取点坐标系数据 .....          | 225 |
| 3.6.15.3 读取点坐标系数据(通过名称) .....    | 225 |
| 3.6.15.4 增加点坐标系 .....            | 226 |
| 3.6.15.5 删除点坐标系 .....            | 227 |
| 3.6.15.6 修改点坐标系数据 .....          | 228 |
| 3.6.15.7 读取线坐标系个数 .....          | 229 |
| 3.6.15.8 读取线坐标系数据 .....          | 230 |
| 3.6.15.9 读取线坐标系数据(通过名称) .....    | 231 |
| 3.6.15.10 增加线坐标系 .....           | 231 |
| 3.6.15.11 删除线坐标系 .....           | 233 |
| 3.6.15.12 修改线坐标系数据 .....         | 234 |
| 3.6.15.13 读取面坐标系个数 .....         | 236 |
| 3.6.15.14 读取面坐标系数据 .....         | 236 |
| 3.6.15.15 读取面坐标系数据(通过名称) .....   | 237 |
| 3.6.15.16 增加面坐标系 .....           | 238 |

|                            |     |
|----------------------------|-----|
| 3.6.15.17 删除面坐标系 .....     | 240 |
| 3.6.15.18 修改面坐标系数据 .....   | 241 |
| 3.6.15.19 读取基坐标系数据 .....   | 243 |
| 3.6.15.20 读取工具坐标系数据 .....  | 243 |
| 3.6.15.21 计算点坐标系 .....     | 244 |
| 3.6.15.22 计算线坐标系 .....     | 245 |
| 3.6.15.23 计算面坐标系 .....     | 246 |
| 3.6.16 串口通讯 .....          | 247 |
| 3.6.16.1 设置串口配置 .....      | 247 |
| 3.6.16.2 读取串口配置 .....      | 248 |
| 3.6.17 网络 .....            | 249 |
| 3.6.17.1 设置控制柜网络配置 .....   | 249 |
| 3.6.17.2 读取控制柜网络配置 .....   | 250 |
| 3.6.17.3 设置控制柜从站配置 .....   | 251 |
| 3.6.17.4 读取控制柜从站配置 .....   | 251 |
| 3.6.18 modbus 主站 .....     | 252 |
| 3.6.18.1 读取从站个数 .....      | 252 |
| 3.6.18.2 读取从站数据 .....      | 253 |
| 3.6.18.3 添加从站 .....        | 254 |
| 3.6.18.4 删除从站 .....        | 255 |
| 3.6.18.5 修改从站数据 .....      | 255 |
| 3.6.18.6 从站连接/断开 .....     | 256 |
| 3.6.18.7 获取从站连接状态 .....    | 257 |
| 3.6.18.8 读取地址映射数量 .....    | 258 |
| 3.6.18.9 读取地址映射数据 .....    | 259 |
| 3.6.18.10 添加地址映射 .....     | 259 |
| 3.6.18.11 删除地址映射 .....     | 260 |
| 3.6.18.12 修改地址映射数据 .....   | 261 |
| 3.6.19 力控 .....            | 262 |
| 3.6.19.1 设置力控配置 .....      | 262 |
| 3.6.19.2 读力控配置 .....       | 264 |
| 3.6.19.3 开启力控并下发相关设置 ..... | 264 |
| 3.6.19.4 关闭力控 .....        | 265 |
| 3.6.19.5 设置力控控制参数 .....    | 266 |
| 3.6.19.6 读力控控制参数 .....     | 267 |
| 3.6.19.7 读力控数据 .....       | 268 |
| 3.6.20 扩展轴 .....           | 268 |
| 3.6.20.1 设置扩展轴使能状态 .....   | 268 |
| 3.6.20.2 读取扩展轴使能状态 .....   | 269 |
| 3.6.20.3 读取机械臂和扩展轴数据 ..... | 270 |
| 3.6.20.4 设置扩展轴配置 .....     | 271 |

|                        |     |
|------------------------|-----|
| 3.6.20.5 获取扩展轴配置 ..... | 272 |
| 3.6.20.6 增加扩展轴 .....   | 272 |
| 3.6.20.7 删除扩展轴 .....   | 273 |
| 四、综合示例 .....           | 274 |
| 4.1 机械臂使能 .....        | 274 |
| 4.2 工程控制 .....         | 276 |
| 4.3 移动控制 .....         | 278 |
| 4.4 轨迹再现 .....         | 282 |
| 4.5 安装变量操作 .....       | 285 |
| 4.6 点坐标系操作 .....       | 287 |
| 4.7 输出弹窗 .....         | 290 |
| 4.8 数学变量 .....         | 292 |
| 4.9 TCP 操作 .....       | 294 |
| 附录 A 速度和加速度范围 .....    | 297 |

# 一、SDK 介绍

CGXi 的 CR SDK 是基于网络通信协议 TCP/IP 实现的，提供了用于操作机械臂的接口，支持 python，C/C++，C#三种语言。

本文是基于 C/C++开发，提供操作机械臂的接口。该 SDK 可以实现多机械臂连接（在连接过程中句柄具有唯一性，与机械臂一一对应），用户可根据具体需求二次开发。

## 二、数据结构

方便接口函数的理解，特将一些常用的数据结构做详细的解释：接口中的长度单位统一为毫米（mm），角度单位统一为度（°）。

### 2.1 机械臂句柄

```
typedef int RobotHandle;
```

### 2.2 接口调用返回值

```
typedef enum CRresult
{
    sucess = 0,           ///<成功；分别有以下 3 种：连接、获取参数、写入参数
    success = 0,          ///<成功；分别有以下 3 种：连接、获取参数、写入参数
    error = 1,            ///<一般错误
    thread_running = 2,   ///<通讯连接重复建立
    operate_timeout = 3,  ///<操作超时，无返回值；注：处理超时导致无结果输出
    result_invalid = 4,   ///<返回结果无效；原因：rpc 获取结果为空
    out_of_range = 5,     ///<输入数值越界；需输入在范围内的值
    mutex_invalid = 6,    ///<内部错误
    para_error = 7,       ///<参数无效；需检查输入参数是否为空
    no_result = 8,        ///<无对应数值
    no_assignTCPindex = 9, ///<未找到指定索引号 TCP 偏移量
    no_handle = 10,       ///<未创建相关句柄
    handle_repeat = 11,   ///<重复建立相同的句柄
    repeat_name = 12,     ///<重命名
    delete_invalid = 13, ///<删除唯一的 TCP 或者负载
    set_bit_reg_invalid = 14, ///<同时设置位寄存器输入与输出
    repeat_id = 15,       ///
```

```

move_error = 18,          ///<移动阻塞运动时，返回的移动状态错误
forcectrl_wrongpara = 19,
///<力控 SDK 参数设置错误：包括参数长度错误、payload 为负数
forcectrl_invalldcmd = 20,
///<力控 SDK 无效的指令：包括 forceEna 等于 1,0 以外的值；forcebasetype 除 0 以
外的值；forcectrltype 除 1,0 以外的值；auto/manual 1,0 以外的值
forcectrl_parablocked = 21,
///<力控 SDK freedrive 模式下部分参数失效被屏蔽
forcectrl_cmdblocked = 22,
///<力控 SDK 机器人运动时或已在力控模式下时禁止再打开力控
forcectrl_configblocked = 23,
///<力控 SDK 机器人运动时或已在力控模式下时禁止下发配置
forcectrl_notenable = 24,          ///<力控 SDK 机器人使能之前不能打开传感器
forcectrl_wrongsensor = 25,        ///<力控 SDK 传感器型号选择错误
outputrule_error = 26,             ///<未设置系统输出，禁止启用自复位
analoginputtype_error = 27
///<模拟量输入类型错误：注：工具端模拟量输入类型只有电压
}CResult;
```

## 2.3 坐标系选择

```

typedef enum CoordinateType
{
    baseCoordinate = 0,          ///<基坐标系
    jointCoordinate = 1,         ///<关节坐标系
    PointCoordinate = 2,         ///<点坐标系
    LineCoordinate = 3,          ///<线坐标系
    PlaneCoordinate = 4,         ///<面坐标系
    ToolBaseCoordinate = 5,      ///<工具坐标系
    visionCoordinate = 6         ///<视角坐标系
}CoordinateType;
```

## 2.4 点转换类型

```

typedef enum PointTransType
{
    pointTransStop = 0,          ///<过渡点停止
    pointTransConstantSpeed = 1, ///<匀速过渡
    pointTransSmooth = 2         ///<柔顺过渡（不匀速，不停止）
}PointTransType;
```



## 2.5 位姿变换

```
typedef enum PoseTranType          ///<预留参数
{
    poseTranMoveToTargetPose = 0,
    poseTranKeepStartPose = 1
}PoseTranType;
```

## 2.6 运动触发模式

```
typedef enum MotiontriggerMode
{
    MovetriggerbySafetySignal = 0,          ///<安全同步信号触发
    MovetriggerbyOnlyRpc = 1,              ///<指令直接触发
    MovetriggerbyMoveCache = 2             ///<移动指令会进入缓存区
}MotiontriggerMode;
```

## 2.7 TCP 偏移量

```
typedef struct TCPMsg
{
    char tcpName[20];                      ///

```

## 2.8 运动参数

```
typedef struct PointControlPara
{
    double pose[ROB_AXIS_NUM];
    ///<坐标点向量: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
    double jointpos[ROB_AXIS_NUM];
    ///<关节角度: 关节 1~6 (单位: °)
    double tcpOffset[ROB_AXIS_NUM];
    ///

```

```

    ///

```

## 2.9 简化版运动参数

```

typedef struct PointControlParaSimple
{
    double pose[ROB_AXIS_NUM];
    ///<坐标点向量: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
    double jointpos[ROB_AXIS_NUM];
    ///<关节角度:关节 1~6 (单位: °)
    double tcpOffset[ROB_AXIS_NUM];
    ///

```

## 2.10 圆弧运动参数

```
typedef struct PointControlParaList
{
    PointControlPara pointcontrolpara[MOVECPOINT_NUM];    ///<过渡点和终点
    int fixedrot;
    ///<方向模式 0-无约束 1-固定
    double centralangle;    ///<圆弧角，单位：°
}PointControlParaList;
```

## 2.11 运动模式

```
typedef enum MoveType
{
    DecStop = 0,    ///<当前运动减速停止
    ImdStop = 1,    ///<当前运动立即停止
    Run = 2    ///<运动
}MoveType;
```

## 2.12 关节状态

```
typedef enum JointModes
{
    JOINT_MODE_POWER_OFF = 1,    ///<断电
    JOINT_MODE_BOOTING = 2,    ///<机械臂上电至空闲状态下
    JOINT_MODE_IDLE = 3,
    ///<上电完成，空闲状态（ethercat——>op）
    JOINT_MODE_BACK_DRIVE = 4,    ///<反向驱动状态
    JOINT_MODE_RELEASE_BRAKE = 5,    ///<机械臂使能，松抱闸过程中
    JOINT_MODE_CSP_STOP = 7,    ///<关节使能静止(位置模式)
    JOINT_MODE_CSP_MOVING = 8,    ///<关节使能运动(位置模式)
    JOINT_MODE_CST_STOP = 9,    ///<关节使能静止(转矩模式)
    JOINT_MODE_CST_MOVING = 10,    ///<关节使能运动(转矩模式)
    JOINT_MODE_FAULT = 11,    ///<错误状态
    JOINT_MODE_READY_FOR_POWER_OFF = 12    ///<准备断电
}JointModes;
```

## 2.13 机械臂状态

```
typedef enum RobotModes
{
    Closed = 0,    ///<关闭
```

```

Disconnect = 1,           ///<断开连接状态
ConfirmSafty = 2,         ///<安全确认状态
Booting = 3,              ///<控制器正在启动
ControlerIdle = 4,        ///<控制器启动完成，空闲状态
ControlerUpdataFirmWare = 5, ///<固件更新
JointPowerOff = 6,        ///<控制器启动完成，本体未上电
JointPowerOn = 7,         ///<机器人上电过程
JointIdle = 8,            ///<机器人上电完成，空闲
BackDrive = 9,            ///<反向驱动
ReleaseBrake = 10,        ///<机器人使能、松抱闸中
Enable = 11,              ///<机器人使能完成，并已完全松开抱闸
CloseBrake = 12,          ///<机器人断使能、关抱闸中
Jog = 100,                ///<点动
Teach = 101,              ///<拖动示教
ForceControlTest = 102,   ///<力控测试
ProgramStop = 103,        ///<使能完成，程序停止
ProgramPause = 104,       ///<使能完成，程序暂停
ProgramStopping = 105,    ///<使能完成，程序和运动正在停止
ProgramPauseing = 106,    ///<使能完成，程序和运动正在暂停
ProgramRun_MotionStop = 107, ///<程序运行，运动停止
ProgramRun_MotionReducing = 108, ///<程序运行，运动减速中
ProgramRun_MotionMoving = 109, ///<程序运行
ProgramRun_MotionCanBlend = 110, ///<程序运行，机器人进入到可交融区域
Imdstop = 111,            ///<紧急停止
ProtectiveStop = 112      ///<防护停止
}RobotModes;

```

## 2.14 安全模式

```
typedef enum SafetyModes
```

```

{
    SAFETY_MODE_UNDEFINED_SAFETY_MODE = 1,    ///<未定义
    SAFETY_MODE_VALIDATE_JOINT_ID = 2,         ///<验证关节的 ID
    SAFETY_MODE_FAULT = 3,                    ///<安全错误，其他错误出现一段时间后，最终保持的错误状态
    SAFETY_MODE_VIOLATION = 4,                 ///<在安全平面处越界
    SAFETY_MODE_ROBOT_EMERGENCY_STOP = 5,      ///<急停按钮按下
    SAFETY_MODE_SYSTEM_EMERGENCY_STOP = 6,     ///<可配置 IO 输入任意两路设置为紧急停机，断开后出现的错误状态
    SAFETY_MODE_SAFEGUARD_STOP = 7,            ///<安全 IO 断开
    SAFETY_MODE_RECOVERY = 8,

```

```

    ///<在违反安全限制后，机器人停止，重启后，进入恢复模式
    SAFETY_MODE_PROTECTIVE_STOP = 9,
    ///<保护性停止，运动过程中，外部强力推动机械手
    SAFETY_MODE_REDUCED = 10,           ///<限制模式
    SAFETY_MODE_NORMAL = 11           ///<正常运行模式
} SafetyModes;

```

## 2.15 程序状态

```

typedef enum ProgramState
{
    PROGRAM_STATE_STOP = 0,           ///<程序停止
    PROGRAM_STATE_PAUSE = 1,         ///<程序暂停
    PROGRAM_STATE_RUN = 2             ///<程序运行
} ProgramState;

```

## 2.16 弹窗信息

```

typedef struct PopUpMsg
{
    int popupType;                    ///<弹窗类型，0-输出弹窗，1-输入弹窗
    char var_data[1024];              ///<弹窗显示的字符串，字符串类型
    char var_title[128];              ///<弹窗标题，字符串类型
    BOOL iswarning;                   ///<是否是警告信息，bool 类型
    BOOL iserror;                     ///<是否是报警信息，bool 类型
    BOOL isblocking;                  ///<是否暂停程序运行，bool 类型
    enum VariableType inVarType;      ///<输入弹窗时，变量类型
    double inVarMinValue;              ///<输入弹窗时，变量数值范围最小值
    double inVarMaxValue;             ///<输入弹窗时，变量数值范围最大值
} PopUpMsg;

```

## 2.17 版本信息

```

typedef struct Version
{
    unsigned char deviceType[20];      ///<机器人类型
    unsigned char versionNo[20];       ///<版本信息
    int bugfix;                        ///

```

## 2.18 Lua 变量

```
typedef enum VariableType
{
    LUA_TNIL = 0,           ///<空
    LUA_TBOOLEAN = 1,      ///<布尔
    LUA_TLIGHTUSERDATA = 2, ///<预留
    LUA_TNUMBER = 3,       ///<数字
    LUA_TSTRING = 4,       ///<字符串
    LUA_TTABLE = 5,        ///<表
    LUA_TFUNCTION = 6,     ///<函数
    LUA_TUSERDATA = 7,     ///<自定义类型
    LUA_TTHREAD = 8,       ///<线程
    LUA_NUMTAGS = 9        ///<预留
} VariableType;
```

## 2.19 变量元素

```
typedef struct VarItemData
{
    enum VariableType varItemType;    ///<数据类型
    BOOL boolValue;                  ///

```

## 2.20 变量信息

```
typedef struct VariableMsg
{
    char variableName[CR6_INSTALL_VAR_NAME_LENGTH]; ///<变量名
    enum VariableType variableType;                ///<数据类型
    int variableID;                                ///<变量 id
    BOOL boolValue;                                ///

```

## 2.21 消息来源

```
typedef enum CommMessageSource
{
    Base = 0,                ///<基座关节（关节 1）
    Shoulder = 1,            ///<肩部关节（关节 2）
    Elbow = 2,               ///<肘部关节（关节 3）
    Wrist1 = 3,             ///<腕关节 1（关节 4）
    Wrist2 = 4,             ///<腕关节 2（关节 5）
    Wrist3 = 5,             ///<腕关节 3（关节 6）
    Tool = 6,               ///<工具模块
    Controller = 100,       ///<主控制器
    SafetyProcessorB = 101,  ///<协处理器
    TeachPendant = 200      ///<示教器
}CommMessageSource;
```

## 2.22 警告等级

```
typedef enum WarningLevel
{
    MESSAGE_WARNING_LEVEL_INFO = 1,        ///<正常消息
    MESSAGE_WARNING_LEVEL_WARNING = 2,      ///<警告不停机
    MESSAGE_WARNING_LEVEL_VIOLATION = 3,    ///<违反限制停机
    MESSAGE_WARNING_LEVEL_FAULT = 4,        ///<系统故障，紧急停止
    MESSAGE_WARNING_LEVEL_USER = 5         ///<用户打印输出
}WarningLevel;
```

## 2.23 世界时间

```
typedef struct WorldTime
{
    unsigned int year;        ///<年
    unsigned int month;       ///<月
    unsigned int day;         ///<日
    unsigned int week;        ///<周
    unsigned int hour;        ///<时
    unsigned int minute;      ///<分
    unsigned int second;      ///<秒
    unsigned int milliSecond; ///<毫秒
}WorldTime;
```

## 2.24 日志信息

```
typedef struct RobotLogMsg
{
    unsigned int robotMessageCode;           ///<消息代码
    unsigned int robotMessageType;           ///<消息类型，预留
    enum CommMessageSource source;           ///<消息来源
    char textMessage[100];                   ///<消息信息
    WorldTime worldTime;                     ///<时间戳
    enum WarningLevel warningLevel;          ///<警告等级
    RobotModes robotMode;                    ///<机器人状态
    unsigned long long timeStamp_us;         ///<精确时间戳
    unsigned long long messageIndex;         ///<消息序号
} RobotLogMsg;
```

## 2.25 版本信息

```
typedef struct SysVersion
{
    Version sjqVersion;                       ///<示教器版本
    Version kzqVersion;                       ///<控制器版本
    Version jpmVersion;                       ///<本体放电板版本
    Version saftyCtrlVersion;                 ///<安全板版本
    Version jointVersion[ROB_AXIS_NUM];      ///<关节版本
    Version etfVersion;                       ///<工具法兰版本
} SysVersion;
```

## 2.26 负载

```
typedef struct PayLoad
{
    char payloadName[20];                     ///<负载名称
    double toolPayload;                       ///<负载重量：kg
    double centerOfGravity[3];                ///<负载中心（CX，CY，CZ）（单位：mm）
} PayLoad;
```

## 2.27 校验码检查

```
typedef enum ParityCheck
{
    no = 0,                                   ///<无校验
    odd = 1,                                  ///<奇校验
}
```



```
        even = 2;                                     ///<偶校验
    }ParityCheck;
```

## 2.28 串口通信

```
typedef enum SerialCommuType
{
    FreeComm = 0,                                     ///<自由格式
    Modbus_RTU = 1,                                   ///
```

## 2.29 波特率

```
typedef enum BaudRate
{
    BaudRate300 = 0,                                  ///<波特率为 300Baud
    BaudRate600 = 1,                                  ///<波特率为 600Baud
    BaudRate1200 = 2,                                  ///<波特率为 1200Baud
    BaudRate2400 = 3,                                  ///<波特率为 2400Baud
    BaudRate4800 = 4,                                  ///<波特率为 4800Baud
    BaudRate9600 = 5,                                  ///<波特率为 9600Baud
    BaudRate19200 = 6,                                  ///<波特率为 19200Baud
    BaudRate28800 = 7,                                  ///<波特率为 28800Baud
    BaudRate38400 = 8,                                  ///<波特率为 38400Baud
    BaudRate57600 = 9,                                  ///<波特率为 57600Baud
    BaudRate115200 = 10,                                  ///<波特率为 115200Baud
    BaudRate192000 = 11,                                  ///<波特率为 192000Baud
    BaudRate256000 = 12,                                  ///<波特率为 256000Baud
    BaudRate288000 = 13,                                  ///<波特率为 288000Baud
    BaudRate384000 = 14,                                  ///<波特率为 384000Baud
    BaudRate512000 = 15,                                  ///<波特率为 512000Baud
    BaudRate576000 = 16,                                  ///<波特率为 576000Baud
    BaudRate768000 = 17,                                  ///<波特率为 768000Baud
    BaudRate1000000 = 18,                                  ///<波特率为 1000000Baud
    BaudRate1200000 = 19,                                  ///<波特率为 1200000Baud
    BaudRate1500000 = 20,                                  ///<波特率为 1500000Baud
    BaudRate2400000 = 21,                                  ///<波特率为 2400000Baud
    BaudRate3000000 = 22,                                  ///<波特率为 3000000Baud
}BaudRate;
```

## 2.30 串口配置

```
typedef struct SerialCommSettings
{
    enum BaudRate baudRate;           ///<波特率
    enum ParityCheck parity;          ///<奇偶校验
    int dataBits;                     ///<数据位
    double stopBits;                  ///<停止位
    int retryNumber;                   ///<重试次数
    int timeout;                       ///<超时时间，单位：ms
    enum SerialCommType serialCommType; ///<串口通信类型
    int modbusSlaveNo;                 ///

```

## 2.31 IO 类型

```
typedef enum VarDigitalIOType          ///

```

## 2.32 安装角度

```
typedef struct InstallAngle
{
    double baseAngle;                  ///<基座旋转角度，单位：°
    double tiltAngle;                  ///<基座倾斜角，单位：°
} InstallAngle;
```

## 2.33 系统功能输入

```
typedef enum InputAction
{
    InputAction_NONE = 0,              ///<无动作
    InputAction_RobotON = 1,           ///<本体上电使能
    InputAction_MovetoHome = 2,        ///<回初始点
    InputAction_RunProgram = 3,        ///<运行程序
}
```

```

    InputAction_SuspendedProgram = 4,    ///<暂停程序
    InputAction_StopProgram = 5,         ///<停止程序
    InputAction_FreeDrive = 6,           ///<自由拖动
    InputAction_DownEnable = 7,          ///<下使能
    InputAction_ClearFault = 8           ///<清除故障
}InputAction;

```

## 2.34 系统功能输出

```

typedef enum OutputAction
{
    OutputAction_NONE = 0,                ///<无动作
    OutputAction_RobotON_HIGH = 1,         ///<机器人使能时为高
    OutputAction_RobotON_LOW = 2,          ///<机器人使能时为低
    OutputAction_RobotPowerOn_HIGH = 3,    ///<已上电时为高
    OutputAction_RobotPowerOn_LOW = 4,     ///<已上电时为低
    OutputAction_PowerOn_HIGH = 5,         ///<已开机时为高
    OutputAction_PowerOn_LOW = 6,          ///<已开机时为低
    OutputAction_WarnErr_HIGH = 7,         ///<有系统错误时为高
    OutputAction_WarnErr_LOW = 8,          ///<有系统错误时为低
    OutputAction_ProgramStatus_Stop_HIGH = 10, ///<程序停止时为高
    OutputAction_ProgramStatus_Stop_LOW = 11, ///<程序停止时为低
    OutputAction_Fault_Stop_HIGH = 12,     ///<异常停止时为高
    OutputAction_Fault_Stop_LOW = 13,      ///<异常停止时为低
    OutputAction_ProgramStatus_Pause_HIGH = 14, ///<程序暂停输出为高
    OutputAction_ProgramStatus_Pause_LOW = 15, ///<程序暂停输出为低
    OutputAction_MotionStatus_Moving_HIGH = 20, ///<运动时为高
    OutputAction_MotionStatus_Moving_LOW = 21, ///<运动时为低
    OutputAction_MovetoHome_HIGH = 22     ///<回初始位到位信号，到位时为高
}OutputAction;

```

## 2.35 IO 选项卡输出

```

typedef enum OutputOptions    ///

```

## 2.36 输出信号状态（自动复位）

```
typedef enum OutputRule
{
    OutputRule_Disable = 0,          ///<禁用
    OutputRule_Enable = 1            ///<启用
}OutputRule;
```

## 2.37 输入信号

```
typedef struct IOInputConfiguration
{
    int inputFilteringTime;           ///<输入信号滤波时间，单位:ms
    char inputName[NAMELENTH];       ///<输入信号名称
    InputAction inputActions;         ///<输入信号操作
}IOInputConfiguration;
```

## 2.38 输出信号

```
typedef struct IOOutputConfiguration
{
    char outputName[NAMELENTH];      ///<输出信号名称
    OutputAction outputActions;       ///<输出信号条件
    OutputOptions outputOptions;      ///<输出信号操作控制选项
    OutputRule outputRule;            ///<输出信号状态（自动复位）
}IOOutputConfiguration;
```

## 2.39 工具 IO 类型

```
typedef enum ToolIoType
{
    IO_OUTPUT = 0,                   ///<输出
    IO_INPUT = 1                     ///<输入
}ToolIoType;
```

## 2.40 模拟量类型

```
typedef enum AnalogType
{
    Current = 0,                     ///<电流
    Voltage = 1                       ///<电压
}AnalogType;
```

## 2.41 模拟量输入

```
typedef struct AnalogInputConfiguration
{
    char analogInputName[NAMELENGTH];    ///<模拟量输入通道名称
    AnalogType analogInputType;          ///<模拟量输入类型
}AnalogInputConfiguration;
```

## 2.42 模拟量输出

```
typedef struct AnalogOutputConfiguration
{
    char analogOutputName[NAMELENGTH];    ///<模拟量输出通道名称
    AnalogType analogOutputType;          ///<模拟量输出类型
}AnalogOutputConfiguration;
```

## 2.43 工具电压

```
typedef enum ToolPower
{
    Power_invalld = 0,                    ///<无效值
    Power_on = 1,                         ///<<24V
    Power_off = 2,                        ///<<0V
}ToolPower;
```

## 2.44 编码器配置

```
typedef struct EncoderConfiguration
{
    int eqepType;                        ///<类型
    int eqepCountMode;                  ///<模式
    int strobeInputMode;                ///<探针
    int eqepEnabled;                    ///<控制
}EncoderConfiguration;
```

## 2.45 BOOL 寄存器

```
typedef struct BitRegisterConfiguration
{
    char GeneralPurposeBOOLeanRegisterName[NAMELENGTH];
    ///
```

```

    ///<BOOL 型寄存器系统功能输入
    OutputAction GeneralPurposeBOOLeanRegisterOutputActionss;
    ///<BOOL 型寄存器系统功能输出
    OutputRule GeneralPurposeBOOLeanRegisterOutputRule;
    ///<BOOL 型寄存器自复位
}BitRegisterConfiguration;

```

## 2.46 寄存器

```

typedef struct RegisterConfiguration
{
    char GeneralPurposeRegisterName[NAMELENTH];    ///<寄存器名称
}RegisterConfiguration;

```

## 2.47 工具端信息

```

typedef struct PoseMessage
{
    double toolPosition[ROB_AXIS_NUM];
    ///<位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
    double toolAxisAngle[ROB_AXIS_NUM];    ///<关节角度: 关节 1~6 (单位: °)
}PoseMessage;

```

## 2.48 点坐标系

```

typedef struct PointCSNode
{
    BOOL isValid;    ///<是否有效
    int id;    ///<id 号
    char name[NAMELENTH];    ///<点坐标系名字
    PoseMessage point;    ///<点坐标系数据
}PointCSNode;    ///<点坐标系

```

## 2.49 线坐标系

```

typedef struct LineCSNode
{
    BOOL isValid;    ///<是否有效
    int id;    ///<id 号
    char name[NAMELENTH];    ///<线坐标系名字
    PointCSNode firstPoint;    ///<第一个点的数据相关信息
    PointCSNode secondPoint;    ///<第二个点的数据相关信息
}

```

```

double coordinatePose[ROB_AXIS_NUM];
///<线坐标系位姿数据: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
double coordinateJointPos[ROB_AXIS_NUM];
///<线坐标系关节数据: 关节 1~6 (单位: °)
}LineCSNode; ///<线坐标系

```

## 2.50 面坐标系

```

typedef struct PlaneCSNode
{
    BOOL isValid;           ///<是否有效
    int id;                 ///<id 号
    char name[NAMELENGTH];  ///<面坐标系名字
    PointCSNode firstPoint;  ///<第一个点的数据相关信息
    PointCSNode secondPoint; ///<第二个点的数据相关信息
    PointCSNode thirdPoint;  ///<第三个点的数据相关信息
    double coordinatePose[ROB_AXIS_NUM];
    ///<面坐标系位姿数据: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
    double coordinateJointPos[ROB_AXIS_NUM];
    ///<面坐标系关节数据: 关节 1~6 (单位: °)
}PlaneCSNode; ///<面坐标系

```

## 2.51 机器人限值等级

```

typedef enum SafetyLimitsValuesType
{
    Userdefined = 0,           ///<自定义
    LimitLevel_1 = 1,          ///<等级 1 (限制最多)
    LimitLevel_2 = 2,          ///<等级 2 (限制偏多)
    LimitLevel_3 = 3,          ///<等级 3 (限制偏少)
    LimitLevel_4 = 4           ///<等级 4 (限制最少)
}SafetyLimitsValuesType; ///<安全限制等级

```

## 2.52 碰撞后处理模式

```

typedef enum SafetyCollisionHandleMode
{
    Collision_EnterReboundMode = 0,  ///<进入反弹模式
    Collision_ProgramPause = 1,      ///<暂停
    Collision_ProgramStop = 2        ///<停止
}SafetyCollisionHandleMode;

```

## 2.53 关节限值模式

```
typedef enum LimitsBoundaryPlaneMode
{
    Disabled = 0,           ///<禁用
    Normal = 1,             ///<正常
    Reduced = 2,            ///<缩减
    Both = 3,               ///<二者都是
    Trigger = 4             ///<触发器缩减模式
}LimitsBoundaryPlaneMode;
```

## 2.54 安全限值模式

```
typedef enum SafetyLimitsMode
{
    invaild = 0,            ///<禁用
    mode_Normal = 1,        ///<正常
    mode_Reduced = 2        ///<缩减
}SafetyLimitsMode;
```

## 2.55 安全限值

```
typedef struct SafetyLimitsValues
{
    double maxTcpSpeed;      ///

```

## 2.56 关节限值

```
typedef struct SafetyLimitsJointAngle
{
    double maxJointSpeed;    ///<最大角速度,单位：°/s
    double minJointPosition; ///<最小位置，单位：°
}
```



```
double maxJointPosition;          ///<最大位置，单位：°
} SafetyLimitsJointAngle;
```

## 2.57 安全平面

```
typedef struct SafetyLimitsBoundaryPlane
{
    char name[NAMELENTH];          ///<安全平面名字
    int id;                        ///

```

## 2.58 安全 IO 输入信号

```
typedef enum SafetyIOInput
{
    input_invalld = 0,             ///<未分配
    emergencyStop = 1,             ///<紧急停机
    reducedMode = 2,               ///<缩减模式
    safeguardStop = 3,             ///<安全停止
    safeguardReset = 6,            ///<安全重置
} SafetyIOInput;
```

## 2.59 安全 IO 输出信号

```
typedef enum SafetyIOOutput
{
    output_invalld = 0,            ///<未分配
    systemEmergencyStop = 1,       ///<紧急停止输出
    robotMoving = 2,               ///<机器人运动
    robotNotStopping = 3,          ///<机器人未停止
} SafetyIOOutput;
```

## 2.60 控制柜网络配置

```
typedef struct IPConfig
```

```
{
    int ipAddr[4];           ///
```

## 2.61 MODBUS 地址映射功能

```
typedef enum ModbusMasterFunctionNo
{
    ReadBool = 0,           ///<读位寄存器
    WriteBool = 1,          ///<写位寄存器
    ReadInt16 = 2,          ///<读 16 位整数寄存器
    WriteInt16 = 3,         ///<写 16 位整数寄存器
    ReadInt32 = 4,          ///<读 32 位整数寄存器
    WriteInt32 = 5,         ///<写 32 位整数寄存器
    ReadFloat = 6,          ///<读浮点寄存器
    WriteFloat = 7,         ///<写浮点寄存器
    ReadWriteBool = 8,      ///<预留
    ReadWriteInt16 = 9,     ///<预留
    ReadWriteInt32 = 10,    ///<预留
    ReadWriteFloat = 11,    ///<预留
}ModbusMasterFunctionNo;  ///
```

## 2.62 MODBUS 地址映射

```
typedef struct ModbusAddrMap
{
    ModbusMasterFunctionNo functionNo;  ///
```

## 2.63 MODBUS 连接状态

```
typedef enum ModbusMasterOperate
{
    Start = 0,           ///<开始连接
    Stop = 1,            ///<断开连接
}ModbusMasterOperate;  ///
```

## 2.64 MODBUS 参数配置

```
typedef struct ModbusMasterConfig
{
    int id; //<从站 id 号
    int slaveNo; //<从站站号
    int slaveIP[4]; //<modbus-tcp 时，从站 ip
    int scanTime; //<扫描周期
    ModbusMasterOperate modbusMasterOperate; //<modbus 连接状态
    int autoConnect; //<自动连接
}ModbusMasterConfig; //<modbusMaster 配置
```

## 2.65 通讯变量选择

```
typedef enum CommuVarType
{
    CommuVarType_Bool = 0, //<bool 类型
    CommuVarType_Int16 = 1, //<16bit 整数类型
    CommuVarType_Int32 = 2, //<32bit 整数类型
    CommuVarType_Float = 3, //<32bit 浮点类型
}CommuVarType;
```

## 2.66 脚本程序状态

```
typedef enum Lua_ScriptStatus
{
    lua_Script_NoneOP = 0, //<脚本为空
    lua_Script_stop = 1, //<脚本程序停止
    lua_Script_pause = 2, //<脚本程序暂停
    lua_Script_run = 3, //<脚本程序运行
    lua_Script_load = 4, //<脚本程序正在加载
}Lua_ScriptStatus;
```

## 2.67 串口类型

```
typedef enum SerialType
{
    rs485_tool_CommSettings = 0, //<工具 485
    rs485_1_CommSettings = 1, //<485-1
    rs485_2_CommSettings = 2, //<485-2
    rs232_CommSettings = 3, //<rs232
    ethernet_CommSettings = 4, //<以太网
}
```

```
}SerialType;
```

## 2.68 轨迹点数据

```
typedef struct PathPoint
{
    double pose[ROB_AXIS_NUM];
    ///<坐标点向量: x,y,z (单位: mm); Rx,Ry,Rz (单位: °)
    double jointpos[ROB_AXIS_NUM];    ///<关节角度: 关节 1~6 (单位: °)
}PathPoint;
```

## 2.69 轨迹参数

```
typedef struct PathPara
{
    int index;                ///<轨迹索引 下载时: 0~49
    int moveType;             ///<运动轨迹类型, 0-透传, 1-样条
    double speed;             ///<速度 moveType 为 3 时有效
    double acc;               ///<加速度 moveType 为 3 时有效
    double blendRadius;       ///<过渡半径 moveType 为 3 时有效
}PathPara;
```

## 2.70 轨迹数据

```
typedef struct PathData
{
    PathPoint *pathPoints;    ///<轨迹序列数据
    int pathPointsNum;        ///<轨迹数量
    int moveTime;             ///<运动时间 moveType 为 1 时有效, 2~10000, 单位: ms
}PathData;
```

## 2.71 下载轨迹数据

```
typedef struct PathDownloadData
{
    PathData pathData;        ///<轨迹数据
    PathPara pathPara;        ///<轨迹参数
}PathDownloadData;
```

## 2.72 轨迹记录参数

```
typedef struct RecordPathPara
{

```

```
int sampleTime;          ///<采样时间 recordControl 为 1 时有效, 单位: ms
int recordControl;       ///<记录控制, 0-停止, 1-启动, 2-暂停
}RecordPathPara;
```

## 2.73 轨迹记录状态

```
typedef struct PathRecordStatus
{
    int recordStatus;      ///<0-记录完成, 1-记录中, 2-暂停记录
    int waypointNumber;    ///<已记录点数
}PathRecordStatus;
```

## 2.74 轨迹运行状态

```
typedef struct PathRunMsg
{
    int pathrunstatus;     ///<0 或 10001-轨迹控制停止 1-轨迹运行中
    int pointIndex;        ///<当前运行轨迹索引
}PathRunMsg;
```

## 2.75 控制模式

```
typedef enum ControlModes
{
    CONTROL_MODE_POSITION = 0,      ///<控制器启动后的各种 RobotModes
    CONTROL_MODE_TEACH = 1,
    ///

```

## 2.76 力控设置

```
typedef struct ForceSetting
{
    int forceBaseType;
    ///<力控类型: 0-基于 TCP 力传感器,1-基于动力学,2-基于底座力传感器
    int forceCtrlType;             ///<力控用法: 0-运动力控,1-力控拖动
    int flexibleAxis[6];           ///<力控轴
}ForceSetting;
```

## 2.77 力控传感器 tcp

```
typedef struct ForceTCPMsg
{
    char forceTcpName[20];           ///<力控传感器 tcp 的名称
    int forceTcpId;                  ///<力控传感器 tcp 的 ID
    double forceTcpOffset[ROB_AXIS_NUM]; ///<力控传感器 tcp 偏移量
}ForceTCPMsg;
```

## 2.78 力控传感器负载

```
typedef struct ForcePayLoad
{
    char forcePayloadName[20];       ///<力控传感器负载名称
    int forcePayloadId;              ///<力控传感器负载 ID
    double forceToolPayload;         ///<力控传感器负载重量，0~10（单位：kg）
    double forceCenterOfGravity[3]; ///<负载中心（CX，CY，CZ）
}ForcePayLoad;
```

## 2.79 力控传感器信息

```
typedef struct SensorMessage
{
    int venderNo;           ///<力传感器厂家号：0-CGX,1-蓝点,2-鑫精诚
    int productNo;          ///<力传感器产品号
    int sequenceNo;         ///<力传感器通讯拓扑顺序号
}SensorMessage;
```

## 2.80 通讯配置

```
typedef struct CommuConfig
{
    int autoConnect;           ///<自动连接 不处理-0 连接-1 关闭-2
    int mannaulOperate;        ///<手动连接 不处理-0 连接-1 关闭-2
    int paraBuf;               ///<预留参数
}CommuConfig;
```

## 2.81 力控传感器工具设置

```
typedef struct ForceToolSetting
{
    char activeForceTCP_name[20];           ///<激活的力控 tcp 名称
}
```

```
char activeForcePayload_name[20];          ///<激活的力控负载名称
ForceTCPMsg availableForceTCP[20];         ///<可以使用的力控 TCP
int availableForceTCPLen;                  ///<可以使用的力控 TCP 数量
ForcePayLoad availableForcePayLoad[20];    ///<可以使用的力控负载
int availableForcePayLoadLen;               ///<可以使用的力控负载数量
}ForceToolSetting;
```

## 2.82 力控配置

```
typedef struct ForceConfig
{
    SensorMessage sensorMessage;            ///<传感器信息
    ForceToolSetting forceCtrlTcpImpl;       ///<力控工具设置
    CommuConfig commuConfig;                ///<力控通讯配置
}ForceConfig;
```

## 2.83 力控控制参数

```
typedef struct ForceCtlPara
{
    double taskFrame[6];                    ///<力控坐标系
    double wrench[6];                       ///<力给定
    double limits[6];                       ///<力控保护限制（范围：大于 0）
    int limitsLen;                           ///<力控保护限制长度 0 或 6
    double mass[6];                         ///<力控虚拟质量（范围：0.5~10）
    int massLen;                             ///<力控虚拟质量长度 0、1 或 6
    double damping[6];                      ///<力控虚拟阻尼（范围：0.1~10）
    int dampingLen;                         ///<力控虚拟阻尼长度 0、1 或 6
    double stiffness[6];                    ///<力控虚拟刚度（范围：0~10）
    int stiffnessLen;                       ///<力控虚拟刚度长度 0、1 或 6
}ForceCtlPara;
```

## 2.84 力控数据

```
typedef struct ForceData
{
    double ftRaw[6];                        ///<力传感器原始值
    double ftCtrl[6];                      ///<力传感器控制值
    ControlModes controlMode;              ///<控制模式
}ForceData;
```

## 2.85 机器人状态数据

```
typedef struct RobotStateData
{
    double tcpActualPose[ROB_AXIS_NUM];           ///<实际 tcp 位置
    double jointActualPos[ROB_AXIS_NUM];           ///<实际关节位置
    RobotModes robotMode;                           ///<机械臂当前状态
    BOOL robotMoveStatus;                           ///<机械臂运动状态
    unsigned int robotSpeedPercent;                 ///<机械臂速度百分比
    int configurableDigitalOutput[ROB_AXIS_NUM];    ///<控制器可配置数字输出
}RobotStateData;
```

## 2.86 机械臂和扩展轴数据

```
typedef struct RobotExjData
{
    double actualExjointPos[EXTERNAL_MAX_NUM];      ///<扩展轴实际位置
    double actualJointPos[ROB_AXIS_NUM];           ///<实际关节角度
    double actualTcpVector[ROB_AXIS_NUM];          ///<实际 TCP 位姿
}RobotExjData;
```

## 2.87 扩展轴关节移动参数

```
typedef struct MoveExjPara
{
    double pose[ROB_EXJ_AXIS_NUM];
    ///<前六个代表关节姿态 (x, y, z (单位: mm); Rx, Ry, Rz (单位: °)), 后
    六个代表扩展轴位置
    double jointPos[ROB_EXJ_AXIS_NUM];
    ///< 前六个代表关节 1~6 关节角度, 后六个代表扩展轴位置
    double speed[ROB_EXJ_AXIS_NUM];
    ///< 前六个代表机械臂速度, 当坐标系类型为关节坐标系时, 分别代表一到六关节
    的速度°/s, 其他时候代表 tcp 速度 mm/s, 第七个代表扩展轴直线速度 mm/s, 第八个代
    表扩展轴旋转速度°/s
    double acc[ROB_EXJ_AXIS_NUM];
    ///< 前六个代表关节加速度, 后六个无效
    ///< acc 可以是 tcp 加速度或者是关节加速度, 当 coordinateType 设置为
    jointCoordinate 时, 该速度为关节加速度, 其它情况是 tcp 加速度
    ///< 作为 tcp 加速度时, 单位为: mm/s²; 作为关节加速度时, 单位为: °/s²
    double tcpOffset[ROB_AXIS_NUM];
    ///< tcp 偏移量: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
    int tcpId;                                     ///< tcp 的 ID 号
```



```
enum CoordinateType coordinateType;          ///< 坐标系类型
double coordinatePose[ROB_AXIS_NUM];
///<参考坐标系位姿:x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
double pointTransRadius;                    ///<过渡半径
enum MotiontriggerMode motiontriggerMode;    ///<运动轨迹触发方式
}MoveExjPara;
```

## 2.88 扩展轴点动运动参数

```
typedef struct MoveJogExjPara
{
    double jointPos[EXTERNAL_MAX_NUM];        ///<扩展轴位置
    double speed[EXTERNAL_MAX_NUM];           ///<扩展轴速度
    double acc[EXTERNAL_MAX_NUM];             ///<扩展轴加速度
}MoveJogExjPara;
```

## 2.89 扩展轴运动参数

```
typedef struct ExjMovePara
{
    int exjCount;                             ///<扩展轴数量
    int index[EXTERNAL_MAX_NUM];              ///<扩展轴索引
    double speed[EXTERNAL_MAX_NUM];           ///<扩展轴速度
    double position[EXTERNAL_MAX_NUM];        ///<扩展轴位置
    double acctime;                           ///<扩展轴加速时间
    enum MotiontriggerMode motiontriggerMode; ///<运动轨迹触发方式
}ExjMovePara;
```

## 2.90 扩展轴类型

```
typedef enum ExternalJointType
{
    line_externalJoint = 0,                  ///<直线
    rotation_externalJoint = 1               ///<旋转
}ExternalJointType;
```

## 2.91 扩展轴配置

```
typedef struct ExjConfig
{
    char name[NAMELENTH];                    ///< 扩展轴名称
    enum ExternalJointType type;              ///< 扩展轴类型
}
```

```

double ratio;           ///< 传动比
double minlimit;        ///< 最小值
double maxlimit;        ///< 最大值
long long zeroPosition; ///< 零位(pulse)
double exjToRef[ROB_AXIS_NUM]; ///< 扩展轴相对于参考系的关系
char masterExjName[NAMELENTH]; ///< 主轴名称
double masterSlaveRate;  ///< 主从比
}ExjConfig;

```

## 2.92 常用宏定义

```

#define ROB_AXIS_NUM 6
#define CR6_DI_NUM 8
#define CR6_DO_NUM 8
#define CR6_CI_NUM 8
#define CR6_CO_NUM 8
#define CR6_AI_NUM 3
#define CR6_AO_NUM 1
#define CR6_TOOL_DI_NUM 10
#define CR6_TOOL_DO_NUM 10
#define CR6_TOOL_AI_NUM 2
#define ModbusMasterConfigNumber 20
#define ModbusAddrMapNumber 100
#define CR6_SAFETY_LIMITS_BOUNDARY_PLANE_NUM 8
#define CR6_BOOL_REG_NUM 1000
#define CR6_INT16_REG_NUM 1000
#define CR6_INT32_REG_NUM 1000
#define CR6_FLOAT_REG_NUM 1000
#define CR6_REG_MAX_NUM 100
#define ENCODER_TICK_NUM 2
#define DH_PARA_MAX_NUM 100
#define DY_PARA_MAX_NUM 100
#define CR6_ISTALL_VAR_NUM 100
#define CR6_ISTALL_VAR_NAME_LENGTH 100
#define CR6_INSTALL_VAR_MAX_LENGTH 100
#define CR6_INSTALL_VAE_TABLE_MAX_LENGTH 10000
#define Script_MAX_TableElement 500
#define Script_MAX_DataLen 100
#define Global_Var_Max_Num 100 ///< 读全局变量最大个数
#define Set_Global_Var_Max_Num 1 ///< 写读全局变量最大个数
#define COR_MAX_NUM 50 ///< 协程最大个数
#define CR6_EQEP_NUM 2
#define TCP_MAX_NUM 20
#define NAMELENTH 50

```

```
#define CR6_DIGITAL_INPUT_NUM 8
#define CR6_DIGITAL_OUTPUT_NUM 8
#define CR6_CONFIGURABLED_INPUT_NUM 8
#define CR6_CONFIGURABLED_OUTPUT_NUM 8
#define CR6_ANALOG_INPUT_NUM 3
#define CR6_ANALOG_OUTPUT_NUM 1
#define CR6_TOOL_ANALOG_INPUT_NUM 2
#define CR6_POSE_NODE_NUM 10
#define CR6_LINE_NODE_NUM 10
#define CR6_PLANE_NODE_NUM 10
#define CR6_COM_NUM 4
#define LOG_MAX_NUM 3000
#define LOG_RPC_MAX_NUM 10
#define MOVECPOINT_NUM 2
#define PATHMAXPOINTS 10000    ///轨迹最大数量为 10000 个点
#define VERSION_MAX_LENGTH 50
#define EXTERNAL_MAX_NUM 10    //扩展轴最大个数
#define ROB_EXJ_AXIS_NUM 12
#define ROB_EXJOG_AXIS_NUM 16
#define TRUE 1
#define FALSE 0
typedef int BOOL;
```

## 三、接口函数

### 3.1 基础

#### 3.1.1 基础接口

##### 3.1.1.1 机器人创建连接

###### 函数

```
CRresult cr_create_robot(RobotHandle* robotHandle, const char * ipAddr, int port, const char *
passwd)
```

###### 简介

创建机器人连接。

###### 参数

| 参数名称        | 数据类型          | 输入/输出 | 描述        |
|-------------|---------------|-------|-----------|
| robotHandle | RobotHandle * | 输出    | 机械臂实例句柄   |
| ipAddr      | const char *  | 输入    | 控制柜 IP 地址 |

|        |              |    |              |
|--------|--------------|----|--------------|
| port   | int          | 输入 | 控制柜端口号为 2323 |
| passwd | const char * | 输入 | 预留           |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.1.1.2 机器人断开连接

### 函数

**CRresult** cr\_destroy\_robot(**RobotHandle** robotHandle)

### 简介

机器人断开连接。

### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.1.1.3 机器人关机

### 函数

**CRresult** cr\_shutdown(**RobotHandle** robotHandle)

## 简介

机器人关机。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_shutdown(robotHandle); //机器人关机
if (result != success) //判断机器人关机是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.1.4 机械臂上电

## 函数

CRresult cr\_poweron(RobotHandle robotHandle)

## 简介

机械臂上电。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_poweron(robotHandle); //机械臂上电
```

```
if (result != success) //判断机械臂上电是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.1.5 机械臂断电

#### 函数

**CRresult** cr\_poweroff(**RobotHandle** robotHandle)

#### 简介

机械臂断电。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_poweroff(robotHandle); //机械臂断电
if (result != success) //判断机械臂断电是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.1.6 机械臂使能

#### 函数

**CRresult** cr\_enable(**RobotHandle** robotHandle)

#### 简介

机械臂使能。

#### 参数

| 参数名称 | 数据类型 | 输入/输出 | 描述 |
|------|------|-------|----|
|------|------|-------|----|

|             |             |    |         |
|-------------|-------------|----|---------|
| robotHandle | RobotHandle | 输入 | 机械臂实例句柄 |
|-------------|-------------|----|---------|

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_enable(robotHandle); //机械臂使能
if (result != success) //判断机械臂使能是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.1.1.7 机械臂关使能

### 函数

CRresult cr\_disable(RobotHandle robotHandle)

### 简介

机械臂关使能。

### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_disable(robotHandle); //机械臂关使能
if (result != success) //判断机械臂关使能是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.1.8 机械臂故障复位

#### 函数

**CRresult** cr\_FaultReset(**RobotHandle** robotHandle)

#### 简介

机械臂故障复位。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_FaultReset(robotHandle); //机械臂故障复位
if (result != success) //判断机械臂故障复位是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.1.9 获取控制柜状态

#### 函数

**CRresult** cr\_get\_controlMode(**RobotHandle** robotHandle, int \*controlmode)

#### 简介

获取控制柜的自动或手动状态。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述              |
|-------------|-------------|-------|-----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄         |
| controlmode | int *       | 输出    | 控制柜状态：0-自动，1-手动 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。



### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int controlmode;
result = cr_get_controlMode(robotHandle, &controlmode); //获取控制柜状态：0-自动，1-手动
printf("%d\n", controlmode);
if (result != success) //判断获取控制柜状态是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.1.10 写机械臂速度百分比

#### 函数

**CRresult** cr\_set\_robotSpeedPercent(**RobotHandle** robotHandle, unsigned int speedPercent)

#### 简介

写机械臂速度百分比。

#### 参数

| 参数名称         | 数据类型         | 输入/输出 | 描述                     |
|--------------|--------------|-------|------------------------|
| robotHandle  | RobotHandle  | 输入    | 机械臂实例句柄                |
| speedPercent | unsigned int | 输入    | 机械臂速度百分比<br>(范围：1-100) |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
unsigned int speedpercent = 50;
result = cr_set_robotSpeedPercent( robotHandle, speedpercent); //将机械臂速度百分比写为 50
if (result != success) //判断写机械臂速度百分比是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
```

```
return -1;
```

### 3.1.1.11 读机械臂速度百分比

#### 函数

**CRresult** cr\_get\_robotSpeedPercent(**RobotHandle** robotHandle, unsigned int \*speedPercent)

#### 简介

读机械臂速度百分比。

#### 参数

| 参数名称         | 数据类型           | 输入/输出 | 描述                      |
|--------------|----------------|-------|-------------------------|
| robotHandle  | RobotHandle    | 输入    | 机械臂实例句柄                 |
| speedPercent | unsigned int * | 输出    | 机械臂速度百分比<br>(范围: 1-100) |

#### 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
unsigned int speedpercent = 0;
result = cr_get_robotSpeedPercent(robotHandle, &speedpercent); //读机械臂速度百分比
if (result != success) //判断读机械臂速度百分比是否成功
    return -1;
printf("%u\n", speedpercent);
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.1.12 获取 SDK 版本号

#### 函数

**CRresult** cr\_get\_sdk\_version(char \*version)

#### 简介

获取 SDK 版本号。

#### 参数

| 参数名称 | 数据类型 | 输入/输出 | 描述 |
|------|------|-------|----|
|------|------|-------|----|

|         |        |    |          |
|---------|--------|----|----------|
| version | char * | 输出 | SDK 版本信息 |
|---------|--------|----|----------|

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
char version[20];
CRresult result = cr_get_sdk_version(version); //获取 SDK 版本号
if (result != success) //判断机器人断开连接是否成功
    return -1;
printf("%s", version);
```

## 3.1.1.13 获取机械臂状态数据

### 函数

```
CRresult cr_get_robotStateData(RobotHandle robotHandle, RobotStateData* robotStateData);
```

### 简介

获取机械臂状态数据。

### 参数

| 参数名称           | 数据类型            | 输入/输出 | 描述      |
|----------------|-----------------|-------|---------|
| robotHandle    | RobotHandle     | 输入    | 机械臂实例句柄 |
| robotStateData | RobotStateData* | 输出    | 机械臂状态数据 |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
RobotStateData robotStateData;
result = cr_get_robotStateData(robotHandle, &robotStateData); //获取机械臂状态数据
if (result != success) //判断获取机械臂状态数据是否成功
    return -1;
printf("实际 tcp 位置: ");
for (int i = 0; i < 6; i++)
{
    printf("%f", robotStateData.tcpActualPose[i]);
    if (i < 5)
        printf(",");
}
```

```

}
printf("\n");

printf("实际关节位置: ");
for (int i = 0; i < 6; i++)
{
    printf("%f", robotStateData.jointActualPos[i]);
    if (i < 5)
        printf(",");
}
printf("\n");

printf("机械臂当前状态: %d\n", robotStateData.robotMode);
printf("机械臂运动状态: %d\n", robotStateData.robotMoveStatus);
printf("机械臂速度百分比: %d\n", robotStateData.robotSpeedPercent);

printf("控制器可配置数字输出: ");
for (int i = 0; i < 8; i++)
{
    printf("%d", robotStateData.configurableDigitalOutput[i]);
    if (i < 7)
        printf(",");
}
printf("\n");
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;

```

## 3.1.2 程序控制

### 3.1.2.1 下载脚本程序

#### 函数

**CRresult** cr\_downloadProgram(**RobotHandle** robotHandle, const char \*programfile)

#### 简介

下载脚本程序。

#### 参数

| 参数名称        | 数据类型         | 输入/输出 | 描述       |
|-------------|--------------|-------|----------|
| robotHandle | RobotHandle  | 输入    | 机械臂实例句柄  |
| programfile | const char * | 输入    | 需下载的脚本程序 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
const char *programfile = "function mainFuncProgram()\n"
    "--start robotConfig\n"
    "TCP_1 = { 0,0,0,0,0 } \n"
    "Payload_1 = { 0,{0,0,0} } \n"
    "set_tcp_payload(Payload_1[1], Payload_1[2])\n"
    "--end robotConfig\n"
    "function RobotProgram()\n"
    "while (true)\n"
    "do\n"
    "--output: CDO_0 = true\n"
    "set_standard_digital_out(0, true)\n"
    "wait(1) --sync\n"
    "end\n"
    "end\n"
    "RobotProgram_Result = task_create(RobotProgram)\n"
    "function PauseFuncProgram()\n"
    "while (true)\n"
    "do\n"
    "wait(100) --sync\n"
    "end\n"
    "end\n"
    "PauseFuncProgram_Result = task_create(PauseFuncProgram)\n"
    "end\n"
    "mainFuncProgram_Result = task_create(mainFuncProgram)";
result = cr_downloadProgram(robotHandle, programfile); //加载程序
if (result != success) //判断下载脚本程序是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.2.2 下载脚本程序（加密）

## 函数

```
CRresult cr_downloadProgram_encryption(RobotHandle robotHandle, const char *programfile)
```

## 简介

下载脚本程序（加密）。

## 参数

| 参数名称        | 数据类型         | 输入/输出 | 描述       |
|-------------|--------------|-------|----------|
| robotHandle | RobotHandle  | 输入    | 机械臂实例句柄  |
| programfile | const char * | 输入    | 需下载的脚本程序 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
const char *programfile = "function mainFuncProgram()\n"
    "--start robotConfig\n"
    "TCP_1 = { 0,0,0,0,0,0 }\n"
    "Payload_1 = { 0,{0,0,0} }\n"
    "set_tcp_payload(Payload_1[1], Payload_1[2])\n"
    "--end robotConfig\n"
    "function RobotProgram()\n"
    "while (true)\n"
    "do\n"
    "--output: CDO_0 = true\n"
    "set_standard_digital_out(0, true)\n"
    "wait(1) --sync\n"
    "end\n"
    "end\n"
    "RobotProgram_Result = task_create(RobotProgram)\n"
    "function PauseFuncProgram()\n"
    "while (true)\n"
    "do\n"
    "wait(100) --sync\n"
    "end\n"
    "end\n"
    "PauseFuncProgram_Result = task_create(PauseFuncProgram)\n"
    "end\n"
    "mainFuncProgram_Result = task_create(mainFuncProgram)";
result = cr_download_program_encryption(robotHandle, programfile); //加载程序（加密）
if (result != success) //判断下载脚本程序（加密）是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
```

```
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.2.3 上传脚本程序

#### 函数

**CRresult** cr\_uploadProgram(**RobotHandle** robotHandle, const char \*programfile)

#### 简介

上传脚本程序。

#### 参数

| 参数名称        | 数据类型         | 输入/输出 | 描述      |
|-------------|--------------|-------|---------|
| robotHandle | RobotHandle  | 输入    | 机械臂实例句柄 |
| programfile | const char * | 输出    | 上传的脚本程序 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char *programfile = new char[1024 * 75 * 10];
result = cr_uploadProgram(robotHandle, programfile); //上传程序
if (result != success) //判断上传脚本程序是否成功
    return -1;
printf("%s\n", programfile);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.2.4 运行程序

#### 函数

**CRresult** cr\_play(**RobotHandle** robotHandle)

#### 简介

运行当前程序。若当前无已加载的程序，请先通过 cr\_downloadProgram 加载一个程序。

#### 参数

| 参数名称 | 数据类型 | 输入/输出 | 描述 |
|------|------|-------|----|
|------|------|-------|----|

|             |             |    |         |
|-------------|-------------|----|---------|
| robotHandle | RobotHandle | 输入 | 机械臂实例句柄 |
|-------------|-------------|----|---------|

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_play(robotHandle); //运行程序
if (result != success) //判断运行程序是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.1.2.5 停止程序运行

### 函数

CRresult cr\_stop(RobotHandle robotHandle)

### 简介

停止运行。

### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_stop(robotHandle); //停止程序运行
if (result != success) //判断停止程序运行是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```



### 3.1.2.6 暂停程序运行

#### 函数

**CRresult** cr\_pause(**RobotHandle** robotHandle)

#### 简介

暂停运行。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_pause(robotHandle); //暂停程序运行
if (result != success) //判断暂停程序运行是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.2.7 读取脚本当前运行行号

#### 函数

**CRresult** cr\_script\_current\_line\_get(**RobotHandle** robotHandle, int \*currentLine, int \*lineLen)

#### 简介

读取脚本当前运行行号。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述       |
|-------------|-------------|-------|----------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄  |
| currentLine | int *       | 输出    | 脚本当前行号   |
| lineLen     | int *       | 输出    | 脚本行号数组长度 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int currentline[50];
int linelen = 0;
result = cr_script_current_line_get(robotHandle, currentline, &linelen); //读取脚本当前运行行号
if (result != success) //判断读取脚本当前运行行号是否成功
    return -1;
printf("%d\n", currentline[0]);
printf("%d\n", linelen);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.2.8 读脚本运行状态

## 函数

```
CResult cr_get_lua_scriptstatus(RobotHandle robotHandle, Lua_ScriptStatus *scriptstatus);
```

## 简介

读脚本运行状态。

## 参数

| 参数名称         | 数据类型               | 输入/输出 | 描述       |
|--------------|--------------------|-------|----------|
| robotHandle  | RobotHandle        | 输入    | 机械臂实例句柄  |
| scriptstatus | Lua_ScriptStatus * | 输出    | 当前脚本运行状态 |

## 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
Lua_ScriptStatus scriptstatus;
result = cr_get_lua_scriptstatus(robotHandle, &scriptstatus); //读脚本运行状态
if (result != success) //判断读脚本运行状态是否成功
    return -1;
printf("%d\n", scriptstatus);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
```

```
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.2.9 上传工程文件

#### 函数

**CRresult** cr\_uploadProject(**RobotHandle** robotHandle, char \*FilePath, char \*filename)

#### 简介

上传 crp 文件和脚本程序。如果先使用 cr\_downloadProgram 下载脚本程序后再上传工程文件，则只有脚本程序有效，crp 文件为空。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述             |
|-------------|-------------|-------|----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄        |
| FilePath    | char *      | 输入    | 文件保存路径         |
| filename    | char *      | 输入    | .crp 和脚本程序的文件名 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char FilePath[100] = "D:/teachpendant/teach_1.6d/teachpendant_x64/program/"; //文件保存路径
char filename[100] = "test"; // .crp 和脚本程序的文件名
result = cr_uploadProject(robotHandle, FilePath, filename); //工程文件上传
if (result != success) //判断工程文件上传是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.2.10 下载工程文件

#### 函数

**CRresult** cr\_downloadProject(**RobotHandle** robotHandle, char \*crpFilepathname, char \*crscriptFilepathname)

#### 简介

下载 crp 文件和脚本程序，下载成功后可从示教器上看到目前处于工程暂停状态。

## 参数

| 参数名称                 | 数据类型        | 输入/输出 | 描述         |
|----------------------|-------------|-------|------------|
| robotHandle          | RobotHandle | 输入    | 机械臂实例句柄    |
| crpFilepathname      | char *      | 输入    | .crp 文件路径名 |
| crscriptFilepathname | char *      | 输入    | 脚本程序路径名    |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char crpFilepathname[100] = "D:/teach_1.6d/teachpendant_x64/program/12.crp";
//.crp 文件路径
char crscriptFilepathname[100] = "D:/teach_1.6d/teachpendant_x64/program/12.crscript";
//脚本程序路径
result = cr_downloadProject(robotHandle, crpFilepathname, crscriptFilepathname);
//工程文件下载
if (result != success) //判断工程文件下载是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.2.11 是否存在弹窗

## 函数

```
CRresult cr_script_popup_exist(RobotHandle robotHandle, BOOL *hasPopup)
```

## 简介

是否存在弹窗。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述              |
|-------------|-------------|-------|-----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄         |
| hasPopup    | BOOL *      | 输出    | 弹窗结果：0-不存在 1-存在 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
BOOL hasPopup;
result = cr_script_popup_exist(robotHandle, &hasPopup); //是否存在弹窗：1-是，0-否
if (result != success) //判断是否存在弹窗是否成功
    return -1;
printf("%d", hasPopup);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.1.2.12 读取弹窗信息

#### 函数

**CResult** cr\_script\_popup\_msg\_get(**RobotHandle** robotHandle, **PopUpMsg \***popupMsg)

#### 简介

读取弹窗信息。如果读取的弹窗信息中含有中文，需要在打印结果前执行一行代码：

- ①在 Windows 系统中执行：system("chcp 65001");
- ②在 Linux 系统中执行：system("export LANG=C.UTF-8");

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| popupMsg    | PopUpMsg *  | 输出    | 弹窗信息    |

#### 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
PopUpMsg popupMsg;
result = cr_script_popup_msg_get(robotHandle, &popupMsg); //读取弹窗信息
if (result != success) //判断读取弹窗信息是否成功
    return -1;
#ifdef _WIN32
```

```

    system("chcp 65001");
#elif __linux__
    system("export LANG=C.UTF-8");
#endif
printf("%s\n", popupMsg.var_data);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;

```

### 3.1.2.13 设置输入弹窗信息

#### 函数

**CRresult** cr\_script\_popup\_msg\_set(**RobotHandle** robotHandle, **VariableMsg** \*invarMsg)

#### 简介

设置输入弹窗信息。

#### 参数

| 参数名称        | 数据类型          | 输入/输出 | 描述      |
|-------------|---------------|-------|---------|
| robotHandle | RobotHandle   | 输入    | 机械臂实例句柄 |
| invarMsg    | VariableMsg * | 输入    | 弹窗信息    |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
VariableMsg invarMsg;
char name[CR6_INSTALL_VAR_NAME_LENGTH] = { "install" };
strcpy_s(invarMsg.variableName, sizeof(name), name);
invarMsg.variableType = LUA_TNUMBER;
invarMsg.variableID = 3;
invarMsg.numberValue = 5;
result = cr_script_popup_msg_set(robotHandle, &invarMsg); //设置输入弹窗信息
if (result != success) //判断设置输入弹窗信息是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;

```

### 3.1.2.14 关闭弹窗

#### 函数

**CRresult** cr\_script\_popup\_close(**RobotHandle** robotHandle)

#### 简介

关闭弹窗。只能关闭输入弹窗，不能关闭输出弹窗。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_script_popup_close(robotHandle); //关闭弹窗
if (result != success) //判断关闭弹窗是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.2 运动模块

### 3.2.1 运动接口

#### 3.2.1.1 轴空间运动 moveJ

#### 函数

**CRresult** cr\_moveJ(**RobotHandle** robotHandle, **PointControlPara** pointControlPara)

#### 简介

轴空间运动，推荐使用 cr\_move\_joint 接口。移动相关速度和加速度范围见附录 A 速度和加速度范围。

#### 参数

| 参数名称 | 数据类型 | 输入/输出 | 描述 |
|------|------|-------|----|
|------|------|-------|----|

|                  |                  |    |         |
|------------------|------------------|----|---------|
| robotHandle      | RobotHandle      | 输入 | 机械臂实例句柄 |
| pointControlPara | PointControlPara | 输入 | 运动相关参数  |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double jointpos[6] = { 60, 60, -60, 60, 60, 60 }; //目标点示例数据
double pose[6] = { 0, 0, 0, 0, 0, 0 }; //在 MoveJ 中无影响
PointControlPara pointControlPara;
for (int index = 0; index < ROB_AXIS_NUM; index++)
{
    pointControlPara.pose[index] = pose[index];
    pointControlPara.jointpos[index] = jointpos[index];
    pointControlPara.tcpOffset[index] = 0;
    pointControlPara.coordinatePose[index] = 0;
    pointControlPara.speed[index] = 30;
    pointControlPara.acc[index] = 30;
    pointControlPara.jerk[index] = 60;
}
pointControlPara.tcpID = -1;
pointControlPara.coordinateType = jointCoordinate;
pointControlPara.pointTransType = pointTransStop;
pointControlPara.pointTransRadius = 0;
pointControlPara.poseTranType = poseTranMoveToTargetPose;
pointControlPara.motiontriggerMode = MovetriggerbyOnlyRpc;
result = cr_moveJ(robotHandle, pointControlPara); //轴空间运动
if (result != success) //判断轴空间运动是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.1.2 轴空间运动 movejoint

#### 函数

```
CRresult cr_move_joint(RobotHandle robotHandle, PointControlPara pointControlPara, BOOL
isBlock)
```



## 简介

轴空间运动。移动相关速度和加速度范围见附录 A 速度和加速度范围。

## 参数

| 参数名称             | 数据类型             | 输入/输出 | 描述                |
|------------------|------------------|-------|-------------------|
| robotHandle      | RobotHandle      | 输入    | 机械臂实例句柄           |
| pointControlPara | PointControlPara | 输入    | 运动相关参数            |
| isBlock          | BOOL             | 输入    | TRUE-阻塞 FALSE-非阻塞 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double jointpos[6] = { 60, 60, -60, 60, 60, 60 }; //目标点示例数据
double pose[6] = { 0, 0, 0, 0, 0, 0 }; //在 MoveJ 中无影响
PointControlParaSimple pointControlParaSimple;
for (int index = 0; index < ROB_AXIS_NUM; index++)
{
    pointControlParaSimple.pose[index] = pose[index];
    pointControlParaSimple.jointpos[index] = jointpos[index];
    pointControlParaSimple.tcpOffset[index] = 0;
    pointControlParaSimple.coordinatePose[index] = 0;
    pointControlParaSimple.speed[index] = 30;
    pointControlParaSimple.acc[index] = 30;
}
pointControlParaSimple.coordinateType = jointCoordinate;
pointControlParaSimple.pointTransType = pointTransStop;
pointControlParaSimple.motiontriggerMode = MovetriggerbyOnlyRpc;
PointControlPara pointControlPara;
result = cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple,
&pointControlPara); //PointControlParaSimple 数据转换为 PointControlPara
if (result != success) //判断 PointControlParaSimple 数据转换为 PointControlPara 是否成功
    return -1;
result = cr_move_joint(robotHandle, pointControlPara, TRUE); //轴空间运动
if (result != success) //判断轴空间运动是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.1.3 直线运动 moveL

#### 函数

**CRresult** cr\_moveL(**RobotHandle** robotHandle, **PointControlPara** pointControlPara)

#### 简介

直线运动，推荐使用 cr\_move\_line 接口。移动相关速度和加速度范围见附录 A 速度和加速度范围。

#### 参数

| 参数名称             | 数据类型             | 输入/输出 | 描述      |
|------------------|------------------|-------|---------|
| robotHandle      | RobotHandle      | 输入    | 机械臂实例句柄 |
| pointControlPara | PointControlPara | 输入    | 运动相关参数  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double jointpos[6] = { 60, 60, -60, 60, 60, 60 }; //起始点示例数据
double pose[6] = { 0, -438, 887, -57.34, 0, -149 }; //目标点示例数据
PointControlPara pointControlPara;
for (int index = 0; index < ROB_AXIS_NUM; index++)
{
    pointControlPara.pose[index] = pose[index];
    pointControlPara.jointpos[index] = jointpos[index];
    pointControlPara.tcpOffset[index] = 0;
    pointControlPara.coordinatePose[index] = 0;
    pointControlPara.speed[index] = 30;
    pointControlPara.acc[index] = 30;
    pointControlPara.jerk[index] = 60;
}
pointControlPara.tcpID = -1;
pointControlPara.coordinateType = baseCoordinate;
pointControlPara.pointTransType = pointTransStop;
pointControlPara.pointTransRadius = 0;
pointControlPara.poseTranType = poseTranMoveToTargetPose;
pointControlPara.motiontriggerMode = MovetriggerbyOnlyRpc;
result = cr_moveL(robotHandle, pointControlPara); //直线运动
if (result != success) //判断直线运动是否成功
```

```

    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;

```

### 3.2.1.4 直线运动 moveline

#### 函数

**CRresult** cr\_move\_line(**RobotHandle** robotHandle, **PointControlPara** pointControlPara, **BOOL** isBlock)

#### 简介

直线运动。移动相关速度和加速度范围见附录 A 速度和加速度范围。

#### 参数

| 参数名称             | 数据类型             | 输入/输出 | 描述                |
|------------------|------------------|-------|-------------------|
| robotHandle      | RobotHandle      | 输入    | 机械臂实例句柄           |
| pointControlPara | PointControlPara | 输入    | 运动相关参数            |
| isBlock          | BOOL             | 输入    | TRUE-阻塞 FALSE-非阻塞 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double jointpos[6] = { 60, 60, -60, 60, 60, 60 }; //起始点示例数据
double pose[6] = { 0, -438, 887, -57.34, 0, -149 }; //目标点示例数据
PointControlParaSimple pointControlParaSimple;
for (int index = 0; index < ROB_AXIS_NUM; index++)
{
    pointControlParaSimple.pose[index] = pose[index];
    pointControlParaSimple.jointpos[index] = jointpos[index];
    pointControlParaSimple.tcpOffset[index] = 0;
    pointControlParaSimple.coordinatePose[index] = 0;
    pointControlParaSimple.speed[index] = 30;
    pointControlParaSimple.acc[index] = 30;
}
pointControlParaSimple.coordinateType = baseCoordinate;
pointControlParaSimple.pointTransType = pointTransStop;
pointControlParaSimple.motiontriggerMode = MovetriggerbyOnlyRpc;
PointControlPara pointControlPara;

```

```
result = cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple,
&pointControlPara); //PointControlParaSimple 数据转换为 PointControlPara
if (result != success) //判断 PointControlParaSimple 数据转换为 PointControlPara 是否成功
    return -1;
result = cr_move_line(robotHandle, pointControlPara, TRUE); //直线运动
if (result != success) //判断直线运动是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.1.5 点动运动

#### 函数

**CRresult** cr\_moveJog(**RobotHandle** robotHandle, **PointControlPara** pointControlPara)

#### 简介

点动运动。连续点动控制变化时位置一次最多改变 100mm，角度最多改变 20°。移动相关速度和加速度范围见附录 A 速度和加速度范围。

#### 参数

| 参数名称             | 数据类型             | 输入/输出 | 描述      |
|------------------|------------------|-------|---------|
| robotHandle      | RobotHandle      | 输入    | 机械臂实例句柄 |
| pointControlPara | PointControlPara | 输入    | 运动相关参数  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double pose[6] = { 0, 5, 0, 0, 0, 0 }; //沿坐标系 y 轴运动 5mm
double pointSysPose[6] = { 114.3, -390.498, 215.347, -180, 0, 0 }; //点坐标系
PointControlPara pointControlPara;
for (int index = 0; index < ROB_AXIS_NUM; index++)
{
    pointControlPara.pose[index] = pose[index];
    pointControlPara.jointpos[index] = 0;
    pointControlPara.tcpOffset[index] = 0;
    pointControlPara.coordinatePose[index] = pointSysPose[index];
    pointControlPara.speed[index] = 30;
```

```

    pointControlPara.acc[index] = 30;
    pointControlPara.jerk[index] = 60;
}
pointControlPara.tcpID = -1;
pointControlPara.coordinateType = PointCoordinate; //基于点坐标系进行点动
pointControlPara.pointTransType = pointTransStop;
pointControlPara.pointTransRadius = 0;
pointControlPara.poseTranType = poseTranMoveToTargetPose;
pointControlPara.motiontriggerMode = MovetriggerbyOnlyRpc;
result = cr_moveJog(robotHandle, pointControlPara); //点动运动
if (result != success) //判断点动运动是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;

```

### 3.2.1.6 圆弧运动

#### 函数

**CRresult** cr\_move\_circle(**RobotHandle** robotHandle, **PointControlParaList** pointControlParaList, **BOOL** isBlock)

#### 简介

圆弧运动。移动相关速度和加速度范围见附录 A 速度和加速度范围。

#### 参数

| 参数名称                 | 数据类型                 | 输入/输出 | 描述                |
|----------------------|----------------------|-------|-------------------|
| robotHandle          | RobotHandle          | 输入    | 机械臂实例句柄           |
| pointControlParaList | PointControlParaList | 输入    | 圆弧运动相关参数          |
| isBlock              | BOOL                 | 输入    | TRUE-阻塞 FALSE-非阻塞 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double jointpos0[6] = { 60, 60, -60, 60, 60, 60 }; //初始点关节角
double pose0[6] = { 0, 0, 0, 0, 0, 0 }; //在 MoveJ 中无影响
PointControlParaSimple pointControlParaSimple0; //初始点简化运动参数
for (int index = 0; index < ROB_AXIS_NUM; index++)

```

```

{
    pointControlParaSimple0.pose[index] = pose0[index];
    pointControlParaSimple0.jointpos[index] = jointpos0[index];
    pointControlParaSimple0.tcpOffset[index] = 0;
    pointControlParaSimple0.coordinatePose[index] = 0;
    pointControlParaSimple0.speed[index] = 30;
    pointControlParaSimple0.acc[index] = 30;
}
pointControlParaSimple0.coordinateType = jointCoordinate;
pointControlParaSimple0.pointTransType = pointTransStop;
pointControlParaSimple0.motiontriggerMode = MovetriggerbyOnlyRpc;
PointControlPara pointControlPara0; //初始点运动参数
result = cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple0,
&pointControlPara0); //PointControlParaSimple 数据转换为 PointControlPara
if (result != success) //判断 PointControlParaSimple 数据转换为 PointControlPara 是否成功
    return -1;
result = cr_move_joint(robotHandle, pointControlPara0, TRUE); //移至初始点

double jointpos1[6] = { 60, 55, -65, 60, 60, 60 }; //中间点关节角
double pose1[6]; //中间点位姿
cr_kineForward(robotHandle, jointpos1, pose1); //正解求中间点位姿
double jointpos2[6] = { 60, 50, -70, 60, 60, 60 }; //终点关节角
double pose2[6]; //终点位姿
cr_kineForward(robotHandle, jointpos2, pose2); //正解求终点位姿
PointControlParaSimple pointControlParaSimple1; //中间点简化运动参数
PointControlParaSimple pointControlParaSimple2; //终点简化运动参数
for (int index = 0; index < ROB_AXIS_NUM; index++)
{
    pointControlParaSimple1.pose[index] = pose1[index];
    pointControlParaSimple1.jointpos[index] = jointpos1[index];
    pointControlParaSimple1.tcpOffset[index] = 0;
    pointControlParaSimple1.coordinatePose[index] = 0;
    pointControlParaSimple1.speed[index] = 30;
    pointControlParaSimple1.acc[index] = 30;
    pointControlParaSimple2.pose[index] = pose2[index];
    pointControlParaSimple2.jointpos[index] = jointpos2[index];
    pointControlParaSimple2.tcpOffset[index] = 0;
    pointControlParaSimple2.coordinatePose[index] = 0;
    pointControlParaSimple2.speed[index] = 30;
    pointControlParaSimple2.acc[index] = 30;
}
pointControlParaSimple1.coordinateType = baseCoordinate;
pointControlParaSimple1.pointTransType = pointTransStop;
pointControlParaSimple1.motiontriggerMode = MovetriggerbyOnlyRpc;

```

```
pointControlParaSimple2.coordinateType = baseCoordinate;
pointControlParaSimple2.pointTransType = pointTransStop;
pointControlParaSimple2.motiontriggerMode = MovetriggerbyOnlyRpc;
PointControlPara pointControlPara1; //中间点运动参数
PointControlPara pointControlPara2; //终点运动参数
result = cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple1,
&pointControlPara1); //PointControlParaSimple 数据转换为 PointControlPara
if (result != success) //判断 PointControlParaSimple 数据转换为 PointControlPara 是否成功
    return -1;
result = cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple2,
&pointControlPara2); //PointControlParaSimple 数据转换为 PointControlPara
if (result != success) //判断 PointControlParaSimple 数据转换为 PointControlPara 是否成功
    return -1;
PointControlParaList pointControlParaList;
pointControlParaList.pointcontrolpara[0] = pointControlPara1;
pointControlParaList.pointcontrolpara[1] = pointControlPara2;
pointControlParaList.fixedrot = 0;
pointControlParaList.centralangle = 0;
result = cr_move_circle(robotHandle, pointControlParaList, TRUE); //圆弧运动
if (result != success) //判断圆弧运动是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.1.7 运动控制

#### 函数

**CRresult** cr\_moveControl(**RobotHandle** robotHandle, enum **MoveType** moveType)

#### 简介

运动控制。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| moveType    | MoveType    | 输入    | 运动控制参数  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
```

```
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
MoveType moveType = ImdStop; //运动模式设置为立即停止
result = cr_moveControl(robotHandle, moveType); //运动控制
if (result != success) //判断运动控制是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.1.8 设置阻塞移动中参数阈值

#### 函数

**CRresult** cr\_move\_block\_threshold\_set(**RobotHandle** robotHandle, double poseDelt, double jointDelt)

#### 简介

设置阻塞移动中参数阈值。在阻塞移动中，当前位姿/关节角度与目标点的位姿/关节角度的相差值小于位姿/关节阈值时，则被判定为已移动到目标点。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述           |
|-------------|-------------|-------|--------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄      |
| poseDelt    | double      | 输入    | 位姿阈值 默认 0.02 |
| jointDelt   | double      | 输入    | 关节阈值 默认 0.1  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_move_block_threshold_set(robotHandle, 0.05, 0.05);
//将阻塞移动中的位姿阈值设置为 0.05，关节阈值设置为 0.05
if (result != success) //判断设置阻塞移动中参数阈值是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```



### 3.2.1.9 PointControlParaSimple 数据转换

#### 函数

`CRresult cr_move_pointControlPara_transfer(RobotHandle robotHandle, PointControlParaSimple pointControlParaSimple, PointControlPara *pointControlPara)`

#### 简介

PointControlParaSimple 转换为 PointControlPara。

#### 参数

| 参数名称                   | 数据类型                   | 输入/输出 | 描述      |
|------------------------|------------------------|-------|---------|
| robotHandle            | RobotHandle            | 输入    | 机械臂实例句柄 |
| pointControlParaSimple | PointControlParaSimple | 输入    | 简化运动参数  |
| pointControlPara       | PointControlPara *     | 输出    | 运动参数    |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double jointpos[6] = { 60, 60, -60, 60, 60, 60 }; //目标点示例数据
double pose[6] = { 0, 0, 0, 0, 0, 0 }; //在 MoveJ 中无影响
PointControlParaSimple pointControlParaSimple;
for (int index = 0; index < ROB_AXIS_NUM; index++)
{
    pointControlParaSimple.pose[index] = pose[index];
    pointControlParaSimple.jointpos[index] = jointpos[index];
    pointControlParaSimple.tcpOffset[index] = 0;
    pointControlParaSimple.coordinatePose[index] = 0;
    pointControlParaSimple.speed[index] = 30;
    pointControlParaSimple.acc[index] = 30;
}
pointControlParaSimple.coordinateType = jointCoordinate;
pointControlParaSimple.pointTransType = pointTransStop;
pointControlParaSimple.motiontriggerMode = MovetriggerbyOnlyRpc;
PointControlPara pointControlPara;
result = cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple,
&pointControlPara); //PointControlParaSimple 数据转换
if (result != success) //判断 PointControlParaSimple 数据转换是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
```

```
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.1.10 设置软件自由驱动启动

#### 函数

**CRresult** cr\_set\_softFreeDriveEnabled(**RobotHandle** robotHandle, **BOOL** softFreeDriveEnabled)

#### 简介

设置软件自由驱动启动。

#### 参数

| 参数名称                 | 数据类型        | 输入/输出 | 描述                             |
|----------------------|-------------|-------|--------------------------------|
| robotHandle          | RobotHandle | 输入    | 机械臂实例句柄                        |
| softFreeDriveEnabled | BOOL        | 输入    | TRUE 为进入拖拽模式，<br>FALSE 为退出拖拽模式 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
BOOL softFreeDriveEnabled = TRUE; //TRUE 为进入拖拽模式
result = cr_set_softFreeDriveEnabled(robotHandle, softFreeDriveEnabled);
//设置软件自由驱动启动
if (result != success) //判断设置软件自由驱动启动是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.1.11 读机械臂运动状态

#### 函数

**CRresult** cr\_get\_robotMoveStatus(**RobotHandle** robotHandle, **BOOL** \*isRobotMoving)

#### 简介

读机械臂运动状态。

### 参数

| 参数名称          | 数据类型        | 输入/输出 | 描述                  |
|---------------|-------------|-------|---------------------|
| robotHandle   | RobotHandle | 输入    | 机械臂实例句柄             |
| isRobotMoving | BOOL *      | 输出    | 机械臂当前是否运动：0-停止，1-运动 |

### 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
BOOL isRobotMoving;
result = cr_get_robotMoveStatus(robotHandle, &isRobotMoving); //读机械臂运动状态
if (result != success) //判断读机械臂运动状态是否成功
    return -1;
printf("%d\n", isRobotMoving);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.2.1.12 读取移动缓存区中移动指令的数量

### 函数

**CResult** cr\_move\_cache\_num\_get(**RobotHandle** robotHandle, int \*cacheNum)

### 简介

读取当前移动缓存区中移动指令的数量，当 PointControlPara 的 MotiontriggerMode 参数为 MovetriggerbyMoveCache 时，对应的移动指令会进入缓存区。

### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述         |
|-------------|-------------|-------|------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄    |
| cacheNum    | int *       | 输出    | 缓存区中移动指令数量 |

### 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
```

```

if (result != success) //判断机器人连接是否成功
    return -1;
int cacheNum;
result = cr_move_cache_num_get(robotHandle, &cacheNum);
//读取移动缓存区中移动指令的数量
if (result != success) //判断读取移动缓存区中移动指令的数量是否成功
    return -1;
printf("%d\n", cacheNum);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
    
```

## 3.2.2 运动状态信息

### 3.2.2.1 读实际 TCP 位置

#### 函数

**CRresult** cr\_get\_tcpActualPose(**RobotHandle** robotHandle, double pose[**ROB\_AXIS\_NUM**])

#### 简介

读实际 TCP 位置。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                                    |
|-------------|-------------|-------|---------------------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                               |
| pose        | double *    | 输出    | 待读取的值：x，y，z（单位：mm）；<br>Rx，Ry，Rz（单位：°） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double pose[ROB_AXIS_NUM];
result = cr_get_tcpActualPose(robotHandle, pose); //读实际 TCP 位置
if (result != success) //判断读实际 TCP 位置是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", pose[i]);
}
    
```

```
}  
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄  
if (result != success) //判断机器人断开连接是否成功  
    return -1;
```

### 3.2.2.2 读目标 TCP 位置

#### 函数

**CRresult** cr\_get\_tcpTargetPose(**RobotHandle** robotHandle, double pose[**ROB\_AXIS\_NUM**])

#### 简介

读目标 TCP 位置。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                                    |
|-------------|-------------|-------|---------------------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                               |
| pose        | double *    | 输出    | 待读取的值：x，y，z（单位：mm）；<br>Rx，Ry，Rz（单位：°） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;  
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接  
if (result != success) //判断机器人连接是否成功  
    return -1;  
double pose[ROB_AXIS_NUM];  
result = cr_get_tcpTargetPose(robotHandle, pose); //读目标 TCP 位置  
if (result != success) //判断读目标 TCP 位置是否成功  
    return -1;  
for (int i = 0; i < ROB_AXIS_NUM; i++)  
{  
    printf("%f\n", pose[i]);  
}  
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄  
if (result != success) //判断机器人断开连接是否成功  
    return -1;
```

### 3.2.2.3 读实际 TCP 速度

#### 函数

**CRresult** cr\_get\_tcpActualSpeed(**RobotHandle** robotHandle, double speed[**ROB\_AXIS\_NUM**])

## 简介

读实际 TCP 速度。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                                                 |
|-------------|-------------|-------|----------------------------------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                                            |
| speed       | double *    | 输出    | 待读取的值: x, y, z (单位: mm/s);<br>Rx, Ry, Rz (单位: °/s) |

## 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double speed[ROB_AXIS_NUM];
result = cr_get_tcpActualSpeed(robotHandle, speed); //读实际 TCP 速度
if (result != success) //判断读实际 TCP 速度是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", speed[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.2.4 读目标 TCP 速度

## 函数

**CRresult cr\_get\_tcpTargetSpeed(RobotHandle robotHandle, double speed[ROB\_AXIS\_NUM])**

## 简介

读目标 TCP 速度。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                                                 |
|-------------|-------------|-------|----------------------------------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                                            |
| speed       | double *    | 输出    | 待读取的值: x, y, z (单位: mm/s);<br>Rx, Ry, Rz (单位: °/s) |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double speed[ROB_AXIS_NUM];
result = cr_get_tcpTargetSpeed(robotHandle, speed); //读目标 TCP 速度
if (result != success) //判断读目标 TCP 速度是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", speed[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.2.5 读实际 TCP 加速度

## 函数

CRresult cr\_get\_tcpActualAcceleration(RobotHandle robotHandle, double acceleration[ROB\_AXIS\_NUM])

## 简介

读实际 TCP 加速度。

## 参数

| 参数名称         | 数据类型        | 输入/输出 | 描述                                                                           |
|--------------|-------------|-------|------------------------------------------------------------------------------|
| robotHandle  | RobotHandle | 输入    | 机械臂实例句柄                                                                      |
| acceleration | double *    | 输出    | 待读取的值：x, y, z (单位：mm/s <sup>2</sup> ) ;<br>Rx, Ry, Rz (单位：°/s <sup>2</sup> ) |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
```

```
double acceleration[ROB_AXIS_NUM];
result = cr_get_tcpActualAcceleration(robotHandle, acceleration); //读实际 TCP 加速度
if (result != success) //判断读实际 TCP 加速度是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", acceleration[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.2.6 读当前使用的 TCP 偏移信息

#### 函数

**CRresult** cr\_get\_currentTCPmsg(**RobotHandle** robotHandle, **TCPMsg \***tcpMsg)

#### 简介

读当前使用的 TCP 偏移信息。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述             |
|-------------|-------------|-------|----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄        |
| tcpMsg      | TCPMsg *    | 输出    | 当前使用的 TCP 偏移信息 |

#### 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
TCPMsg tcpMsgList;
result = cr_get_currentTCPmsg(robotHandle, &tcpMsgList); //读当前使用的 TCP 偏移信息
if (result != success) //判断读当前使用的 TCP 偏移信息是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", tcpMsgList.tcpOffset[i]);
}
printf("%s\n", tcpMsgList.tcpName);
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
```



```
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.2.7 读取当前使用负载信息

#### 函数

**CRresult** cr\_get\_currentPayloadmsg(**RobotHandle** robotHandle, **PayLoad** \*payloadMsg)

#### 简介

读取当前使用负载信息。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| payloadMsg  | PayLoad *   | 输出    | 当前负载信息  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
PayLoad payloadMsg;
result = cr_get_currentPayloadmsg(robotHandle, &payloadMsg); //读取当前使用负载信息
if (result != success) //判断读取当前使用负载信息是否成功
    return -1;
printf("%f\n", payloadMsg.toolPayload);
for (int i = 0; i < 3; i++)
{
    printf("%f\n", payloadMsg.centerOfGravity[i]);
}
printf("%s\n", payloadMsg.payloadName);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.2.3 运动配置

### 3.2.3.1 读所有 TCP 偏移信息列表

#### 函数

`CRresult cr_get_allTCPmsg(RobotHandle robotHandle, TCPMsg *tcpMsgList, int *validNumber)`

#### 简介

读所有的 TCP 偏移信息。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                 |
|-------------|-------------|-------|--------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄            |
| tcpMsgList  | TCPMsg *    | 输出    | 所有 TCP 偏移信息所在的数组   |
| validNumber | int *       | 输出    | TCP 偏移信息数组中有效数据的长度 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int validNumber;
TCPMsg tcpMsgList[20];
result = cr_get_allTCPmsg(robotHandle, tcpMsgList, &validNumber); //读所有的 TCP 偏移信息
if (result != success) //判断读所有的 TCP 偏移信息是否成功
    return -1;
for (int j = 0; j < validNumber; j++)
{
    for (int i = 0; i < ROB_AXIS_NUM; i++)
    {
        printf("%f\n", tcpMsgList[j].tcpOffset[i]);
    }
    printf("%s\n", tcpMsgList[j].tcpName);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.3.2 读取所有负载信息

#### 函数

```
CRresult cr_get_allPayloadmsg(RobotHandle robotHandle, PayLoad *payloadMsgList, int *validNumber)
```

#### 简介

读取所有的负载信息。

#### 参数

| 参数名称           | 数据类型        | 输入/输出 | 描述           |
|----------------|-------------|-------|--------------|
| robotHandle    | RobotHandle | 输入    | 机械臂实例句柄      |
| payloadMsgList | PayLoad *   | 输出    | 所有负载所在的数组    |
| validNumber    | int *       | 输出    | 负载数组中有效数据的长度 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int validNumber;
PayLoad payloadMsg[20];
result = cr_get_allPayloadmsg(robotHandle, payloadMsg, &validNumber);
//读取所有的负载信息
if (result != success) //判断读取所有的负载信息是否成功
    return -1;
for (int j = 0; j < validNumber; j++)
{
    printf("%f\n", payloadMsg[j].toolPayload);
    for (int i = 0; i < 3; i++)
    {
        printf("%f\n", payloadMsg[j].centerOfGravity[i]);
    }
    printf("%s\n", payloadMsg[j].payloadName);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.3.3 获取指定 TCPoffset 的位姿

#### 函数

```
CRresult cr_get_AssignTCP_Pose(RobotHandle robotHandle, double
dstTCPoffset[ROB_AXIS_NUM], double dstPose[ROB_AXIS_NUM])
```

#### 简介

获取指定 TCPoffset 的位姿。

#### 参数

| 参数名称         | 数据类型        | 输入/输出 | 描述                                                          |
|--------------|-------------|-------|-------------------------------------------------------------|
| robotHandle  | RobotHandle | 输入    | 机械臂实例句柄                                                     |
| dstTCPoffset | double *    | 输入    | 需变换后 TCP 偏移量: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)          |
| dstPose      | double *    | 输出    | 变化为 dstTCPoffset 后的位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °) |

#### 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double dstTCPoffset[ROB_AXIS_NUM] = { 10, 20, 30, 60, 60, 60 };
double dstPose[ROB_AXIS_NUM];
result = cr_get_AssignTCP_Pose(robotHandle, dstTCPoffset, dstPose);
//获取指定 TCPoffset 的位姿
if (result != success) //判断获取指定 TCPoffset 的位姿是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", dstPose[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.2.4 关节状态

### 3.2.4.1 读实际关节位置

#### 函数

`CRresult cr_get_jointActualPos(RobotHandle robotHandle, double pos[ROB_AXIS_NUM])`

#### 简介

读实际关节位置。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                 |
|-------------|-------------|-------|--------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄            |
| pos         | double *    | 输出    | 待读取的值，关节 1~6（单位：°） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double pos[ROB_AXIS_NUM];
result = cr_get_jointActualPos(robotHandle, pos); //读实际关节位置
if (result != success) //判断读实际关节位置是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", pos[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.4.2 读目标关节位置

#### 函数

`CRresult cr_get_jointTargetPos(RobotHandle robotHandle, double pos[ROB_AXIS_NUM])`

#### 简介

读目标关节位置。

### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                 |
|-------------|-------------|-------|--------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄            |
| pos         | double *    | 输出    | 待读取的值，关节 1~6（单位：°） |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double pos[ROB_AXIS_NUM];
result = cr_get_jointTargetPos(robotHandle, pos); //读目标关节位置
if (result != success) //判断读目标关节位置是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", pos[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.2.4.3 读实际关节速度

### 函数

CRresult cr\_get\_jointActualVelocity(RobotHandle robotHandle, double velocity[ROB\_AXIS\_NUM])

### 简介

读实际关节速度。

### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                   |
|-------------|-------------|-------|----------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄              |
| velocity    | double *    | 输出    | 待读取的值，关节 1~6（单位：°/s） |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double velocity[ROB_AXIS_NUM];
result = cr_get_jointActualVelocity(robotHandle,velocity); //读实际关节速度
if (result != success) //判断读实际关节速度是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", velocity[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.4.4 读目标关节速度

## 函数

**CResult** cr\_get\_jointTargetVelocity(**RobotHandle** robotHandle, double velocity[**ROB\_AXIS\_NUM**])

## 简介

读目标关节速度。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                   |
|-------------|-------------|-------|----------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄              |
| velocity    | double *    | 输出    | 待读取的值，关节 1~6（单位：°/s） |

## 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double velocity[ROB_AXIS_NUM];
result = cr_get_jointTargetVelocity(robotHandle,velocity); //读目标关节速度
if (result != success) //判断读目标关节速度是否成功
    return -1;
```

```
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", velocity[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.4.5 读实际关节加速度

#### 函数

**CRresult** cr\_get\_jointActualAcceleration(**RobotHandle** robotHandle, double acceleration[**ROB\_AXIS\_NUM**])

#### 简介

读实际关节加速度。

#### 参数

| 参数名称         | 数据类型        | 输入/输出 | 描述                                 |
|--------------|-------------|-------|------------------------------------|
| robotHandle  | RobotHandle | 输入    | 机械臂实例句柄                            |
| acceleration | double *    | 输出    | 待读取的值，关节 1~6（单位：°/s <sup>2</sup> ） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double acceleration[ROB_AXIS_NUM];
result = cr_get_jointActualAcceleration(robotHandle, acceleration); //读实际关节加速度
if (result != success) //判断读实际关节加速度是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", acceleration[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```



### 3.2.4.6 读目标关节加速度

#### 函数

```
CRresult cr_get_jointTargetAcceleration(RobotHandle robotHandle, double  
acceleration[ROB_AXIS_NUM])
```

#### 简介

读目标关节加速度。

#### 参数

| 参数名称         | 数据类型        | 输入/输出 | 描述                                 |
|--------------|-------------|-------|------------------------------------|
| robotHandle  | RobotHandle | 输入    | 机械臂实例句柄                            |
| acceleration | double *    | 输出    | 待读取的值，关节 1~6（单位：°/s <sup>2</sup> ） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;  
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接  
if (result != success) //判断机器人连接是否成功  
    return -1;  
double acceleration[ROB_AXIS_NUM];  
result = cr_get_jointTargetAcceleration(robotHandle, acceleration); //读目标关节加速度  
if (result != success) //判断读目标关节加速度是否成功  
    return -1;  
for (int i = 0; i < ROB_AXIS_NUM; i++)  
{  
    printf("%f\n", acceleration[i]);  
}  
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄  
if (result != success) //判断机器人断开连接是否成功  
    return -1;
```

### 3.2.4.7 读实际关节电机电流

#### 函数

```
CRresult cr_get_jointActualCurrent(RobotHandle robotHandle, double  
current[ROB_AXIS_NUM])
```

#### 简介

读实际关节电机电流（数值和关节转矩等效，三关节符号相反）。

### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                  |
|-------------|-------------|-------|---------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄             |
| current     | double *    | 输出    | 待读取的值，关节 1~6（单位：mA） |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double current[ROB_AXIS_NUM];
result = cr_get_jointActualCurrent(robotHandle, current); //读实际关节电机电流
if (result != success) //判断读实际关节电机电流是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", current[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.2.4.8 读目标关节电机电流

### 函数

```
CRresult cr_get_jointTargetCurrent(RobotHandle robotHandle, double
current[ROB_AXIS_NUM])
```

### 简介

读目标关节电机电流（数值和关节转矩等效，三关节符号相反）。

### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                  |
|-------------|-------------|-------|---------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄             |
| current     | double *    | 输出    | 待读取的值，关节 1~6（单位：mA） |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123");
if (result != success)
    return -1;
double current[ROB_AXIS_NUM];
result = cr_get_jointTargetCurrent(robotHandle, current); //读目标关节电机电流
if (result != success) //判断读目标关节电机电流是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", current[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.4.9 读目标关节转矩

#### 函数

**CResult** cr\_get\_jointTargetTorque(**RobotHandle** robotHandle, double torque[**ROB\_AXIS\_NUM**])

#### 简介

读目标关节转矩。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                             |
|-------------|-------------|-------|--------------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                        |
| torque      | double *    | 输出    | 待读取的值，关节 1~6<br>(单位：额定转矩/1000) |

#### 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double torque[ROB_AXIS_NUM];
result = cr_get_jointTargetTorque(robotHandle, torque); //读目标关节转矩
if (result != success) //判断读目标关节转矩是否成功
```

```

        return -1;
    for (int i = 0; i < ROB_AXIS_NUM; i++)
    {
        printf("%f\n", torque[i]);
    }
    result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
    if (result != success) //判断机器人断开连接是否成功
        return -1;

```

### 3.2.4.10 读实际关节采集的母线电压

#### 函数

**CRresult** cr\_get\_jointActualVoltage(**RobotHandle** robotHandle, double voltage[**ROB\_AXIS\_NUM**])

#### 简介

读实际关节采集的母线电压。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                 |
|-------------|-------------|-------|--------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄            |
| voltage     | double *    | 输出    | 待读取的值，关节 1~6（单位：V） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double voltage[ROB_AXIS_NUM];
result = cr_get_jointActualVoltage(robotHandle, voltage); //读实际关节采集的母线电压
if (result != success) //判断读实际关节采集的母线电压是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", voltage[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;

```

### 3.2.4.11 读关节温度

#### 函数

```
CRresult cr_get_jointTemperature(RobotHandle robotHandle, double  
temperature[ROB_AXIS_NUM])
```

#### 简介

读关节温度。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                  |
|-------------|-------------|-------|---------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄             |
| temperature | double *    | 输出    | 待读取的值，关节 1~6（单位：°C） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;  
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接  
if (result != success) //判断机器人连接是否成功  
    return -1;  
double temperature[ROB_AXIS_NUM];  
result = cr_get_jointTemperature(robotHandle, temperature); //读关节温度  
if (result != success) //判断读关节温度是否成功  
    return -1;  
for (int i = 0; i < ROB_AXIS_NUM; i++)  
{  
    printf("%f\n", temperature[i]);  
}  
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄  
if (result != success) //判断机器人断开连接是否成功  
    return -1;
```

### 3.2.4.12 读关节模式

#### 函数

```
CRresult cr_get_jointMode(RobotHandle robotHandle, enum JointModes  
jointMode[ROB_AXIS_NUM])
```

#### 简介

读关节模式。

## 参数

| 参数名称        | 数据类型         | 输入/输出 | 描述      |
|-------------|--------------|-------|---------|
| robotHandle | RobotHandle  | 输入    | 机械臂实例句柄 |
| jointMode   | JointModes * | 输出    | 关节模式    |

## 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
enum JointModes jointMode[ROB_AXIS_NUM];
result = cr_get_jointMode(robotHandle, jointMode); //读关节模式
if (result != success) //判断读关节模式是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%d\n", jointMode[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.2.5 轨迹复现

### 3.2.5.1 读取控制柜当前存在的轨迹索引

## 函数

CResult cr\_path\_all\_index\_get(RobotHandle robotHandle, int \*allPathIndex, int \*pathIndexLen)

## 简介

读取控制柜当前存在的轨迹索引。

## 参数

| 参数名称         | 数据类型        | 输入/输出 | 描述            |
|--------------|-------------|-------|---------------|
| robotHandle  | RobotHandle | 输入    | 机械臂实例句柄       |
| allPathIndex | int *       | 输出    | 所有轨迹索引，最大为 51 |
| pathIndexLen | int *       | 输出    | 轨迹索引数组有效长度    |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int allPathIndex[51];
int pathIndexLen;
result = cr_path_all_index_get(robotHandle, allPathIndex, &pathIndexLen);
//读取控制柜当前存在的轨迹索引
if (result != success) //判断读取控制柜当前存在的轨迹索引是否成功
    return -1;
for (int i = 0; i < pathIndexLen; i++)
{
    printf("%d\n", allPathIndex[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.5.2 上传轨迹

## 函数

CRresult cr\_path\_upload(RobotHandle robotHandle, int index, PathData \*pathData)

## 简介

上传轨迹。运行前可通过 cr\_path\_all\_index\_get 先判断 index 是否存在。上传轨迹的请求索引的取值范围为：-1~49。其中 0~49 索引下的轨迹保存在控制器 Flash 中，-1 索引下的轨迹数据是上一次轨迹记录控制功能中记录下的轨迹数据，存放在控制器 RAM 中，下电清空。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述         |
|-------------|-------------|-------|------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄    |
| index       | int         | 输入    | 轨迹索引：-1~49 |
| pathData    | PathData *  | 输出    | 上传的轨迹数据    |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
PathData pathData;
pathData.pathPoints = new PathPoint[10000];
result = cr_path_upload(robotHandle, index, &pathData);
//上传索引为 1 的轨迹，使用前需判断索引为 1 的轨迹是否存在
if (result != success) //上传轨迹是否成功
    return -1;
printf("%f\n", pathData.pathPoints[0].jointpos[0]);
printf("%d\n", pathData.pathPointsNum);
delete[] pathData.pathPoints; //内存释放
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.5.3 下载轨迹

## 函数

**CResult** cr\_path\_download(**RobotHandle** robotHandle, **PathDownloadData** pathDownloadData)

## 简介

下载轨迹。该功能会覆盖控制柜已存在的轨迹，建议运行前通过 cr\_path\_all\_index\_get 接口先判断该索引是否存在。轨迹下载接口是多个功能复用的接口，其支持 TCP 轨迹复现、关节轨迹复现和分段 movex 功能。当 moveType 为 0 时，调用该接口时机器人会直接运行，提供的轨迹数据需要按照 2ms 规划，运行轨迹之前需将机器人移动到轨迹的第一个点位；当 moveType 为 1 时为 TCP 轨迹复现，需要调用 cr\_path\_action 接口来运行轨迹；当 moveType 为 4 时为关节轨迹复现，需要调用 cr\_path\_action 接口来运行轨迹；当 moveType 为 3 时，movex 运动，需要设置速度、加速度、交融半径，需要调用 cr\_path\_action 接口来运行轨迹。

**注：**在使用相关功能时，需要保证机械臂已经移动到轨迹起始点对应的位置。

## 参数

| 参数名称             | 数据类型             | 输入/输出 | 描述      |
|------------------|------------------|-------|---------|
| robotHandle      | RobotHandle      | 输入    | 机械臂实例句柄 |
| pathDownloadData | PathDownloadData | 输入    | 下载轨迹数据  |

## 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。



**示例 1：下载轨迹**

```

RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char filePath[100] = "D:/file/path.cpath";
PathData pathData;
pathData.pathPoints = new PathPoint[10000];
result = cr_path_file2pathData(filePath, &pathData); //轨迹文件转换为轨迹数据
if (result != success) //判断轨迹文件转换为轨迹数据是否成功
    return -1;
PathDownloadData pathDownloadData;
pathDownloadData.pathData = pathData;
pathDownloadData.pathPara.index = 1;
pathDownloadData.pathPara.moveType = 1;
result = cr_path_download(robotHandle, pathDownloadData); //下载轨迹到索引 1
if (result != success) //判断下载轨迹是否成功
    return -1;
delete[] pathData.pathPoints; //内存释放
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;

```

**示例 2：TCP 轨迹复现**

(1) 调用 `cr_path_download` 接口将所有 TCP 位置和关节位置下载到机械臂，运动轨迹类型 `movetype` 写 1，运动时间 `moveTime` 写对应执行周期

(2) 调用 `cr_path_action` 执行对应索引的轨迹，`runContol` 参数写 1

```

RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char filePath[100] = "D:/file/path.cpath";
PathData pathData;
pathData.pathPoints = new PathPoint[10000];
result = cr_path_file2pathData(filePath, &pathData); //轨迹文件转换为轨迹数据
if (result != success) //判断轨迹文件转换为轨迹数据是否成功
    return -1;
PathDownloadData pathDownloadData;
pathDownloadData.pathData = pathData;
pathDownloadData.pathPara.index = 1;
pathDownloadData.pathPara.moveType = 1;
result = cr_path_download(robotHandle, pathDownloadData); //下载轨迹到索引 1
if (result != success) //判断下载轨迹是否成功

```

```

    return -1;
PointControlParaSimple pointControlParaSimple;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    pointControlParaSimple.pose[i] = pathData.pathPoints[0].pose[i];
    pointControlParaSimple.jointpos[i] = pathData.pathPoints[0].jointpos[i];
    pointControlParaSimple.tcpOffset[i] = 0;
    pointControlParaSimple.coordinatePose[i] = 0;
    pointControlParaSimple.speed[i] = 30;
    pointControlParaSimple.acc[i] = 30;
}
pointControlParaSimple.coordinateType = jointCoordinate;
pointControlParaSimple.pointTransType = pointTransStop;
pointControlParaSimple.motiontriggerMode = MovetriggerbyOnlyRpc;
PointControlPara pointControlPara;
result = cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple,
&pointControlPara);
// pointControlParaSimple 转化为 pointControlPara
if (result != success) //判断 pointControlParaSimple 转化为 pointControlPara 是否成功
    return -1;
result = cr_move_joint(robotHandle, pointControlPara, TRUE); //移动至第一个点位
if (result != success) //判断移动至第一个点位是否成功
    return -1;
Sleep(200);
result = cr_path_action(robotHandle, pathDownloadData.pathPara.index, 1); //运行轨迹索引 1
if (result != success) //判断运行轨迹是否成功
    return -1;
delete[] pathData.pathPoints; //内存释放
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;

```

### 示例 3：关节轨迹复现

- (1) 调用 cr\_path\_download 接口将所有关节位置下载到机械臂, 运动轨迹类型 movetype 写 4, 运动时间 moveTime 写对应的执行周期
- (2) 调用 cr\_path\_action 执行对应索引的轨迹, runContol 参数写 1

```

RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char filePath[100] = "D:/file/path.crpah";
PathData pathData;
pathData.pathPoints = new PathPoint[10000];
result = cr_path_file2pathData(filePath, &pathData); //轨迹文件转换为轨迹数据

```

```

if (result != success) //判断轨迹文件转换为轨迹数据是否成功
    return -1;
PathDownloadData pathDownloadData;
pathDownloadData.pathData = pathData;
pathDownloadData.pathPara.index = 1;
pathDownloadData.pathPara.moveType = 4;
result = cr_path_download(robotHandle, pathDownloadData); //下载轨迹到索引 1
if (result != success) //判断下载轨迹是否成功
    return -1;
PointControlParaSimple pointControlParaSimple;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    pointControlParaSimple.pose[i] = pathData.pathPoints[0].pose[i];
    pointControlParaSimple.jointpos[i] = pathData.pathPoints[0].jointpos[i];
    pointControlParaSimple.tcpOffset[i] = 0;
    pointControlParaSimple.coordinatePose[i] = 0;
    pointControlParaSimple.speed[i] = 30;
    pointControlParaSimple.acc[i] = 30;
}
pointControlParaSimple.coordinateType = jointCoordinate;
pointControlParaSimple.pointTransType = pointTransStop;
pointControlParaSimple.motiontriggerMode = MovetriggerbyOnlyRpc;
PointControlPara pointControlPara;
result = cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple,
&pointControlPara);
// pointControlParaSimple 转化为 pointControlPara
if (result != success) //判断 pointControlParaSimple 转化为 pointControlPara 是否成功
    return -1;
result = cr_move_joint(robotHandle, pointControlPara, TRUE); //移动至第一个点位
if (result != success) //判断移动至第一个点位是否成功
    return -1;
Sleep(200);
result = cr_path_action(robotHandle, pathDownloadData.pathPara.index, 1); //运行轨迹索引 1
if (result != success) //判断运行轨迹是否成功
    return -1;
delete[] pathData.pathPoints; //内存释放
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;

```

#### 示例 4：分段 moveX

(1) 调用 `cr_path_download` 接口将部分 TCP 位置和关节位置下载到机械臂，运动轨迹类型 `movetype` 写 3，运动时间 `moveTime` 写 2，速度 `speed` 写 `movex` 的运动速度，加速度 `acc` 写 `movex` 的加速度，交融半径 `blendRadius` 写 `movex` 的交融半径

(2) 调用 `cr_path_action` 执行对应索引的分段 Movex, `runContol` 参数写 1

(3) 多次调用 `cr_path_download` 接口写入后续的 TCP 位置和关节位置轨迹, 参数和第一步中的参数一致

(4) 调用 `cr_path_action` 关闭分段 Movex, `runControl` 参数写 0

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
PathData pathData;
pathData.pathPoints = new PathPoint[10000];
PathDownloadData pathDownloadData;
double joints[6], pose[6], startPose[6];
result = cr_get_jointActualPos(robotHandle, joints);
if (result != success)
    return -1;
result = cr_get_tcpActualPose(robotHandle, startPose);
if (result != success)
    return -1;
memcpy(pose, startPose, sizeof(startPose));
// 下载第一段 Movex 轨迹
for (int i = 0; i < 5; i++)
{
    pose[0] = startPose[0] + pow(-1, i) * 50.0;
    pose[1] = startPose[1] + 50.0 * i;
    memcpy(pathData.pathPoints[i].pose, pose, sizeof(pose));
    memcpy(pathData.pathPoints[i].jointpos, joints, sizeof(joints));
}
pathData.pathPointsNum = 5;
pathData.moveTime = 2;
pathDownloadData.pathData = pathData;
pathDownloadData.pathPara.index = 1;
pathDownloadData.pathPara.moveType = 3; // 分段 Movex 类型
pathDownloadData.pathPara.speed = 50; // Movex 速度 mm/s
pathDownloadData.pathPara.acc = 100; // Movex 加速度 mm/s^2
pathDownloadData.pathPara.blendRadius = 10; // Movex 交融半径 mm
result = cr_path_download(robotHandle, pathDownloadData);
if (result != success)
    return -1;
// 开启分段 Movex, runControl 写 1
result = cr_path_action(robotHandle, pathDownloadData.pathPara.index, 1);
if (result != success)
    return -1;
// 等待 5s, 下载第二段轨迹
Sleep(5000);
```

```

for (int i = 0; i < 5; i++)
{
    pose[0] = startPose[0] - 50.0 * i;
    pose[1] = startPose[1] + pow(-1, i) * 50.0;
    memcpy(pathData.pathPoints[i].pose, pose, sizeof(pose));
    memcpy(pathData.pathPoints[i].jointpos, joints, sizeof(joints));
}
pathData.pathPointsNum = 5;
pathDownloadData.pathData = pathData;
result = cr_path_download(robotHandle, pathDownloadData);
if (result != success)
    return -1;
// 等待 5s, 下载第三段轨迹
Sleep(5000);
for (int i = 0; i < 5; i++)
{
    pose[0] = startPose[0] - pow(-1, i) * 50.0;
    pose[2] = startPose[2] - 50.0 * i;
    memcpy(pathData.pathPoints[i].pose, pose, sizeof(pose));
    memcpy(pathData.pathPoints[i].jointpos, joints, sizeof(joints));
}
pathData.pathPointsNum = 5;
pathDownloadData.pathData = pathData;
result = cr_path_download(robotHandle, pathDownloadData);
if (result != success)
    return -1;
// 关闭分段 Movex, runControl 写 0
result = cr_path_action(robotHandle, pathDownloadData.pathPara.index, 0);
if (result != success)
    return -1;
delete[] pathData.pathPoints;
if (result != success) //判断机器人断开连接是否成功
    return -1;

```

### 3.2.5.4 轨迹控制

#### 函数

**CRresult** cr\_path\_action(**RobotHandle** robotHandle,int pathIndex,int runControl)

#### 简介

轨迹控制。运行前可通过 cr\_path\_all\_index\_get 先判断轨迹索引 pathIndex 是否存在。启动轨迹之前需将机器人移动到轨迹的第一个点位。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述               |
|-------------|-------------|-------|------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄          |
| pathIndex   | int         | 输入    | 轨迹索引 0~49        |
| runControl  | int         | 输入    | 0-停止, 1-启动, 2-暂停 |

## 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

## 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
PathData pathData;
pathData.pathPoints = new PathPoint[10000];
result = cr_path_upload(robotHandle, index, &pathData);
//上传索引为 1 的轨迹, 使用前需判断索引为 1 的轨迹是否存在
if (result != success) //判断上传索引是否成功
    return -1;

PointControlParaSimple pointControlParaSimple;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    pointControlParaSimple.pose[i] = pathData.pathPoints[0].pose[i];
    pointControlParaSimple.jointpos[i] = pathData.pathPoints[0].jointpos[i];
    pointControlParaSimple.tcpOffset[i] = 0;
    pointControlParaSimple.coordinatePose[i] = 0;
    pointControlParaSimple.speed[i] = 30;
    pointControlParaSimple.acc[i] = 30;
}
pointControlParaSimple.coordinateType = jointCoordinate;
pointControlParaSimple.pointTransType = pointTransStop;
pointControlParaSimple.motiontriggerMode = MovetriggerbyOnlyRpc;
PointControlPara pointControlPara;
result = cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple,
&pointControlPara);
// pointControlParaSimple 转化为 pointControlPara
if (result != success) //判断 pointControlParaSimple 转化为 pointControlPara 是否成功
    return -1;
result = cr_move_joint(robotHandle, pointControlPara, TRUE); //移动至第一个点位
if (result != success) //判断移动至第一个点位是否成功
    return -1;

```

```
Sleep(200);

result = cr_path_action(robotHandle, index, 1);
//启动轨迹索引为 1 的轨迹，使用前需判断索引为 1 的轨迹是否存在
if (result != success) //判断轨迹索引为 1 的轨迹是否开启成功
    return -1;
delete[] pathData.pathPoints; //释放内存
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.5.5 轨迹数据转换为轨迹文件

#### 函数

**CRresult** cr\_path\_pathData2file(**PathData** pathData, char \*filePath)

#### 简介

轨迹数据转换为轨迹文件。

#### 参数

| 参数名称     | 数据类型     | 输入/输出 | 描述     |
|----------|----------|-------|--------|
| pathData | PathData | 输入    | 轨迹数据   |
| filePath | char *   | 输出    | 文件保存路径 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char filePath[100] = "D:/file/path.crpah";
PathData pathData;
pathData.pathPoints = new PathPoint[10000];
result = cr_path_upload(robotHandle, 1, &pathData);
//上传索引为 1 的轨迹，使用前需判断索引为 1 的轨迹是否存在
if (result != success) //判断轨迹是否上传成功
    return -1;
result = cr_path_pathData2file(pathData, filePath); //轨迹数据转换为轨迹文件
if (result != success) //判断轨迹数据转换为轨迹文件是否成功
    return -1;
delete[] pathData.pathPoints; //内存释放
```

```
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.5.6 轨迹文件转换为轨迹数据

#### 函数

**CRresult** cr\_path\_file2pathData(char \*filePath, **PathData** \*pathData)

#### 简介

轨迹文件转换为轨迹数据。

#### 参数

| 参数名称     | 数据类型       | 输入/输出 | 描述   |
|----------|------------|-------|------|
| filePath | char *     | 输入    | 文件路径 |
| pathData | PathData * | 输出    | 轨迹数据 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char filePath[100] = "D:/file/path.crpah";
PathData pathData;
pathData.pathPoints = new PathPoint[10000];
result = cr_path_file2pathData(filePath, &pathData); //轨迹文件转换为轨迹数据
if (result != success) //判断轨迹文件转换为轨迹数据是否成功
    return -1;
printf("%d\n", pathData.pathPointsNum);
delete[] pathData.pathPoints; //内存释放
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.5.7 设置轨迹记录参数

#### 函数

**CRresult** cr\_path\_recordPara\_set(**RobotHandle** robotHandle, **RecordPathPara** recordPathPara)



## 简介

设置轨迹记录参数。

## 参数

| 参数名称           | 数据类型           | 输入/输出 | 描述      |
|----------------|----------------|-------|---------|
| robotHandle    | RobotHandle    | 输入    | 机械臂实例句柄 |
| recordPathPara | RecordPathPara | 输入    | 轨迹记录参数  |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
RecordPathPara recordPathPara;
recordPathPara.sampleTime = 2;
recordPathPara.recordControl = 1;
result = cr_path_recordPara_set(robotHandle, recordPathPara); //设置轨迹记录参数
if (result != success) //判断设置轨迹记录参数是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.5.8 获取当前轨迹记录状态

## 函数

```
CRresult cr_path_recordStatus_get(RobotHandle robotHandle, PathRecordStatus
*pathRecordStatus)
```

## 简介

获取当前轨迹记录状态。

## 参数

| 参数名称             | 数据类型               | 输入/输出 | 描述       |
|------------------|--------------------|-------|----------|
| robotHandle      | RobotHandle        | 输入    | 机械臂实例句柄  |
| pathRecordStatus | PathRecordStatus * | 输出    | 当前轨迹记录状态 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
PathRecordStatus pathRecordStatus;
result = cr_path_recordStatus_get(robotHandle, &pathRecordStatus); //获取当前轨迹记录状态
if (result != success) //判断获取当前轨迹记录状态是否成功
    return -1;
printf("%d\n", pathRecordStatus.recordStatus);
printf("%d\n", pathRecordStatus.waypointNumber);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.5.9 获取当前轨迹运行状态

## 函数

**CResult** cr\_path\_currentRunStatus\_get(**RobotHandle** robotHandle, **PathRunMsg \***pathRunMsg)

## 简介

获取当前轨迹运行状态。

## 参数

| 参数名称        | 数据类型         | 输入/输出 | 描述       |
|-------------|--------------|-------|----------|
| robotHandle | RobotHandle  | 输入    | 机械臂实例句柄  |
| pathRunMsg  | PathRunMsg * | 输出    | 当前轨迹运行状态 |

## 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
PathRunMsg pathRunMsg;
result = cr_path_currentRunStatus_get(robotHandle, &pathRunMsg); //获取当前轨迹运行状态
if (result != success) //判断获取当前轨迹运行状态是否成功
    return -1;
printf("%d\n", pathRunMsg.pathrunstatus);
printf("%d\n", pathRunMsg.pointIndex);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
```

```
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.2.6 扩展轴运动

### 3.2.6.1 扩展轴关节移动

#### 函数

**CRresult** cr\_move\_joint\_exj(**RobotHandle** robotHandle, **MoveExjPara** moveExjPara, **BOOL** isBlock)

#### 简介

扩展轴关节移动。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                |
|-------------|-------------|-------|-------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄           |
| moveExjPara | MoveExjPara | 输入    | 扩展轴关节移动相关参数       |
| isBlock     | BOOL        | 输入    | TRUE-阻塞 FALSE-非阻塞 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double jointPos[12] = { 60, 60, -60, 60, 60, 60, 300, 200, 0, 0, 0, 0 };
//前六个代表关节 1~6 关节角度，后六个代表扩展轴位置
double pose[12] = { 110, -613, 1129, -57.34, 0, 148.98, 0, 0, 0, 0, 0, 0 }; //前六个代表关节姿态
MoveExjPara moveExjPara;
for (int index = 0; index < ROB_EXJ_AXIS_NUM; index++)
{
    moveExjPara.pose[index] = pose[index];
    moveExjPara.jointPos[index] = jointPos[index];
    if (index < ROB_AXIS_NUM)
    {
        moveExjPara.speed[index] = 60;
        moveExjPara.acc[index] = 100;
        moveExjPara.tcpOffset[index] = 0;
        moveExjPara.coordinatePose[index] = 0;
    }
}
```

```
moveExjPara.speed[6] = 100; //扩展轴直线速度
moveExjPara.speed[7] = 200; //扩展轴旋转速度
moveExjPara.tcpId = -1;
moveExjPara.coordinateType = jointCoordinate;
moveExjPara.pointTransRadius = 10;
moveExjPara.motiontriggerMode = MovetriggerbyOnlyRpc;
result = cr_move_joint_exj(robotHandle, moveExjPara, TRUE); //扩展轴关节运动
if (result != success) //判断扩展轴关节运动是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.6.2 扩展轴直线移动

#### 函数

**CRresult** cr\_move\_line\_exj(**RobotHandle** robotHandle, **MoveExjPara** moveExjPara, **BOOL** isBlock)

#### 简介

扩展轴直线移动。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                |
|-------------|-------------|-------|-------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄           |
| moveExjPara | MoveExjPara | 输入    | 扩展轴直线移动相关参数       |
| isBlock     | BOOL        | 输入    | TRUE-阻塞 FALSE-非阻塞 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double jointPos[12] = { 60, 60, -60, 60, 60, 60, 0, 0, 0, 0, 0, 0 };
//前六个代表关节 1~6 关节角度
double pose[12] = { 110, -613, 1129, -57.34, 0, 148.98, 300, 200, 0, 0, 0, 0 };
//前六个代表关节姿态,后六个代表扩展轴位置
MoveExjPara moveExjPara;
for (int index = 0; index < ROB_EXJ_AXIS_NUM; index++)
{
```

```

moveExjPara.pose[index] = pose[index];
moveExjPara.jointPos[index] = jointPos[index];
if (index < ROB_AXIS_NUM)
{
    moveExjPara.speed[index] = 60;
    moveExjPara.acc[index] = 100;
    moveExjPara.tcpOffset[index] = 0;
    moveExjPara.coordinatePose[index] = 0;
}
}
moveExjPara.speed[6] = 100; //扩展轴直线速度
moveExjPara.speed[7] = 200; //扩展轴旋转速度
moveExjPara.tcpId = -1;
moveExjPara.coordinateType = baseCoordinate;
moveExjPara.pointTransRadius = 10;
moveExjPara.motiontriggerMode = MovetriggerbyOnlyRpc;
result = cr_move_line_exj(robotHandle, moveExjPara, TRUE); //扩展轴直线运动
if (result != success) //判断扩展轴直线运动是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;

```

### 3.2.6.3 扩展轴点动运动

#### 函数

**CRresult** cr\_moveJog\_exj(**RobotHandle** robotHandle, **MoveJogExjPara** moveJogExjPara)

#### 简介

扩展轴点动运动。

#### 参数

| 参数名称           | 数据类型           | 输入/输出 | 描述          |
|----------------|----------------|-------|-------------|
| robotHandle    | RobotHandle    | 输入    | 机械臂实例句柄     |
| moveJogExjPara | MoveJogExjPara | 输入    | 扩展轴点动运动相关参数 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;

```

```
MoveJogExjPara moveJogExjPara;
for (int index = 0; index < EXTERNAL_MAX_NUM; index++)
{
    moveJogExjPara.acc[index] = 100;
    moveJogExjPara.speed[index] = 50;
    moveJogExjPara.jointPos[index] = 0;
}
moveJogExjPara.jointPos[1] = 20;
result = cr_moveJog_exj(robotHandle, moveJogExjPara); //扩展轴点动运动
if (result != success) //判断扩展轴点动运动是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.2.6.4 扩展轴运动

#### 函数

**CRresult** cr\_exj\_move(**RobotHandle** robotHandle, **ExjMovePara** exjMovePara, **BOOL** isBlock)

#### 简介

扩展轴运动。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                |
|-------------|-------------|-------|-------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄           |
| exjMovePara | ExjMovePara | 输入    | 扩展轴点动运动相关参数       |
| isBlock     | BOOL        | 输入    | TRUE-阻塞 FALSE-非阻塞 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
ExjMovePara exjMovePara;
exjMovePara.exjCount = 1;
exjMovePara.acctime = 100.0;
exjMovePara.index[0] = 0;
exjMovePara.position[0] = 100.0;
exjMovePara.speed[0] = 50.0;
exjMovePara.motiontriggerMode = MovetriggerbyOnlyRpc;
```

```
result = cr_exj_move(robotHandle, exjMovePara, TRUE); //扩展轴运动
if (result != success) //判断扩展轴运动是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.3 系统属性

### 3.3.1 硬件信息

#### 3.3.1.1 读取控制柜硬件序列号

##### 函数

**CRresult** cr\_get\_productInfo(**RobotHandle** robotHandle, char \*productSn)

##### 简介

读取控制柜硬件序列号。

##### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述          |
|-------------|-------------|-------|-------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄     |
| productSn   | char *      | 输出    | 读取的控制柜硬件序列号 |

##### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

##### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char productSn[24];
result = cr_get_productInfo(robotHandle, productSn); //读取控制柜硬件序列号
if (result != success) //判断读取控制柜硬件序列号是否成功
    return -1;
printf("%s\n", productSn);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.3.1.2 读取整臂序列号

#### 函数

`CRresult cr_get_productSn(RobotHandle robotHandle, char *Sn)`

#### 简介

读取整臂序列号。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述       |
|-------------|-------------|-------|----------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄  |
| Sn          | char *      | 输出    | 读取的整臂序列号 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char Sn[24];
result = cr_get_productSn(robotHandle, Sn); //读取整臂序列号
if (result != success) //判断读取整臂序列号是否成功
    return -1;
printf("%s\n", Sn);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.3.1.3 获取版本信息

#### 函数

`CRresult cr_get_sysVersion(RobotHandle robotHandle, SysVersion *version)`

#### 简介

获取版本信息。

#### 参数

| 参数名称        | 数据类型         | 输入/输出 | 描述      |
|-------------|--------------|-------|---------|
| robotHandle | RobotHandle  | 输入    | 机械臂实例句柄 |
| version     | SysVersion * | 输出    | 返回版本号信息 |



## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SysVersion version;
result = cr_get_sysVersion( robotHandle, &version); //获取版本信息
if (result != success) //判断获取版本信息是否成功
    return -1;
printf("%s\n", version.jointVersion->hardwareID);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.3.2 本体参数

### 3.3.2.1 读机械臂 DH 参数

## 函数

```
CRresult cr_get_DhParmeter(RobotHandle robotHandle, double
dhPara[DH_PARAM_MAX_NUM])
```

## 简介

读机械臂 DH 参数。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述        |
|-------------|-------------|-------|-----------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄   |
| dhPara      | double *    | 输出    | 机械臂 DH 参数 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double dhPara[DH_PARAM_MAX_NUM];
result = cr_get_DhParmeter(robotHandle, dhPara); //读机械臂 DH 参数
```

```
if (result != success) //判断读机械臂 DH 参数是否成功
    return -1;
for (int i = 0; i < DH_PARAM_MAX_NUM; i++)
{
    printf("%f\n", dhPara[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.3.2.1 读机械臂标准 DH 参数

#### 函数

**CRresult** cr\_get\_stdDhParmeter(**RobotHandle** robotHandle, double dhPara[DH\_PARAM\_MAX\_NUM])

#### 简介

读机械臂标准 DH 参数。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述        |
|-------------|-------------|-------|-----------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄   |
| dhPara      | double *    | 输出    | 机械臂 DH 参数 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double dhPara[DH_PARAM_MAX_NUM];
result = cr_get_stdDhParmeter(robotHandle, dhPara); //读机械臂标准 DH 参数
if (result != success) //判断读机械臂标准 DH 参数是否成功
    return -1;
for (int i = 0; i < DH_PARAM_MAX_NUM; i++)
{
    printf("%f\n", dhPara[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.3.2.2 读机械臂动力学参数

#### 函数

**CRresult** cr\_get\_DynamicParmeter(**RobotHandle** robotHandle, double dynamicPara[**DY\_PARA\_MAX\_NUM**])

#### 简介

读机械臂动力学参数。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述       |
|-------------|-------------|-------|----------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄  |
| dynamicPara | double *    | 输出    | 机械臂动力学参数 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double dynamicPara[DY_PARA_MAX_NUM];
result = cr_get_DynamicParmeter(robotHandle, dynamicPara); //读机械臂动力学参数
if (result != success) //判断读机械臂动力学参数是否成功
    return -1;
for (int i = 0; i < DH_PARA_MAX_NUM; i++)
{
    printf("%f\n", dynamicPara[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.3.2.3 读机械臂零位补偿值

#### 函数

**CRresult** cr\_get\_zeroCompensationOffset(**RobotHandle** robotHandle, double zeroCompensationOffset[**ROB\_AXIS\_NUM**])

#### 简介

读机械臂零位补偿值。

### 参数

| 参数名称                   | 数据类型        | 输入/输出 | 描述       |
|------------------------|-------------|-------|----------|
| robotHandle            | RobotHandle | 输入    | 机械臂实例句柄  |
| zeroCompensationOffset | double *    | 输出    | 机械臂零位补偿值 |

### 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double zeroCompensationOffset[ROB_AXIS_NUM];
result = cr_get_zeroCompensationOffset(robotHandle, zeroCompensationOffset);
//读机械臂零位补偿值
if (result != success) //判断读机械臂零位补偿值是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", zeroCompensationOffset[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.3.2.4 读机械臂减速比

### 函数

**CResult** cr\_get\_reducerRatio(**RobotHandle** robotHandle, double reducerRatio[**ROB\_AXIS\_NUM**])

### 简介

读机械臂减速比。

### 参数

| 参数名称         | 数据类型        | 输入/输出 | 描述      |
|--------------|-------------|-------|---------|
| robotHandle  | RobotHandle | 输入    | 机械臂实例句柄 |
| reducerRatio | double *    | 输出    | 机械臂减速比  |

### 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double reducerRatio[ROB_AXIS_NUM];
result = cr_get_reducerRatio(robotHandle,reducerRatio); //读机械臂减速比
if (result != success) //判断读机械臂减速比是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", reducerRatio[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.3.3 系统信息

### 3.3.3.1 读机械臂当前状态

#### 函数

**CResult** cr\_get\_robotMode(**RobotHandle** robotHandle, enum **RobotModes** \*robotMode)

#### 简介

读机械臂当前状态。

#### 参数

| 参数名称        | 数据类型         | 输入/输出 | 描述      |
|-------------|--------------|-------|---------|
| robotHandle | RobotHandle  | 输入    | 机械臂实例句柄 |
| robotMode   | RobotModes * | 输出    | 机械臂当前状态 |

#### 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
enum RobotModes robotMode;
result = cr_get_robotMode(robotHandle,&robotMode); //读机械臂当前状态
```

```
if (result != success) //判断读机械臂当前状态是否成功
    return -1;
printf("%d\n", robotMode);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.3.3.2 获取日志数量

#### 函数

**CRresult** cr\_cfg\_log\_number\_get(**RobotHandle** robotHandle, int \*logNumber)

#### 简介

获取日志数量。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| logNumber   | int *       | 输出    | 日志个数    |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int logNumber;
result = cr_cfg_log_number_get(robotHandle, &logNumber); //获取日志数量
if (result != success) //判断获取日志数量是否成功
    return -1;
printf("%d\n", logNumber);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.3.3.3 获取机械臂实时日志

#### 函数

**CRresult** cr\_get\_logMsgs(**RobotHandle** robotHandle, **RobotLogMsg** \*logMsgs, int \*logMsgNumber)

## 简介

获取机械臂实时日志。

## 参数

| 参数名称         | 数据类型          | 输入/输出 | 描述        |
|--------------|---------------|-------|-----------|
| robotHandle  | RobotHandle   | 输入    | 机械臂实例句柄   |
| logMsgs      | RobotLogMsg * | 输出    | 机械臂日志信息数组 |
| logMsgNumber | int *         | 输出    | 日志个数      |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int logMsgNumber = 10;
RobotLogMsg logMsgs[10];
result = cr_get_logMsgs(robotHandle, logMsgs, &logMsgNumber); //获取机械臂日志
if (result != success) //判断获取机械臂日志是否成功
    return -1;
#ifdef _WIN32
    system("chcp 65001");
#elif __linux__
    system("export LANG=C.UTF-8");
#endif
if(logMsgNumber>0)
{
    printf("%d\n", logMsgs[0].warningLevel);
    printf("%s\n", logMsgs[0].textMessage);
    printf("%d\n", logMsgNumber);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.3.3.4 获取机械臂历史所有日志

## 函数

```
CRresult cr_sys_history_log_get(RobotHandle robotHandle, RobotLogMsg *logMsgs, int
*logMsgNumber)
```

## 简介

获取机械臂历史所有日志。

## 参数

| 参数名称         | 数据类型          | 输入/输出 | 描述        |
|--------------|---------------|-------|-----------|
| robotHandle  | RobotHandle   | 输入    | 机械臂实例句柄   |
| logMsgs      | RobotLogMsg * | 输出    | 机械臂日志信息数组 |
| logMsgNumber | int *         | 输出    | 日志个数      |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int logMsgNumber;
RobotLogMsg *logMsgs=new RobotLogMsg[3000];
result = cr_sys_history_log_get(robotHandle, logMsgs, &logMsgNumber);
//获取机械臂历史所有日志
if (result != success) //判断获取机械臂历史所有日志是否成功
    return -1;
#ifdef _WIN32
    system("chcp 65001");
#elif __linux__
    system("export LANG=C.UTF-8");
#endif
if(logMsgNumber>0)
{
    printf("%d\n", logMsgs[0].warningLevel);
    printf("%s\n", logMsgs[0].textMessage);
    printf("%d\n", logMsgNumber);
}
delete[] logMsgs;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.3.3.5 日志功能是否开启

## 函数

CRresult cr\_log\_enable(BOOL enable)



## 简介

日志功能是否开启。需要在机器人创建连接之前使用。

## 参数

| 参数名称   | 数据类型 | 输入/输出 | 描述                     |
|--------|------|-------|------------------------|
| enable | BOOL | 输入    | 是否开启 TRUE-开启 FALSE-不开启 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```

BOOL enable = TRUE; //TRUE-开启
CRresult result = cr_log_enable(enable); //开启日志功能
if (result != success) //判断日志功能是否开启成功
    return -1;
    
```

### 3.3.3.6 日志大小设置

## 函数

**CRresult** cr\_log\_set\_size(long size,int num)

## 简介

日志大小设置。需要在机器人创建连接之前使用。

## 参数

| 参数名称 | 数据类型 | 输入/输出 | 描述          |
|------|------|-------|-------------|
| size | long | 输入    | 单个日志文件大小 字节 |
| num  | int  | 输入    | 日志文件数量      |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```

CRresult result = cr_log_enable(TRUE); //开启日志功能
if (result != success) //判断日志功能是否开启成功
    return -1;
long size = 1024*1024;
int num = 10;
result = cr_log_set_size(size,num); //日志大小设置
if (result != success) //判断日志大小设置是否成功
    return -1;
    
```

### 3.3.3.7 读系统时间

#### 函数

**CRresult** cr\_get\_SystemDateTime(**RobotHandle** robotHandle, char \*systemDateTime)

#### 简介

读控制柜系统日期时间。

#### 参数

| 参数名称           | 数据类型        | 输入/输出 | 描述           |
|----------------|-------------|-------|--------------|
| robotHandle    | RobotHandle | 输入    | 机械臂实例句柄      |
| systemDateTime | char *      | 输出    | 读取的控制柜系统日期时间 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char systemDateTime[100];
result = cr_get_SystemDateTime(robotHandle, systemDateTime); //读系统时间
if (result != success) //判断读系统时间是否成功
    return -1;
printf("%s\n", systemDateTime);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.3.3.8 读取机型型号

#### 函数

**CRresult** cr\_sys\_robotModel\_get(**RobotHandle** robotHandle, char \*robotModel, int len)

#### 简介

读取机型型号。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| robotModel  | char *      | 输出    | 机械臂型号   |

|     |     |    |           |
|-----|-----|----|-----------|
| len | int | 输入 | 机械臂型号数组长度 |
|-----|-----|----|-----------|

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char robotModel[10];
result = cr_sys_robotModel_get(robotHandle, robotModel, 10); //读机型型号
if (result != success) //判断读机型型号是否成功
    return -1;
printf("%s\n", robotModel);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.3.3.9 读取控制器型号

### 函数

**CRresult** cr\_get\_controllerType(**RobotHandle** robotHandle, char \*controllerType)

### 简介

读取控制器型号，返回码说明如下。

1-K20  
2-MKE  
3-K10  
4-DK10  
5-MK  
6-MK40-H  
7-MK300  
8-MKCAN

### 参数

| 参数名称           | 数据类型        | 输入/输出 | 描述      |
|----------------|-------------|-------|---------|
| robotHandle    | RobotHandle | 输入    | 机械臂实例句柄 |
| controllerType | char *      | 输出    | 控制器型号   |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char controllerType[10];
result = cr_get_controllerType(robotHandle, controllerType); //读机型型号
if (result != success) //判断读机型型号是否成功
    return -1;
printf("%s\n", controllerType);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.3.3.10 触发黑匣子数据

## 函数

**CResult** cr\_sys\_blackBoxData\_trigger(**RobotHandle** robotHandle)

## 简介

触发黑匣子数据。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

## 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_sys_blackBoxData_trigger(robotHandle); //触发黑匣子数据
if (result != success) //判断触发黑匣子数据是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.3.3.11 获取配置改变标志

#### 函数

**CRresult** cr\_cfg\_change\_code\_get(**RobotHandle** robotHandle, int\* changeCode)

#### 简介

获取配置改变标志。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| changeCode  | int*        | 输出    | 配置改变标志  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int changeCode;
result = cr_cfg_change_code_get(robotHandle, &changeCode); //获取配置改变标志位
if (result != success) //判断获取配置改变标志位是否成功
    return -1;
printf("%d\n", changeCode);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.3.3.12 触发系统更新

#### 函数

**CRresult** cr\_sys\_update\_trigger(**RobotHandle** robotHandle, int\* errCode)

#### 简介

触发系统更新。错误码说明如下：

- 1-机械臂未断电
- 2-安全控制器 ECAT 未处于 OP 状态
- 3-通知安全控制器自更新失败
- 4-控制器写自更新标识失败
- 5-ECAT 网络故障

## 7-当前控制柜 boot 版本不支持软更新

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述       |
|-------------|-------------|-------|----------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄  |
| errCode     | int*        | 输出    | 控制器返回错误码 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int errCode;
result = cr_sys_update_trigger(robotHandle, &errCode); //触发系统更新
if (result != success) //判断触发系统更新是否成功
    return -1;
printf("%d\n", errCode);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.4 变量

## 3.4.1 数字量

## 3.4.1.1 读控制柜标准数字输出

## 函数

```
CRresult cr_get_stdDigitalOut(RobotHandle robotHandle, int index, BOOL* val)
```

## 简介

读控制柜标准数字输出。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                     |
|-------------|-------------|-------|------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                |
| index       | int         | 输入    | 索引号（范围：0-7）            |
| val         | BOOL*       | 输出    | 标准数字输出值<br>(0-无效 1-有效) |

## 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
BOOL val;
result = cr_get_stdDigitalOut(robotHandle, index, &val); //读索引号为 1 的控制柜标准数字输出
if (result != success) //判断读控制柜标准数字输出是否成功
    return -1;
printf("%d\n", val);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.1.2 写控制柜标准数字输出

## 函数

**CResult** cr\_set\_stdDigitalOut(**RobotHandle** robotHandle, int index, **BOOL** val)

## 简介

写控制柜标准数字输出。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述          |
|-------------|-------------|-------|-------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄     |
| index       | int         | 输入    | 索引号（范围：0-7） |
| val         | BOOL        | 输入    | 标准数字输出值     |

## 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
BOOL val = TRUE;
result = cr_set_stdDigitalOut(robotHandle, index, val);
```

//将索引号为 1 的控制柜标准数字输出写为 TRUE

```
if (result != success) //判断写控制柜标准数字输出是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.1.3 读控制柜可配置数字输出

#### 函数

**CRresult** cr\_get\_configDigitalOut(**RobotHandle** robotHandle, int index, **BOOL\*** val)

#### 简介

读控制柜可配置数字输出。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                      |
|-------------|-------------|-------|-------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                 |
| index       | int         | 输入    | 索引号（范围：0-7）             |
| val         | BOOL*       | 输出    | 可配置数字输出值<br>（0-无效 1-有效） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
BOOL val;
result = cr_get_configDigitalOut(robotHandle, index, &val);
//读索引号为 1 的控制柜可配置数字输出
if (result != success) //判断读控制柜可配置数字输出是否成功
    return -1;
printf("%d\n", val);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```



### 3.4.1.4 读控制柜所有可配置数字输出

#### 函数

**CRresult** cr\_get\_all\_configDigitalOut(**RobotHandle** robotHandle, **BOOL\*** arrayVal, int arraySize)

#### 简介

读控制柜所有可配置数字输出。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述          |
|-------------|-------------|-------|-------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄     |
| arrayVal    | BOOL*       | 输出    | 待读取的值所存放的位置 |
| arraySize   | int         | 输入    | 读取的数量       |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
BOOL arrayVal[8];
int arraySize = 8;
result = cr_get_all_configDigitalOut(robotHandle, arrayVal, arraySize);
//读控制柜所有可配置数字输出
if (result != success) //判断读控制柜所有可配置数字输出是否成功
    return -1;
for (int i = 0; i < arraySize; i++)
{
    printf("%d\n", arrayVal[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.1.5 写控制柜可配置数字输出

#### 函数

**CRresult** cr\_set\_configDigitalOut(**RobotHandle** robotHandle, int index, **BOOL** val)

#### 简介

写控制柜可配置数字输出。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                      |
|-------------|-------------|-------|-------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                 |
| index       | int         | 输入    | 索引号 (范围: 0-7)           |
| val         | BOOL        | 输入    | 可配置数字输出值<br>(0-无效 1-有效) |

## 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
BOOL val = TRUE;
result = cr_set_configDigitalOut(robotHandle, index, val);
//将索引号为 1 的控制柜可配置数字输出写为 TRUE
if (result != success) //判断写控制柜可配置数字输出是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.4.1.6 读工具端数字输出

## 函数

```
CRresult cr_get_toolDigitalOut(RobotHandle robotHandle, int index, BOOL* val)
```

## 简介

读工具端数字输出。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                      |
|-------------|-------------|-------|-------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                 |
| index       | int         | 输入    | 索引号 (范围: 0-1)           |
| val         | BOOL*       | 输出    | 工具端数字输出值<br>(0-无效 1-有效) |

## 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 0;
BOOL val;
result = cr_get_toolDigitalOut(robotHandle, index, &val); //读索引号为 0 的工具端数字输出
if (result != success) //判断读工具端数字输出是否成功
    return -1;
printf("%d\n", val);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.1.7 写工具端数字输出

## 函数

**CResult** cr\_set\_toolDigitalOut(**RobotHandle** robotHandle, int index, **BOOL** val)

## 简介

写工具端数字输出。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                      |
|-------------|-------------|-------|-------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                 |
| index       | int         | 输入    | 索引号（范围：0-1）             |
| val         | BOOL        | 输入    | 工具端数字输出值<br>（0-无效 1-有效） |

## 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 0;
BOOL val = FALSE;
result = cr_set_toolDigitalOut(robotHandle, index, val);
//将索引值为 0 的工具端数字输出写为 FALSE
```

```
if (result != success) //判断写工具端数字输出是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.1.8 读控制柜标准数字输入

#### 函数

**CRresult** cr\_get\_stdDigitalIn(**RobotHandle** robotHandle, int index, **BOOL\*** val)

#### 简介

读控制柜标准数字输入。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                     |
|-------------|-------------|-------|------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                |
| index       | int         | 输入    | 索引号（范围：0-7）            |
| val         | BOOL*       | 输出    | 标准数字输入值<br>（0-无效 1-有效） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 0;
BOOL val;
result = cr_get_stdDigitalIn(robotHandle, index, &val); //读索引号为 0 的控制柜标准数字输入
if (result != success) //判断读控制柜标准数字输入是否成功
    return -1;
printf("%d\n", val);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.1.9 读控制柜可配置数字输入

#### 函数

```
CRresult cr_get_configDigitalIn(RobotHandle robotHandle, int index, BOOL* val)
```

#### 简介

读控制柜可配置数字输入。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                     |
|-------------|-------------|-------|------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                |
| index       | int         | 输入    | 索引号（范围：0-7）            |
| val         | BOOL*       | 输出    | 标准数字输入值<br>(0-无效 1-有效) |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 0;
BOOL val;
result = cr_get_configDigitalIn(robotHandle, index, &val);
//读索引号为 0 的控制柜可配置数字输入
if (result != success) //判断读控制柜可配置数字输入是否成功
    return -1;
printf("%d\n", val);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.1.10 读控制柜所有可配置数字输入

#### 函数

```
CRresult cr_get_all_configDigitalIn(RobotHandle robotHandle, BOOL* arrayVal, int arraySize)
```

#### 简介

读控制柜所有可配置数字输入。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述          |
|-------------|-------------|-------|-------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄     |
| arrayVal    | BOOL*       | 输出    | 待读取的值所存放的位置 |
| arraySize   | int         | 输入    | 读取的数量       |

## 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
BOOL arrayVal[8];
int arraySize = 8;
result = cr_get_all_configDigitalIn(robotHandle, arrayVal, arraySize);
//读控制柜所有可配置数字输入
if (result != success) //判断读控制柜所有可配置数字输入是否成功
    return -1;
for (int i = 0; i < arraySize; i++)
{
    printf("%d\n", arrayVal[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.4.1.11 读工具端数字输入

## 函数

CResult cr\_get\_toolDigitalIn(RobotHandle robotHandle, int index, BOOL \*val)

## 简介

读工具端数字输入。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                      |
|-------------|-------------|-------|-------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                 |
| index       | int         | 输入    | 索引号（范围：0-1）             |
| val         | BOOL*       | 输出    | 工具端数字输入值<br>（0-无效 1-有效） |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 0;
BOOL val;
result = cr_get_toolDigitalIn(robotHandle, index, &val); //读索引号为 0 的工具端数字输入
if (result != success) //判断读工具端数字输入是否成功
    return -1;
printf("%d\n", val);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.4.2 模拟量

### 3.4.2.1 读控制柜模拟量输出

## 函数

CRresult cr\_get\_stdAnalogOut(RobotHandle robotHandle, int index, double \*val)

## 简介

读控制柜模拟量输出。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述        |
|-------------|-------------|-------|-----------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄   |
| index       | int         | 输入    | 索引号（范围：0） |
| val         | double *    | 输出    | 模拟量输出值    |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
```

```
int index = 0;
double val;
result = cr_get_stdAnalogOut(robotHandle, index, &val); //读索引号为 0 的控制柜模拟量输出
if (result != success) //判断读控制柜模拟量输出是否成功
    return -1;
printf("%f\n", val);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.2.2 写控制柜模拟量输出

#### 函数

**CRresult** cr\_set\_stdAnalogOut(**RobotHandle** robotHandle, int index, double val)

#### 简介

写控制柜模拟量输出。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述        |
|-------------|-------------|-------|-----------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄   |
| index       | int         | 输入    | 索引号（范围：0） |
| val         | double      | 输入    | 模拟量输出值    |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 0;
double val = 3;
result = cr_set_stdAnalogOut(robotHandle, index, val);
//将索引号为 0 的控制柜模拟量输出写为 3V 或 3mA
if (result != success) //判断写控制柜模拟量输出是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```



### 3.4.2.3 读工具端模拟量输入

#### 函数

**CRresult** cr\_get\_toolAnalogIn(**RobotHandle** robotHandle, int index, double \*val)

#### 简介

读工具端模拟量输入。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述          |
|-------------|-------------|-------|-------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄     |
| index       | int         | 输入    | 索引号（范围：0-1） |
| val         | double *    | 输出    | 模拟量输入值      |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
double val;
result = cr_get_toolAnalogIn(robotHandle, index, &val); //读索引号为 1 的工具端模拟量输入
if (result != success) //判断读工具端模拟量输入是否成功
    return -1;
printf("%f\n", val);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.2.4 读控制柜模拟量输入

#### 函数

**CRresult** cr\_get\_stdAnalogIn(**RobotHandle** robotHandle, int index, double \*val)

#### 简介

读控制柜模拟量输入。

#### 参数

| 参数名称 | 数据类型 | 输入/输出 | 描述 |
|------|------|-------|----|
|------|------|-------|----|

|             |             |    |               |
|-------------|-------------|----|---------------|
| robotHandle | RobotHandle | 输入 | 机械臂实例句柄       |
| index       | int         | 输入 | 索引号 (范围: 0-2) |
| val         | double *    | 输出 | 模拟量输入值        |

### 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
double val;
result = cr_get_stdAnalogIn(robotHandle, index, &val); //读索引号为 1 的控制柜模拟量输入
if (result != success) //判断读控制柜模拟量输入是否成功
    return -1;
printf("%f\n", val);
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.4.2.5 读机械臂控制柜和工具端上所有输入数字量和模拟量

### 函数

```
CRresult cr_get_allDAInput(RobotHandle robotHandle, BOOL controllerDI[CR6_DI_NUM],
    BOOL controllerCI[CR6_CI_NUM], double controllerAI[CR6_AI_NUM], BOOL
    toolDI[CR6_TOOL_DI_NUM], double toolAI[CR6_TOOL_AI_NUM])
```

### 简介

读机械臂控制柜和工具端上所有输入数字量和模拟量。

### 参数

| 参数名称         | 数据类型        | 输入/输出 | 描述          |
|--------------|-------------|-------|-------------|
| robotHandle  | RobotHandle | 输入    | 机械臂实例句柄     |
| controllerDI | BOOL *      | 输出    | 控制柜标准数字输入量  |
| controllerCI | BOOL *      | 输出    | 控制柜可配置数字输入量 |
| controllerAI | double *    | 输出    | 控制柜模拟输入量    |
| toolDI       | BOOL *      | 输出    | 工具端标准数字输入量  |
| toolAI       | double *    | 输出    | 工具端模拟输入量    |

### 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
BOOL controllerDI[CR6_DI_NUM];
BOOL controllerCI[CR6_CI_NUM];
double controllerAI[CR6_AI_NUM];
BOOL toolDI[CR6_TOOL_DI_NUM];
double toolAI[CR6_TOOL_AI_NUM];
result = cr_get_allDAInput(robotHandle, controllerDI, controllerCI, controllerAI, toolDI, toolAI);
//读机械臂控制柜和工具端上所有输入数字量和模拟量
if (result != success) //判断读机械臂控制柜和工具端上所有输入数字量和模拟量是否成功
    return -1;
for (int i = 0; i < CR6_DI_NUM; i++)
{
    printf("%d\n", controllerDI[i]);
    printf("%d\n", controllerCI[i]);
}
for (int i = 0; i < CR6_AI_NUM; i++)
{
    printf("%f\n", controllerAI[i]);
}
for (int i = 0; i < CR6_TOOL_DI_NUM; i++)
{
    printf("%d\n", toolDI[i]);
    printf("%f\n", toolAI[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.2.6 读机械臂控制柜和工具端上所有输出数字量和模拟量

## 函数

**CRresult** cr\_get\_allDAOutput(**RobotHandle** robotHandle, **BOOL** controllerDO[CR6\_DO\_NUM], **BOOL** controllerCO[CR6\_CO\_NUM], **double** controllerAO[CR6\_AO\_NUM], **BOOL** toolDO[CR6\_TOOL\_DO\_NUM])

## 简介

读机械臂控制柜和工具端上所有输出数字量和模拟量。

## 参数

| 参数名称 | 数据类型 | 输入/输出 | 描述 |
|------|------|-------|----|
|------|------|-------|----|

|              |             |    |             |
|--------------|-------------|----|-------------|
| robotHandle  | RobotHandle | 输入 | 机械臂实例句柄     |
| controllerDO | BOOL *      | 输出 | 控制柜标准数字输出量  |
| controllerCO | BOOL *      | 输出 | 控制柜可配置数字输出量 |
| controllerAO | double *    | 输出 | 控制柜模拟输出量    |
| toolDO       | BOOL *      | 输出 | 工具端标准数字输出量  |

### 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
BOOL controllerDO[CR6_DO_NUM];
BOOL controllerCO[CR6_CO_NUM];
double controllerAO[CR6_AO_NUM];
BOOL toolDO[CR6_TOOL_DO_NUM];
result = cr_get_allDAOOutput(robotHandle, controllerDO, controllerCO, controllerAO, toolDO);
//读机械臂控制柜和工具端上所有输出数字量和模拟量
if (result != success) //判断读机械臂控制柜和工具端上所有输出数字量和模拟量是否成功
    return -1;
for (int i = 0; i < CR6_DO_NUM; i++)
{
    printf("%d\n", controllerDO[i]);
    printf("%d\n", controllerCO[i]);
}
for (int i = 0; i < CR6_AO_NUM; i++)
{
    printf("%f\n", controllerAO[i]);
}
for (int i = 0; i < CR6_TOOL_DO_NUM; i++)
{
    printf("%d\n", toolDO[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.3 通讯变量

#### 3.4.3.1 读 bool 寄存器

##### 函数

```
CRresult cr_get_boolRegValue(RobotHandle robotHandle, unsigned int startAddress, unsigned int offsetLength, BOOL* regData)
```

##### 简介

读 bool 寄存器，单次最多读取 100 个。

##### 参数

| 参数名称         | 数据类型         | 输入/输出 | 描述                                |
|--------------|--------------|-------|-----------------------------------|
| robotHandle  | RobotHandle  | 输入    | 机械臂实例句柄                           |
| startAddress | unsigned int | 输入    | 寄存器起始地址（范围：0-999）                 |
| offsetLength | unsigned int | 输入    | 寄存器偏移地址<br>（范围：小于或等于 1000 - 起始地址） |
| regData      | BOOL *       | 输出    | bool 寄存器数据                        |

##### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

##### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
unsigned int startAddress = 0;
unsigned int offsetLength = 10;
BOOL regData[10];
result = cr_get_boolRegValue(robotHandle, startAddress, offsetLength, regData);
//读地址为 0~9 的 bool 寄存器
if (result != success) //判断读 bool 寄存器是否成功
    return -1;
printf("%d\n", regData[1]);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.3.2 写 bool 寄存器

#### 函数

```
CRresult cr_set_boolRegValue(RobotHandle robotHandle, unsigned int startAddress,unsigned int offsetLength, BOOL *regData)
```

#### 简介

写 bool 寄存器。

#### 参数

| 参数名称         | 数据类型         | 输入/输出 | 描述                                |
|--------------|--------------|-------|-----------------------------------|
| robotHandle  | RobotHandle  | 输入    | 机械臂实例句柄                           |
| startAddress | unsigned int | 输入    | 寄存器起始地址（范围：0-999）                 |
| offsetLength | unsigned int | 输入    | 寄存器偏移地址<br>（范围：小于或等于 1000 - 起始地址） |
| regData      | BOOL *       | 输入    | bool 寄存器数据                        |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
unsigned int startAddress = 0;
unsigned int offsetLength = 10;
BOOL regData[10] = { 0,1,0,1,0,1,0,1,0,1 }; //写入数据
result = cr_set_boolRegValue(robotHandle,startAddress,offsetLength, regData);
//写地址为 0~9 的 bool 寄存器
if (result != success) //判断写 bool 寄存器是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.3.3 读 int16 寄存器

#### 函数

```
CRresult cr_get_int16RegValue(RobotHandle robotHandle, unsigned int startAddress,unsigned int offsetLength, int *regData)
```

## 简介

读 int16 寄存器，单次最多读取 100 个。

## 参数

| 参数名称         | 数据类型         | 输入/输出 | 描述                                |
|--------------|--------------|-------|-----------------------------------|
| robotHandle  | RobotHandle  | 输入    | 机械臂实例句柄                           |
| startAddress | unsigned int | 输入    | 寄存器起始地址（范围：0-999）                 |
| offsetLength | unsigned int | 输入    | 寄存器偏移地址<br>（范围：小于或等于 1000 - 起始地址） |
| regData      | int *        | 输出    | int16 寄存器数据                       |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
unsigned int startAddress = 0;
unsigned int offsetLength = 10;
int regData[10];
result = cr_get_int16RegValue(robotHandle, startAddress, offsetLength, regData);
//读地址为 0~9 的 int16 寄存器
if (result != success) //判断读 int16 寄存器是否成功
    return -1;
printf("%d\n", regData[1]);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.3.4 写 int16 寄存器

## 函数

**CRresult** cr\_set\_int16RegValue(**RobotHandle** robotHandle, unsigned int startAddress, unsigned int offsetLength, int \*regData)

## 简介

写 int16 寄存器。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

|              |              |    |                                    |
|--------------|--------------|----|------------------------------------|
| startAddress | unsigned int | 输入 | 寄存器起始地址 (范围: 0-999)                |
| offsetLength | unsigned int | 输入 | 寄存器偏移地址<br>(范围: 小于或等于 1000 - 起始地址) |
| regData      | int *        | 输入 | int16 寄存器数据 (范围: -32768-32767)     |

#### 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
unsigned int startAddress = 0;
unsigned int offsetLength = 10;
int regData[10] = {1,2,3,4,5,6,7,8,9,10};
result = cr_set_int16RegValue(robotHandle, startAddress, offsetLength, regData);
//写地址为 0~9 的 int16 寄存器
if (result != success) //判断写 int16 寄存器是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.3.5 读 int32 寄存器

#### 函数

**CRresult** cr\_get\_int32RegValue(**RobotHandle** robotHandle, unsigned int startAddress, unsigned int offsetLength, int \*regData)

#### 简介

读 int32 寄存器, 单次最多读取 100 个。

#### 参数

| 参数名称         | 数据类型         | 输入/输出 | 描述                                 |
|--------------|--------------|-------|------------------------------------|
| robotHandle  | RobotHandle  | 输入    | 机械臂实例句柄                            |
| startAddress | unsigned int | 输入    | 寄存器起始地址 (范围: 0-999)                |
| offsetLength | unsigned int | 输入    | 寄存器偏移地址<br>(范围: 小于或等于 1000 - 起始地址) |
| regData      | int *        | 输出    | int32 寄存器数据                        |

#### 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。



## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
unsigned int startAddress = 0;
unsigned int offsetLength = 10;
int regData[10];
result = cr_get_int32RegValue(robotHandle, startAddress, offsetLength, regData);
//读地址为 0~9 的 int32 寄存器
if (result != success) //判断读 int32 寄存器是否成功
    return -1;
printf("%d\n", regData[1]);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.3.6 写 int32 寄存器

## 函数

**CResult** cr\_set\_int32RegValue(**RobotHandle** robotHandle, unsigned int startAddress, unsigned int offsetLength, int \*regData)

## 简介

写 int32 寄存器。

## 参数

| 参数名称         | 数据类型         | 输入/输出 | 描述                                         |
|--------------|--------------|-------|--------------------------------------------|
| robotHandle  | RobotHandle  | 输入    | 机械臂实例句柄                                    |
| startAddress | unsigned int | 输入    | 寄存器起始地址（范围：0-999）                          |
| offsetLength | unsigned int | 输入    | 寄存器偏移地址<br>（范围：小于或等于 1000 - 起始地址）          |
| regData      | int *        | 输入    | int32 寄存器数据<br>（范围：-2147483648-2147483647） |

## 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
```

```

unsigned int startAddress = 0;
unsigned int offsetLength = 10;
int regData[10] = {1,2,3,4,5,6,7,8,9,10};
result = cr_set_int32RegValue(robotHandle,startAddress,offsetLength,regData);
//写地址为 0~9 的 int32 寄存器
if (result != success) //判断写 int32 寄存器是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
    
```

### 3.4.3.7 读 float 寄存器

#### 函数

**CRresult** cr\_get\_floatRegValue(**RobotHandle** robotHandle, unsigned int startAddress,unsigned int offsetLength, float \*regData)

#### 简介

读 float 寄存器，单次最多读取 100 个。

#### 参数

| 参数名称         | 数据类型         | 输入/输出 | 描述                                |
|--------------|--------------|-------|-----------------------------------|
| robotHandle  | RobotHandle  | 输入    | 机械臂实例句柄                           |
| startAddress | unsigned int | 输入    | 寄存器起始地址（范围：0-999）                 |
| offsetLength | unsigned int | 输入    | 寄存器偏移地址<br>（范围：小于或等于 1000 - 起始地址） |
| regData      | float *      | 输出    | float 寄存器数据                       |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
unsigned int startAddress = 0;
unsigned int offsetLength = 10;
float regData[10];
result = cr_get_floatRegValue(robotHandle,startAddress,offsetLength, regData);
//读地址为 0~9 的 float 寄存器
if (result != success) //判断读 float 寄存器是否成功
    return -1;
    
```

```
printf("%f\n", regData[0]);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.3.8 写 float 寄存器

#### 函数

**CRresult** cr\_set\_floatRegValue(**RobotHandle** robotHandle, unsigned int startAddress, unsigned int offsetLength, float \*regData)

#### 简介

写 float 寄存器。

#### 参数

| 参数名称         | 数据类型         | 输入/输出 | 描述                                |
|--------------|--------------|-------|-----------------------------------|
| robotHandle  | RobotHandle  | 输入    | 机械臂实例句柄                           |
| startAddress | unsigned int | 输入    | 寄存器起始地址（范围：0-999）                 |
| offsetLength | unsigned int | 输入    | 寄存器偏移地址<br>（范围：小于或等于 1000 - 起始地址） |
| regData      | float *      | 输入    | float 寄存器数据                       |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
unsigned int startAddress = 0;
unsigned int offsetLength = 10;
float regData[10] = { 1,2,3,4,5,6,7,8,9,10 };
result = cr_set_floatRegValue(robotHandle, startAddress, offsetLength, regData);
//写地址为 0~9 的 float 寄存器
if (result != success) //判断写 float 寄存器是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.4.4 脚本变量

### 3.4.4.1 读安装变量

#### 函数

```
CRresult cr_get_intallVarValue(RobotHandle robotHandle, const char *installVarName,
VariableMsg *installVar)
```

#### 简介

读安装变量。

#### 参数

| 参数名称           | 数据类型          | 输入/输出 | 描述        |
|----------------|---------------|-------|-----------|
| robotHandle    | RobotHandle   | 输入    | 机械臂实例句柄   |
| installVarName | const char *  | 输入    | 需读取安装变量名称 |
| installVar     | VariableMsg * | 输出    | 读取的安装变量值  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
const char installVarName[CR6_ISTALL_VAR_NAME_LENGTH] = {"i_var_1"};
VariableMsg *installVar = new VariableMsg;
result = cr_get_intallVarValue(robotHandle, installVarName, installVar);
//读名称为 i_var_1 的安装变量的值
if (result != success) //判断读安装变量是否成功
    return -1;
printf("%d\n", installVar.numberValue);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.4.2 修改安装变量

#### 函数

```
CRresult cr_set_installVarValue(RobotHandle robotHandle, const char *installVarName,
VariableMsg *installVar)
```

## 简介

修改安装变量。

## 参数

| 参数名称           | 数据类型          | 输入/输出 | 描述        |
|----------------|---------------|-------|-----------|
| robotHandle    | RobotHandle   | 输入    | 机械臂实例句柄   |
| installVarName | const char *  | 输入    | 需读取安装变量名称 |
| installVar     | VariableMsg * | 输入    | 需赋值的安装变量值 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
const char installVarName[CR6_INSTALL_VAR_NAME_LENGTH] = {"i_var_1"};
VariableMsg installVar;
installVar.boolValue = true;
installVar.variableType = LUA_TBOOLEAN;
char name[CR6_INSTALL_VAR_NAME_LENGTH] = {"i_var_2"};
strcpy_s(installVar.variableName, name);
result = cr_set_installVarValue(robotHandle, installVarName, &installVar);
//修改名称为 i_var_1 的安装变量
if (result != success) //判断修改安装变量是否成功
    return -1;
printf("%d\n", installVar.numberValue);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.4.3 读取程序变量数据

## 函数

```
CRresult cr_script_var_get(RobotHandle robotHandle, char *varName, VariableMsg
*variableMsg)
```

## 简介

读取程序变量数据。

## 参数

| 参数名称 | 数据类型 | 输入/输出 | 描述 |
|------|------|-------|----|
|------|------|-------|----|

|             |               |    |         |
|-------------|---------------|----|---------|
| robotHandle | RobotHandle   | 输入 | 机械臂实例句柄 |
| varName     | char *        | 输入 | 程序变量名   |
| variableMsg | VariableMsg * | 输出 | 程序变量数据  |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char varName[20] = "var_1";
VariableMsg variableMsg;
result = cr_script_var_get(robotHandle, varName, &variableMsg);
//读取名称为 var_1 的程序变量的数据
if (result != success) //判断读取程序变量数据是否成功
    return -1;
printf("%d\n", variableMsg.variableType);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.4.4.4 修改程序变量数据

### 函数

```
CRresult cr_script_var_set(RobotHandle robotHandle, VariableMsg *variableMsg)
```

### 简介

修改程序变量数据。该接口只在程序运行或暂停时有效。使用该接口时需保证修改的值的类型需要与程序中的变量类型对应。

### 参数

| 参数名称        | 数据类型          | 输入/输出 | 描述      |
|-------------|---------------|-------|---------|
| robotHandle | RobotHandle   | 输入    | 机械臂实例句柄 |
| variableMsg | VariableMsg * | 输入    | 程序变量数据  |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
```

```

if (result != success) //判断机器人连接是否成功
    return -1;
VariableMsg variableMsg;
variableMsg.numberValue = 2;
variableMsg.variableType = LUA_TNUMBER;
char name[CR6_ISTALL_VAR_NAME_LENGTH] = { "var_1" };
strcpy_s(variableMsg.variableName, sizeof(name), name);
result = cr_script_var_set(robotHandle, &variableMsg);
//修改名称为 var_1 的程序变量的数据
if (result != success) //判断修改程序变量数据是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
    
```

### 3.4.5 其他 IO

#### 3.4.5.1 获取编码器计数数量

##### 函数

```

CRresult cr_get_encoderTickCnt(RobotHandle robotHandle, int encoderChannel, long long*
encoderTickCount);
    
```

##### 简介

获取编码器计数数量。

##### 参数

| 参数名称             | 数据类型        | 输入/输出 | 描述      |
|------------------|-------------|-------|---------|
| robotHandle      | RobotHandle | 输入    | 机械臂实例句柄 |
| encoderChannel   | int         | 输入    | 编码器通道   |
| encoderTickCount | long long*  | 输出    | 编码器计数数量 |

##### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

##### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int encoderChannel = 0;
long long encoderTickCount;
result = cr_get_encoderTickCnt(robotHandle, encoderChannel, &encoderTickCount);
    
```

```
//获取编码器通道 0 的计数数量
if (result != success) //判断获取编码器计数数量是否成功
    return -1;
printf("%lld\n", encoderTickCount);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.5.2 设置监控界面的工具输出电压

#### 函数

**CRresult** cr\_set\_ToolOutputVoltage(**RobotHandle** robotHandle,enum **ToolPower** toolPower);

#### 简介

设置监控界面的工具输出电压。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| toolPower   | ToolPower   | 输入    | 工具输出电压  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
enum ToolPower toolPower;
toolPower = Power_on;
result = cr_set_ToolOutputVoltage(robotHandle, toolPower);
//设置监控界面的工具输出电压为 24v;
if (result != success) //判断设置监控界面的工具输出电压是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.4.5.3 读取监控界面的工具输出电压

#### 函数

**CRresult** cr\_get\_ToolOutputVoltage(**RobotHandle** robotHandle,enum **ToolPower\*** toolPower);



## 简介

读取监控界面的工具输出电压。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| toolPower   | ToolPower*  | 输出    | 工具输出电压  |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
enum ToolPower toolPower;
result = cr_get_ToolOutputVoltage(robotHandle, &toolPower);
//读取监控界面的工具输出电压
if (result != success) //判断读取监控界面的工具输出电压是否成功
    return -1;
printf("%d\n", toolPower);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.5 数学模块

### 3.5.1 位姿变换

#### 3.5.1.1 位姿转换

## 函数

```
CRresult cr_poseTrans(double poseFrom[ROB_AXIS_NUM], double
poseTrans[ROB_AXIS_NUM], double poseTo[ROB_AXIS_NUM])
```

## 简介

位姿转换。

## 参数

| 参数名称     | 数据类型     | 输入/输出 | 描述                      |
|----------|----------|-------|-------------------------|
| poseFrom | double * | 输入    | 原位姿：x, y, z (单位：mm)；Rx, |

|           |          |    |                                            |
|-----------|----------|----|--------------------------------------------|
|           |          |    | Ry, Rz (单位: °)                             |
| poseTrans | double * | 输入 | 变换位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °) |
| poseTo    | double * | 输出 | 结果位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °) |

### 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

### 示例

```
double poseFrom[ROB_AXIS_NUM] = { 1,2,3,0,0,0 };
double poseTrans[ROB_AXIS_NUM] = { 4,4,4,0,0,0 };
double poseTo[ROB_AXIS_NUM];
CRresult result = cr_poseTrans(poseFrom,poseTrans,poseTo); //位姿转换
if (result != success) //判断位姿转换是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", poseTo[i]);
}
```

## 3.5.1.2 位姿求逆

### 函数

**CRresult** cr\_compute\_pose\_inv(double \*pose,int poseLen,double \*poseInv,int poseInvLen)

### 简介

位姿求逆。

### 参数

| 参数名称       | 数据类型     | 输入/输出 | 描述                                           |
|------------|----------|-------|----------------------------------------------|
| pose       | double * | 输入    | 轴角位姿数据: x, y, z (单位: mm); Rx, Ry, Rz (单位: °) |
| poseLen    | int      | 输入    | 位姿长度, 至少为 6                                  |
| poseInv    | double * | 输出    | 位姿的逆: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)   |
| poseInvLen | int      | 输入    | 位姿长度, 至少为 6                                  |

### 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

### 示例

```
int poseLen = 6;
```

```
int poseInvLen = 6;
double pose[6] = { -66, -438, 887, -57, 0, -148 };
double poseInv[6];
CRresult result = cr_compute_pose_inv(pose, poseLen, poseInv, poseInvLen); //位姿求逆
if (result != success) //判断位姿求逆是否成功
    return -1;
for (int i = 0; i < 6; i++)
{
    printf("%f\n", poseInv[i]);
}
```

### 3.5.1.3 轴角转欧拉角

#### 函数

```
CRresult cr_AxisAngle2Eule(double axisAnglePose[ROB_AXIS_NUM], double
eulePose[ROB_AXIS_NUM])
```

#### 简介

轴角转欧拉角。

#### 参数

| 参数名称          | 数据类型     | 输入/输出 | 描述                                |
|---------------|----------|-------|-----------------------------------|
| axisAnglePose | double * | 输入    | 轴角位姿：x，y，z（单位：mm）；Rx，Ry，Rz（单位：°）  |
| eulePose      | double * | 输出    | 欧拉角位姿：x，y，z（单位：mm）；Rx，Ry，Rz（单位：°） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
double axisAnglePose[ROB_AXIS_NUM] = {50.06,467.28,672.44,19.81,1.07,-66.3};
double eulePose[ROB_AXIS_NUM];
CRresult result = cr_AxisAngle2Eule(axisAnglePose,eulePose); //轴角转欧拉角
if (result != success) //判断轴角转欧拉角是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", eulePose[i]);
}
```

### 3.5.1.4 欧拉角转轴角

#### 函数

```
CRresult cr_Eule2AxisAngle(double eulePose[ROB_AXIS_NUM], double  
axisAnglePose[ROB_AXIS_NUM])
```

#### 简介

欧拉角转轴角。

#### 参数

| 参数名称          | 数据类型     | 输入/输出 | 描述                                          |
|---------------|----------|-------|---------------------------------------------|
| eulePose      | double * | 输入    | 欧拉角位姿：x, y, z (单位：mm)；<br>Rx, Ry, Rz (单位：°) |
| axisAnglePose | double * | 输出    | 轴角位姿：x, y, z (单位：mm)；Rx,<br>Ry, Rz (单位：°)   |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
double eulePose[ROB_AXIS_NUM] = { 50.06,467.28,672.44,19.81,1.07,-66.3 };  
double axisAnglePose[ROB_AXIS_NUM] ;  
CRresult result = cr_Eule2AxisAngle(eulePose,axisAnglePose); //欧拉角转轴角  
if (result != success) //判断欧拉角转轴角是否成功  
    return -1;  
for (int i = 0; i < ROB_AXIS_NUM; i++)  
{  
    printf("%f\n", axisAnglePose[i]);  
}
```

### 3.5.1.5 TCP 位姿转法兰位姿

#### 函数

```
CRresult cr_TcpToFlangePose( double tcpPose[ROB_AXIS_NUM], double  
toolPose[ROB_AXIS_NUM],double flangePose[ROB_AXIS_NUM])
```

#### 简介

TCP 位姿转法兰位姿。

#### 参数

| 参数名称    | 数据类型     | 输入/输出 | 描述                                                    |
|---------|----------|-------|-------------------------------------------------------|
| tcpPose | double * | 输入    | tcp 位姿在用户坐标系下的描述：x, y, z<br>(单位：mm)；Rx, Ry, Rz (单位：°) |

|            |          |    |                                                    |
|------------|----------|----|----------------------------------------------------|
| toolPose   | double * | 输入 | 当前使用的 tcp: x, y, z (单位: mm);<br>Rx, Ry, Rz (单位: °) |
| flangePose | double * | 输出 | 法兰在基座坐标系下的位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °) |

### 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

### 示例

```
double tcpPose[ROB_AXIS_NUM] = {223,240,550,23,24,55};
double toolPose[ROB_AXIS_NUM] = {22,44,55,66,77,88};
double flangePose[ROB_AXIS_NUM];
CRresult result = cr_TcpToFlangePose(tcpPose,toolPose,flangePose); //TCP 位姿转法兰位姿
if (result != success) //判断 TCP 位姿转法兰位姿是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", flangePose[i]);
}
```

## 3.5.1.6 基坐标系下位姿转移到用户坐标系下

### 函数

**CRresult** cr\_compute\_pose\_base\_to\_user(double \*basePose,int basePoseLen,double \*userPose,int userPoseLen, double \*poseInUser,int poseInUserLen)

### 简介

基坐标系下位姿转移到用户坐标系下。

### 参数

| 参数名称          | 数据类型     | 输入/输出 | 描述                                              |
|---------------|----------|-------|-------------------------------------------------|
| basePose      | double * | 输入    | 轴角位姿数据: x, y, z (单位: mm);<br>Rx, Ry, Rz (单位: °) |
| basePoseLen   | int      | 输入    | 位姿长度, 至少为 6                                     |
| userPose      | double * | 输入    | 用户坐标系位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)   |
| userPoseLen   | int      | 输入    | 位姿长度, 至少为 6                                     |
| poseInUser    | double * | 输出    | 用户坐标系下的位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °) |
| poseInUserLen | int      | 输入    | 位姿长度, 至少为 6                                     |

### 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

## 示例

```
int basePoseLen = 6;
int userPoseLen = 6;
int poseInUserLen = 6;
double basePose[6] = { -66, -438, 887, -57, 0, -148 };
double userPose[6] = { 10, 20, 30, 0, 0, 0 };
double poseInUser[6];
CRresult result = cr_compute_pose_base_to_user(basePose, basePoseLen, userPose, userPoseLen,
poseInUser, poseInUserLen); //基坐标系下位姿转移到用户坐标系下
if (result != success) //判断基坐标系下位姿转移到用户坐标系下是否成功
    return -1;
for (int i = 0; i < 6; i++)
{
    printf("%f\n", poseInUser[i]);
}
```

## 3.5.1.7 目标点旋转变换

### 函数

**CRresult** cr\_compute\_rotate\_around\_axis(double \*axisPosition, double \*axisDirection, double angle, double \*srcPose, double \*dstPose)

### 简介

目标点旋转变换。

### 参数

| 参数名称          | 数据类型     | 输入/输出 | 描述                                           |
|---------------|----------|-------|----------------------------------------------|
| axisPosition  | double * | 输入    | 旋转轴位置(x,y,z (单位: mm) )                       |
| axisDirection | double * | 输入    | 旋转轴方向(x,y,z (单位: mm) )                       |
| angle         | double   | 输入    | 旋转角度 (° )                                    |
| srcPose       | double * | 输入    | 原始目标位姿(x,y,z (单位: mm) );Rx,Ry,Rz (单位: ° ) )  |
| dstPose       | double * | 输出    | 旋转后目标位姿(x,y,z (单位: mm) );Rx,Ry,Rz (单位: ° ) ) |

### 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

### 示例

```
double axisPosition[3] = {0,0,0};
double axisDirection[3] = {0,0,100};
double angle = 180;
double srcPose[ROB_AXIS_NUM] = {-66.81,-448.64,890.89,-57.07,-0.23,-148.83};
```

```
double dstPose[ROB_AXIS_NUM] = {0};
CRresult result = cr_compute_rotate_around_axis(axisPosition, axisDirection, angle, srcPose,
dstPose); //目标点旋转变换
if (result != success) //判断目标点旋转变换是否成功
    return -1;
for (int i = 0; i < 6; i++)
{
    printf("%f\n", dstPose[i]);
}
```

## 3.5.2 运动学

### 3.5.2.1 正解

#### 函数

```
CRresult cr_kineForward(RobotHandle robotHandle, double srcJointPose[ROB_AXIS_NUM],
double tarPose[ROB_AXIS_NUM])
```

#### 简介

正解。

#### 参数

| 参数名称         | 数据类型     | 输入/输出 | 描述                                      |
|--------------|----------|-------|-----------------------------------------|
| robotHandle  | int      | 输入    | 机械臂实例句柄                                 |
| srcJointPose | double * | 输入    | 关节空间位置：关节 1~6（单位：°）                     |
| tarPose      | double * | 输出    | 笛卡尔空间位姿计算结果：x，y，z（单位：mm）；Rx，Ry，Rz（单位：°） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double srcJointPose[ROB_AXIS_NUM] = {60,60,60,60,60,60};
double tarPose[ROB_AXIS_NUM];
result = cr_kineForward(robotHandle, srcJointPose, tarPose); //正解
if (result != success) //判断正解是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
```

```
printf("%f\n", tarPose[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.5.2.2 逆解

#### 函数

**CRresult** cr\_kineInverse(**RobotHandle** robotHandle, double srcPose[ROB\_AXIS\_NUM], double refJointPose[ROB\_AXIS\_NUM], double tarJointPose[ROB\_AXIS\_NUM])

#### 简介

逆解。

#### 参数

| 参数名称         | 数据类型        | 输入/输出 | 描述                                             |
|--------------|-------------|-------|------------------------------------------------|
| robotHandle  | RobotHandle | 输入    | 机械臂实例句柄                                        |
| srcPose      | double *    | 输入    | 笛卡尔空间位姿值: x, y, z (单位: mm); Rx, Ry, Rz (单位: °) |
| refJointPose | double *    | 输入    | 逆解计算用的参考关节空间位置: 关节 1~6 (单位: °)                 |
| tarJointPose | double *    | 输出    | 计算成功时关节空间位置计算结果: 关节 1~6 (单位: °)                |

#### 返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double srcPose[ROB_AXIS_NUM] = {-66.01, -438.24, 887.68, -57.34, 0, -148.98};
double refJointPose[ROB_AXIS_NUM] = { 0, 0, 0, 0, 0, 0 };
double tarJointPose[ROB_AXIS_NUM];
result = cr_kineInverse(robotHandle, srcPose, refJointPose, tarJointPose); //逆解
if (result != success) //判断逆解是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", tarJointPose[i]);
}
```



```
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6 配置

### 3.6.1 安装

#### 3.6.1.1 设置安装角度

##### 函数

**CRresult** cr\_cfg\_install\_angle\_set(**RobotHandle** robotHandle, **InstallAngle** installAngle)

##### 简介

设置安装角度。

##### 参数

| 参数名称         | 数据类型         | 输入/输出 | 描述                                             |
|--------------|--------------|-------|------------------------------------------------|
| robotHandle  | RobotHandle  | 输入    | 机械臂实例句柄                                        |
| installAngle | InstallAngle | 输入    | 安装角度设置<br>(基座旋转角度范围: 0-360<br>基座倾斜角度范围: 0-180) |

##### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

##### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
InstallAngle installAngle;
installAngle.baseAngle = 0;
installAngle.tiltAngle = 0;
result = cr_cfg_install_angle_set(robotHandle, installAngle); //设置安装角度
if (result != success) //判断设置安装角度是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.1.2 读取安装角度

#### 函数

`CRresult cr_cfg_install_angle_get(RobotHandle robotHandle, InstallAngle *installAngle)`

#### 简介

读取安装角度。

#### 参数

| 参数名称         | 数据类型           | 输入/输出 | 描述      |
|--------------|----------------|-------|---------|
| robotHandle  | RobotHandle    | 输入    | 机械臂实例句柄 |
| installAngle | InstallAngle * | 输出    | 安装角度    |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
InstallAngle installAngle;
result = cr_cfg_install_angle_get(robotHandle, &installAngle); //读取安装角度
if (result != success) //判断读取安装角度是否成功
    return -1;
printf("%f\n", installAngle.tiltAngle);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.2 TCP

### 3.6.2.1 读取 TCP 个数

#### 函数

`CRresult cr_cfg_tcp_count(RobotHandle robotHandle, int *count)`

#### 简介

读取 TCP 个数。

#### 参数

| 参数名称 | 数据类型 | 输入/输出 | 描述 |
|------|------|-------|----|
|------|------|-------|----|

|             |             |    |         |
|-------------|-------------|----|---------|
| robotHandle | RobotHandle | 输入 | 机械臂实例句柄 |
| count       | int *       | 输出 | tcp 个数  |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int count;
result = cr_cfg_tcp_count(robotHandle, &count); //读取 TCP 个数
if (result != success) //判断读取 TCP 个数是否成功
    return -1;
printf("%d\n", count);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.2.2 读取 TCP 数据

### 函数

**CRresult cr\_cfg\_tcp\_get(RobotHandle robotHandle, int index, TCPMsg \*tcpMsg)**

### 简介

读取 TCP 数据。

### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                |
|-------------|-------------|-------|-------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄           |
| index       | int         | 输入    | tcp 索引 (范围: 0-19) |
| tcpMsg      | TCPMsg *    | 输出    | tcp 数据            |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
TCPMsg tcpMsg;
```

```
int index = 1;
result = cr_cfg_tcp_get(robotHandle, index, &tcpMsg); //读取索引号为 1 的 TCP 数据
if (result != success) //判断读取 TCP 数据是否成功
    return -1;
printf("%s\n", tcpMsg.tcpName);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.2.3 读取 TCP 数据(通过名称)

#### 函数

```
CRresult cr_cfg_tcp_accord_name_get(RobotHandle robotHandle, char* tcpName, TCPMsg
*tcpMsg)
```

#### 简介

读取 TCP 数据(通过名称)。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| tcpName     | char*       | 输入    | tcp 名称  |
| tcpMsg      | TCPMsg *    | 输出    | tcp 数据  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
TCPMsg tcpMsg;
char tcpName[10] = "TCP_1";
result = cr_cfg_tcp_accord_name_get(robotHandle, tcpName, &tcpMsg);
//读取名称为 TCP_1 的 TCP 数据
if (result != success) //判断读取 TCP 数据是否成功
    return -1;
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", tcpMsg.tcpOffset[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
```

```
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.2.4 增加 TCP

#### 函数

`CRresult cr_cfg_tcp_add(RobotHandle robotHandle, TCPMsg tcpMsg)`

#### 简介

增加 TCP。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| tcpMsg      | TCPMsg      | 输入    | tcp 数据  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
TCPMsg tcpMsg;
int i;
char name[20] = { "TCP" };
strcpy_s(tcpMsg.tcpName, sizeof(name), name);
double offset[6] = { 3, 10, 9, 90, 80, 70 };
for (i = 0; i < 6; i++)
{
    tcpMsg.tcpOffset[i] = offset[i];
}
result = cr_cfg_tcp_add(robotHandle, tcpMsg); //增加 TCP
if (result != success) //判断增加 TCP 是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.2.5 删除 TCP

#### 函数

**CRresult** cr\_cfg\_tcp\_delete(**RobotHandle** robotHandle,int index)

#### 简介

删除 TCP。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述              |
|-------------|-------------|-------|-----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄         |
| index       | int         | 输入    | tcp 索引（范围：0-19） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 2;
result = cr_cfg_tcp_delete(robotHandle, index); //删除索引号为 2 的 TCP
if (result != success) //判断删除 TCP 是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.2.6 编辑 TCP 数据

#### 函数

**CRresult** cr\_cfg\_tcp\_set(**RobotHandle** robotHandle,int index,**TCPMsg** tcpMsg)

#### 简介

编辑 TCP 数据。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述              |
|-------------|-------------|-------|-----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄         |
| index       | int         | 输入    | tcp 索引（范围：0-19） |
| tcpMsg      | TCPMsg      | 输入    | tcp 数据          |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
TCPMsg tcpMsg;
char name[20] = { "TCP" };
strcpy_s(tcpMsg.tcpName, sizeof(name), name);
double offset[6] = { 3, 10, 9, 90, 80, 70 };
for (int i = 0; i < 6; i++)
{
    tcpMsg.tcpOffset[i] = offset[i];
}
result = cr_cfg_tcp_set(robotHandle, index, tcpMsg); //编辑索引号为 1 的 TCP 数据
if (result != success) //判断编辑 TCP 数据是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.2.7 读取当前激活的 TCP 索引

## 函数

CRresult cr\_cfg\_tcp\_active\_get(RobotHandle robotHandle, int \*index)

## 简介

读取当前激活的 TCP 索引。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述              |
|-------------|-------------|-------|-----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄         |
| index       | int *       | 输出    | tcp 索引（范围：0-19） |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
```

```
if (result != success) //判断机器人连接是否成功
    return -1;
int index;
result = cr_cfg_tcp_active_get(robotHandle, &index); //读取当前激活的 TCP 索引
if (result != success) //判断读取当前激活的 TCP 索引是否成功
    return -1;
printf("%d\n", index);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.2.8 激活 TCP

#### 函数

**CRresult** cr\_cfg\_tcp\_active\_set(**RobotHandle** robotHandle,int index)

#### 简介

激活 TCP。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述              |
|-------------|-------------|-------|-----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄         |
| index       | int         | 输入    | tcp 索引（范围：0-19） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 2;
result = cr_cfg_tcp_active_set(robotHandle, index); //激活索引号为 2 的 TCP
if (result != success) //判断激活 TCP 是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```



### 3.6.2.9 计算 TCP 位置

#### 函数

```
CRresult cr_compute_tcp_position(double poses[][6], int posesLen, double *position, int positionLen, double *err)
```

#### 简介

计算 TCP 位置。

#### 参数

| 参数名称        | 数据类型     | 输入/输出 | 描述                                            |
|-------------|----------|-------|-----------------------------------------------|
| poses       | double * | 输入    | 多组位姿数据，每组位姿数据都是轴角：x，y，z（单位：mm）；Rx，Ry，Rz（单位：°） |
| posesLen    | int      | 输入    | 位姿数量，至少为 4                                    |
| positionLen | int      | 输入    | xyz 的长度，至少为 3                                 |
| position    | double * | 输出    | tcp 的 xyz 值：x，y，z（单位：mm）                      |
| err         | double * | 输出    | 位置误差                                          |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
double poses[][6] = { {4.31,-66.66,934.53,20.30,-121.50,-73.277},
                      {-3.71,-68.75,926.57,12.589,-124.54,-77.42},
                      {13.98,-65.98,934.53,20.30,-121.5,-73.27},
                      {4.2558,-66.66,934.53,8.77,-107.15,-71.457} };

int posesLen = 4;
double position[3];
int positionLen = 3;
double err = 0;
CRresult result = cr_compute_tcp_position(poses, posesLen, position, positionLen, &err);
//计算 TCP 位置
if (result != success) //判断计算 TCP 位置是否成功
    return -1;
for (int i = 0; i < 3; i++)
{
    printf("%f\n", position[i]);
}
```

### 3.6.2.10 计算 TCP 角度

#### 函数

```
CRresult cr_compute_tcp_orientation(double *csPose,int csLen,double *pointPose,int pointLen,
double *orientation,int orientationLen)
```

#### 简介

计算 TCP 角度。

#### 参数

| 参数名称           | 数据类型    | 输入/输出 | 描述                                   |
|----------------|---------|-------|--------------------------------------|
| csPose         | double* | 输入    | 参考坐标系的位姿：x，y，z（单位：mm）；Rx，Ry，Rz（单位：°） |
| csLen          | int     | 输入    | 参考坐标系位姿的长度，至少为 6                     |
| pointPose      | double* | 输入    | 任意一点的位姿：x，y，z（单位：mm）；Rx，Ry，Rz（单位：°）  |
| pointLen       | int     | 输入    | 点位姿的长度，至少为 6                         |
| orientationLen | int     | 输入    | rxyz 的长度，至少为 3                       |
| orientation    | double* | 输出    | tcp 的 rxyz 值：Rx，Ry，Rz（单位：°）          |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
double csPose[6] = { 0, 0, 0, 0, 0, 0 };
int csLen = 6;
double pointPose[6] = { -36.10, -4.42, 32.62, 13.54, -91.33, 39.52 };
int pointLen = 6;
double orientation[3];
int orientationLen = 3;
CRresult result = cr_compute_tcp_orientation(csPose, csLen, pointPose, pointLen, orientation,
orientationLen); //计算 TCP 角度
if (result != success) //判断计算 TCP 角度是否成功
    return -1;
for (int i = 0; i < 3; i++)
{
    printf("%f\n", orientation[i]);
}
```

## 3.6.3 负载

### 3.6.3.1 读取负载个数

#### 函数

**CRresult** cr\_cfg\_payload\_count(**RobotHandle** robotHandle,int \*count)

#### 简介

读取负载个数。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| count       | int *       | 输出    | 负载个数    |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int count;
result = cr_cfg_payload_count(robotHandle, &count); //读取负载个数
if (result != success) //判断读取负载个数是否成功
    return -1;
printf("%d\n", count);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.3.2 读取负载数据

#### 函数

**CRresult** cr\_cfg\_payload\_get(**RobotHandle** robotHandle,int index,**PayLoad** \*payLoad)

#### 简介

读取负载数据。

#### 参数

| 参数名称 | 数据类型 | 输入/输出 | 描述 |
|------|------|-------|----|
|------|------|-------|----|

|             |             |    |               |
|-------------|-------------|----|---------------|
| robotHandle | RobotHandle | 输入 | 机械臂实例句柄       |
| index       | int         | 输入 | 负载索引（范围：0-19） |
| payLoad     | PayLoad *   | 输出 | 负载信息          |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
PayLoad payLoad;
int index = 1;
result = cr_cfg_payload_get(robotHandle, index, &payLoad); //读取索引号为 1 的负载数据
if (result != success) //判断读取负载数据是否成功
    return -1;
printf("%s\n", payLoad.payloadName);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.3.3 读取负载数据(通过名称)

#### 函数

```
CRresult cr_cfg_payload_accord_name_get(RobotHandle robotHandle, char* payloadName,
PayLoad *payLoad)
```

#### 简介

读取负载数据(通过名称)。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| payloadName | char*       | 输入    | 负载名称    |
| payLoad     | PayLoad *   | 输出    | 负载信息    |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
```

```

if (result != success) //判断机器人连接是否成功
    return -1;
Payload payload;
char payloadName[10] = "Payload_1";
result = cr_cfg_payload_accord_name_get(robotHandle, payloadName, &payload);
//读取名称为 Payload_1 的负载数据
if (result != success) //判断读取负载数据是否成功
    return -1;
printf("%f\n", payload.toolPayload);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
    
```

### 3.6.3.4 增加负载

#### 函数

**CRresult** cr\_cfg\_payload\_add(**RobotHandle** robotHandle, **Payload** payload)

#### 简介

增加负载。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| payload     | Payload     | 输入    | 负载信息    |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
Payload payload;
char name[20] = { "payload" };
strcpy_s(payload.payloadName, sizeof(name), name);
payload.toolPayload = 6;
double centerOfGravity[3] = { 0,0,30 };
for (int i = 0; i < 3; i++)
{
    payload.centerOfGravity[i] = centerOfGravity[i];
}
result = cr_cfg_payload_add(robotHandle, payload); //增加负载
    
```

```
if (result != success) //判断增加负载是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.3.5 删除负载

#### 函数

**CRresult** cr\_cfg\_payload\_delete(**RobotHandle** robotHandle,int index)

#### 简介

删除负载。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述            |
|-------------|-------------|-------|---------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄       |
| index       | int         | 输入    | 负载索引（范围：0-19） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 2;
result = cr_cfg_payload_delete(robotHandle, index); //删除索引号为 2 的负载
if (result != success) //判断删除负载是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.3.6 编辑负载

#### 函数

**CRresult** cr\_cfg\_payload\_set(**RobotHandle** robotHandle,int index,**PayLoad** payLoad)

#### 简介

编辑负载。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述            |
|-------------|-------------|-------|---------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄       |
| index       | int         | 输入    | 负载索引（范围：0-19） |
| payLoad     | PayLoad     | 输入    | 负载信息          |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
PayLoad payLoad;
int index = 1;
char name[20] = { "payload" };
strcpy_s(payLoad.payloadName, sizeof(name), name);
payLoad.toolPayload = 6;
double centerOfGravity[3] = { 0,0,30 };
for (int i = 0; i < 3; i++)
{
    payLoad.centerOfGravity[i] = centerOfGravity[i];
}
result = cr_cfg_payload_set(robotHandle, index, payLoad); //编辑索引号为 1 的负载
if (result != success) //判断编辑负载是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.3.7 读取当前激活的负载索引

## 函数

CRresult cr\_cfg\_payload\_active\_get(RobotHandle robotHandle, int \*index)

## 简介

读取当前激活的负载索引。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述            |
|-------------|-------------|-------|---------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄       |
| index       | int *       | 输出    | 负载索引（范围：0-19） |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index;
result = cr_cfg_payload_active_get(robotHandle, &index); //读取当前激活的负载索引
if (result != success) //判断读取当前激活的负载索引是否成功
    return -1;
printf("%d\n", index);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.3.8 激活负载

## 函数

**CRresult** cr\_cfg\_payload\_active\_set(**RobotHandle** robotHandle, int index)

## 简介

激活负载。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述            |
|-------------|-------------|-------|---------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄       |
| index       | int         | 输入    | 负载索引（范围：0-19） |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 2;
result = cr_cfg_payload_active_set(robotHandle, index); //激活索引号为 2 的负载
if (result != success) //判断激活负载是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
```



```
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.4 初始位/包装位

### 3.6.4.1 设置初始位

#### 函数

**CRresult** cr\_cfg\_home\_pose\_set(**RobotHandle** robotHandle, double \*homePose, int len)

#### 简介

设置初始位。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述         |
|-------------|-------------|-------|------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄    |
| homePose    | double *    | 输入    | 初始位姿数据     |
| len         | int         | 输入    | 数组长度，长度为 6 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double homePose[6] = { 60,60,60,60,60,60 };
int len = 6;
result = cr_cfg_home_pose_set(robotHandle, homePose, len); //设置初始位
if (result != success) //判断设置初始位是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.4.2 读取初始位

#### 函数

**CRresult** cr\_cfg\_home\_pose\_get(**RobotHandle** robotHandle, double \*homePose, int len)

#### 简介

读取初始位。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述         |
|-------------|-------------|-------|------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄    |
| homePose    | double *    | 输出    | 初始位姿数据     |
| len         | int         | 输入    | 数组长度，长度为 6 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double homePose[6];
int len = 6;
result = cr_cfg_home_pose_get(robotHandle, homePose, len); //读取初始位
if (result != success) //判断读取初始位是否成功
    return -1;
for (int i = 0; i < 6; i++)
{
    printf("%f\n", homePose[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.4.3 删除初始位

## 函数

CRresult cr\_cfg\_home\_pose\_delete(RobotHandle robotHandle)

## 简介

删除初始位。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_cfg_home_pose_delete(robotHandle); //删除初始位
if (result != success) //判断删除初始位是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.4.4 读取包装位

## 函数

**CRresult** cr\_cfg\_pack\_pose\_get(**RobotHandle** robotHandle, double \*packPose, int len)

## 简介

读取包装位。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述         |
|-------------|-------------|-------|------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄    |
| packPose    | double *    | 输出    | 包装位数据      |
| len         | int         | 输入    | 数组长度，长度为 6 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double packPose[6];
int len = 6;
result = cr_cfg_pack_pose_get(robotHandle, packPose, len); //读取包装位
if (result != success) //判断读取包装位是否成功
    return -1;
for (int i = 0; i < 6; i++)
{
    printf("%f\n", packPose[i]);
}
```

```
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.5 启动

### 3.6.5.1 设置机器人是否自动上电使能

#### 函数

**CRresult** cr\_cfg\_poweron\_auto\_set(**RobotHandle** robotHandle, **BOOL** isAuto)

#### 简介

设置机器人是否自动上电使能。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                                |
|-------------|-------------|-------|-----------------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                           |
| isAuto      | BOOL        | 输入    | 是否自动上电<br>(FALSE-不自动上电 TRUE-自动上电) |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
BOOL isAuto = TRUE;
result = cr_cfg_poweron_auto_set(robotHandle, isAuto); //设置机器人自动上电使能
if (result != success) //判断设置机器人自动上电使能是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.5.2 读取机器人自动上电状态

#### 函数

**CRresult** cr\_cfg\_poweron\_auto\_get(**RobotHandle** robotHandle, **BOOL** \*isAuto)

## 简介

读取机器人自动上电状态。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                |
|-------------|-------------|-------|-------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄           |
| isAuto      | BOOL *      | 输出    | 是否自动上电的状态：0-否，1-是 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
BOOL isAuto;
result = cr_cfg_poweron_auto_get(robotHandle, &isAuto); //读取机器人自动上电状态
if (result != success) //判断读取机器人自动上电状态是否成功
    return -1;
printf("%d\n", isAuto);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.6 控制参数

### 3.6.6.1 设置机器人拖动阻尼

## 函数

```
CRresult cr_cfg_joint_drag_damping_set(RobotHandle robotHandle, int* jointDargDamping,
const int arrSize)
```

## 简介

设置机器人拖动阻尼。

## 参数

| 参数名称             | 数据类型        | 输入/输出 | 描述                 |
|------------------|-------------|-------|--------------------|
| robotHandle      | RobotHandle | 输入    | 机械臂实例句柄            |
| jointDargDamping | int*        | 输入    | 机器人拖动阻尼数组(关节 1-6)  |
| arrSize          | const int   | 输入    | 机器人拖动阻尼数组大小(一般为 6) |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int jointDargDamping[6] = {50,50,50,50,50,50};
int arrSize = 6;
result = cr_cfg_joint_drag_damping_set(robotHandle, jointDargDamping, arrSize);
//设置机器人拖动阻尼
if (result != success) //判断设置机器人拖动阻尼是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.6.2 读取机器人拖动阻尼

## 函数

**CRresult** cr\_cfg\_joint\_drag\_damping\_get(**RobotHandle** robotHandle, int\* jointDargDamping, const int arrSize)

## 简介

读取机器人拖动阻尼。

## 参数

| 参数名称             | 数据类型        | 输入/输出 | 描述                 |
|------------------|-------------|-------|--------------------|
| robotHandle      | RobotHandle | 输入    | 机械臂实例句柄            |
| jointDargDamping | int*        | 输出    | 机器人拖动阻尼数组(关节 1-6)  |
| arrSize          | const int   | 输入    | 机器人拖动阻尼数组大小(一般为 6) |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int jointDargDamping[6] = {0};
int arrSize = 6;
```

```
result = cr_cfg_joint_drag_damping_get(robotHandle, jointDargDamping, arrSize);
//读取机器人拖动阻尼
if (result != success) //判断读取机器人拖动阻尼是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.6.3 设置转矩异常门限倍率

#### 函数

**CRresult** cr\_cfg\_torque\_threshold\_ratio\_set(**RobotHandle** robotHandle, double\* torqueThresholdRatio, const int arrSize)

#### 简介

设置转矩异常门限倍率。

#### 参数

| 参数名称                 | 数据类型        | 输入/输出 | 描述                     |
|----------------------|-------------|-------|------------------------|
| robotHandle          | RobotHandle | 输入    | 机械臂实例句柄                |
| torqueThresholdRatio | double*     | 输入    | 机器人转矩异常门限倍率数组(关节 1-6)  |
| arrSize              | const int   | 输入    | 机器人转矩异常门限倍率数组大小(一般为 6) |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double torqueThresholdRatio[6] = {1,1,1,1,1,1};
const int arrSize = 6;
result = cr_cfg_torque_threshold_ratio_set(robotHandle, torqueThresholdRatio, arrSize);
//设置转矩异常门限倍率
if (result != success) //判断设置转矩异常门限倍率是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.6.4 读转矩异常门限倍率

#### 函数

```
CRresult cr_cfg_torque_threshold_ratio_get(RobotHandle robotHandle, double* torqueThresholdRatio, const int arrSize)
```

#### 简介

读转矩异常门限倍率。

#### 参数

| 参数名称                 | 数据类型        | 输入/输出 | 描述                     |
|----------------------|-------------|-------|------------------------|
| robotHandle          | RobotHandle | 输入    | 机械臂实例句柄                |
| torqueThresholdRatio | double*     | 输出    | 机器人转矩异常门限倍率数组(关节 1-6)  |
| arrSize              | const int   | 输入    | 机器人转矩异常门限倍率数组大小(一般为 6) |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double torqueThresholdRatio[6];
const int arrSize = 6;
result = cr_cfg_torque_threshold_ratio_get(robotHandle, torqueThresholdRatio, arrSize);
//读转矩异常门限倍率
if (result != success) //判断读转矩异常门限倍率是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.6.5 设置扩展轴步长及速度倍率

#### 函数

```
CRresult cr_cfg_exj_move_speed_ratio_set(RobotHandle robotHandle, double exjMoveSpeedRatio)
```

#### 简介

设置扩展轴步长及速度倍率。



### 参数

| 参数名称              | 数据类型        | 输入/输出 | 描述         |
|-------------------|-------------|-------|------------|
| robotHandle       | RobotHandle | 输入    | 机械臂实例句柄    |
| exjMoveSpeedRatio | double      | 输入    | 扩展轴步长及速度倍率 |

### 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
double exjMoveSpeedRatio = 1;
result = cr_cfg_exj_move_speed_ratio_set(robotHandle, exjMoveSpeedRatio);
//设置扩展轴步长及速度倍率
if (result != success) //判断设置扩展轴步长及速度倍率是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.6.6 读扩展轴步长及速度倍率

### 函数

```
CResult cr_cfg_exj_move_speed_ratio_get(RobotHandle robotHandle, double*
exjMoveSpeedRatio)
```

### 简介

读扩展轴步长及速度倍率。

### 参数

| 参数名称              | 数据类型        | 输入/输出 | 描述         |
|-------------------|-------------|-------|------------|
| robotHandle       | RobotHandle | 输入    | 机械臂实例句柄    |
| exjMoveSpeedRatio | double*     | 输出    | 扩展轴步长及速度倍率 |

### 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
```

```

        return -1;
double exjMoveSpeedRatio;
result = cr_cfg_exj_move_speed_ratio_get(robotHandle, &exjMoveSpeedRatio);
//读扩展轴步长及速度倍率
if (result != success) //判断读扩展轴步长及速度倍率是否成功
    return -1;
printf("%f\n", exjMoveSpeedRatio);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
    
```

## 3.6.7 机器人限值

### 3.6.7.1 设置限制模式

#### 函数

**CRresult cr\_cfg\_safety\_limit\_type\_set(RobotHandle robotHandle, SafetyLimitsValuesType safetyLimitsValuesType)**

#### 简介

设置限制模式。

#### 参数

| 参数名称                   | 数据类型                   | 输入/输出 | 描述      |
|------------------------|------------------------|-------|---------|
| robotHandle            | RobotHandle            | 输入    | 机械臂实例句柄 |
| safetyLimitsValuesType | SafetyLimitsValuesType | 输入    | 机器人限制等级 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SafetyLimitsValuesType safetyLimitsValuesType;
safetyLimitsValuesType = LimitLevel_1;
result = cr_cfg_safety_limit_type_set(robotHandle, safetyLimitsValuesType);
//设置限制模式为等级 1（限制最多）
if (result != success) //判断设置限制模式是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    
```

```
return -1;
```

### 3.6.7.2 读取限制模式

#### 函数

```
CRresult cr_cfg_safety_limit_type_get(RobotHandle robotHandle, SafetyLimitsValuesType
*safetyLimitsValuesType)
```

#### 简介

读取限制模式。

#### 参数

| 参数名称                   | 数据类型                     | 输入/输出 | 描述      |
|------------------------|--------------------------|-------|---------|
| robotHandle            | RobotHandle              | 输入    | 机械臂实例句柄 |
| safetyLimitsValuesType | SafetyLimitsValuesType * | 输出    | 限制模式    |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SafetyLimitsValuesType safetyLimitsValuesType;
result = cr_cfg_safety_limit_type_get(robotHandle, &safetyLimitsValuesType); //读取限制模式
if (result != success) //判断读取限制模式是否成功
    return -1;
printf("%d\n", safetyLimitsValuesType);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.7.3 设置自定义模式下参数

#### 函数

```
CRresult cr_cfg_safety_limit_para_set(RobotHandle robotHandle, SafetyLimitsMode
mode, SafetyLimitsValues safetyLimitsValues);
```

#### 简介

设置自定义模式下参数。

#### 参数

| 参数名称 | 数据类型 | 输入/输出 | 描述 |
|------|------|-------|----|
|------|------|-------|----|

|                    |                    |    |         |
|--------------------|--------------------|----|---------|
| robotHandle        | RobotHandle        | 输入 | 机械臂实例句柄 |
| mode               | SafetyLimitsMode   | 输入 | 安全限值模式  |
| safetyLimitsValues | SafetyLimitsValues | 输入 | 安全限值    |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SafetyLimitsValues safetyLimitsValues;
SafetyLimitsMode mode = mode_Reduced;
safetyLimitsValues.maxPower = 300;
safetyLimitsValues.maxMomentum = 25;
safetyLimitsValues.maxStoppingTime = 400;
safetyLimitsValues.maxStoppingDistance = 500;
safetyLimitsValues.maxTcpSpeed = 1500;
safetyLimitsValues.maxForce = 150;
safetyLimitsValues.maxElbowSpeed = 1500;
safetyLimitsValues.maxElbowForce = 100;
result = cr_cfg_safety_limit_para_set(robotHandle, mode, safetyLimitsValues);
//设置自定义模式下参数
if (result != success) //判断设置自定义模式下参数是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.7.4 读取自定义模式下参数

### 函数

**CRresult** cr\_cfg\_safety\_limit\_para\_get(**RobotHandle** robotHandle, **SafetyLimitsMode** mode, **SafetyLimitsValues** \*safetyLimitsValues)

### 简介

读取自定义模式下参数。

### 参数

| 参数名称        | 数据类型             | 输入/输出 | 描述      |
|-------------|------------------|-------|---------|
| robotHandle | RobotHandle      | 输入    | 机械臂实例句柄 |
| mode        | SafetyLimitsMode | 输入    | 安全限值模式  |

|                    |                      |    |      |
|--------------------|----------------------|----|------|
| safetyLimitsValues | SafetyLimitsValues * | 输出 | 安全限值 |
|--------------------|----------------------|----|------|

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SafetyLimitsValues safetyLimitsValues;
SafetyLimitsMode mode = mode_Normal;
result = cr_cfg_safety_limit_para_get(robotHandle, mode, &safetyLimitsValues);
//读取自定义模式下参数
if (result != success) //判断读取自定义模式下参数是否成功
    return -1;
printf("%f\n", safetyLimitsValues.maxElbowSpeed);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.7.5 设置碰撞后处理方式

### 函数

```
CRresult cr_cfg_safety_collihandle_type_set(RobotHandle robotHandle,
SafetyCollisionHandleMode safetyCollisionHandleMode);
```

### 简介

设置碰撞后处理方式。

### 参数

| 参数名称                      | 数据类型                      | 输入/输出 | 描述      |
|---------------------------|---------------------------|-------|---------|
| robotHandle               | RobotHandle               | 输入    | 机械臂实例句柄 |
| safetyCollisionHandleMode | SafetyCollisionHandleMode | 输入    | 碰撞后处理方式 |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SafetyCollisionHandleMode safetyCollisionHandleMode = Collision_EnterReboundMode;
```

```
result = cr_cfg_safety_collihandle_type_set(robotHandle, safetyCollisionHandleMode);
//将碰撞后处理方式设置为进入反弹模式
if (result != success) //判断设置碰撞后处理方式是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.7.6 读取碰撞后处理方式

#### 函数

```
CRresult cr_cfg_safety_collihandle_type_get(RobotHandle robotHandle,
SafetyCollisionHandleMode *safetyCollisionHandleMode);
```

#### 简介

读取碰撞后处理方式。

#### 参数

| 参数名称                      | 数据类型                        | 输入/输出 | 描述      |
|---------------------------|-----------------------------|-------|---------|
| robotHandle               | RobotHandle                 | 输入    | 机械臂实例句柄 |
| safetyCollisionHandleMode | SafetyCollisionHandleMode * | 输出    | 碰撞后处理方式 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SafetyCollisionHandleMode safetyCollisionHandleMode;
result = cr_cfg_safety_collihandle_type_get(robotHandle, &safetyCollisionHandleMode);
//读取碰撞后处理方式
if (result != success) //判断读取碰撞后处理方式是否成功
    return -1;
printf("碰撞后处理方式:%d", safetyCollisionHandleMode);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.8 关节限值

### 3.6.8.1 设置关节限值

#### 函数

```
CRresult cr_cfg_safety_joint_limit_set(RobotHandle robotHandle, SafetyLimitsMode mode, int jointIndex, SafetyLimitsJointAngle safetyLimitJoint)
```

#### 简介

设置关节限值。

#### 参数

| 参数名称             | 数据类型                   | 输入/输出 | 描述           |
|------------------|------------------------|-------|--------------|
| robotHandle      | RobotHandle            | 输入    | 机械臂实例句柄      |
| mode             | SafetyLimitsMode       | 输入    | 关节模式         |
| jointIndex       | int                    | 输入    | 关节序号（范围：0-5） |
| safetyLimitJoint | SafetyLimitsJointAngle | 输入    | 关节限值         |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
SafetyLimitsJointAngle safetyLimitJoint;
safetyLimitJoint.maxJointPosition = 200;
safetyLimitJoint.minJointPosition = -200;
safetyLimitJoint.maxJointSpeed = 150;
SafetyLimitsMode mode = mode_Reduced;
result = cr_cfg_safety_joint_limit_set(robotHandle, mode, index, safetyLimitJoint);
//设置 1 关节的关节限值
if (result != success) //判断设置关节限值是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.8.2 读取关节限值

#### 函数

```
CRresult cr_cfg_safety_joint_limit_get(RobotHandle robotHandle, SafetyLimitsMode mode, int jointIndex, SafetyLimitsJointAngle *safetyLimitJoint)
```

#### 简介

读取关节限值。

#### 参数

| 参数名称             | 数据类型                    | 输入/输出 | 描述           |
|------------------|-------------------------|-------|--------------|
| robotHandle      | RobotHandle             | 输入    | 机械臂实例句柄      |
| mode             | SafetyLimitsMode        | 输入    | 关节模式         |
| jointIndex       | int                     | 输入    | 关节序号（范围：0-5） |
| safetyLimitJoint | SafetyLimitsJointAngle* | 输出    | 关节限值         |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SafetyLimitsJointAngle safetyLimitJoint;
SafetyLimitsMode mode = mode_Normal;
int jointIndex = 1;
result = cr_cfg_safety_joint_limit_get(robotHandle, mode, jointIndex, &safetyLimitJoint);
//读取 1 关节的关节限值
if (result != success) //判断读取关节限值是否成功
    return -1;
printf("%f\n", safetyLimitJoint.maxJointPosition);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.9 安全平面

### 3.6.9.1 读取安全平面个数

#### 函数

```
CRresult cr_cfg_safety_plane_count(RobotHandle robotHandle, int *count)
```



## 简介

读取安全平面个数。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| count       | int *       | 输出    | 安全平面数量  |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int count;
result = cr_cfg_safety_plane_count(robotHandle, &count); //读取安全平面个数
if (result != success) //判断读取安全平面个数是否成功
    return -1;
printf("%d\n", count);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.9.2 读取安全平面信息

## 函数

**CRresult** cr\_cfg\_safety\_plane\_get(**RobotHandle** robotHandle, int index, **SafetyLimitsBoundaryPlane** \*safetyLimitsBoundaryPlane)

## 简介

读取安全平面信息。

## 参数

| 参数名称                      | 数据类型                       | 输入/输出 | 描述                 |
|---------------------------|----------------------------|-------|--------------------|
| robotHandle               | RobotHandle                | 输入    | 机械臂实例句柄            |
| index                     | int                        | 输入    | 安全平面序号<br>(范围：0-7) |
| safetyLimitsBoundaryPlane | SafetyLimitsBoundaryPlane* | 输出    | 安全平面信息             |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
SafetyLimitsBoundaryPlane safetyLimitsBoundaryPlane;
result = cr_cfg_safety_plane_get(robotHandle, index, &safetyLimitsBoundaryPlane);
//读取序号为 1 的安全平面信息
if (result != success) //判断读取安全平面信息是否成功
    return -1;
printf("%s\n", safetyLimitsBoundaryPlane.name);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.9.3 增加安全平面

## 函数

**CRresult** cr\_cfg\_safety\_plane\_add(**RobotHandle** robotHandle, **SafetyLimitsBoundaryPlane** safetyLimitsBoundaryPlane)

## 简介

增加安全平面。

## 参数

| 参数名称                      | 数据类型                      | 输入/输出 | 描述      |
|---------------------------|---------------------------|-------|---------|
| robotHandle               | RobotHandle               | 输入    | 机械臂实例句柄 |
| safetyLimitsBoundaryPlane | SafetyLimitsBoundaryPlane | 输入    | 安全平面信息  |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SafetyLimitsBoundaryPlane safetyLimitsBoundaryPlane;
safetyLimitsBoundaryPlane.mode = Normal;
safetyLimitsBoundaryPlane.id = 4;
safetyLimitsBoundaryPlane.elbowRestricted = 0;
safetyLimitsBoundaryPlane.sourceGeomFeatureType = baseCoordinate;
```

```
safetyLimitsBoundaryPlane.displacement = 11;
char name[20] = { "safeplane" };
strcpy_s(safetyLimitsBoundaryPlane.name, sizeof(name), name);
result = cr_cfg_safety_plane_add(robotHandle, safetyLimitsBoundaryPlane); //增加安全平面
if (result != success) //判断增加安全平面是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.9.4 删除安全平面

#### 函数

**CRresult** cr\_cfg\_safety\_plane\_delete(**RobotHandle** robotHandle, int index)

#### 简介

删除安全平面。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述             |
|-------------|-------------|-------|----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄        |
| index       | int         | 输入    | 安全平面序号（范围：0-7） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
result = cr_cfg_safety_plane_delete(robotHandle, index); //删除序号为 1 的安全平面
if (result != success) //判断删除安全平面是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.9.5 修改安全平面数据

#### 函数

**CRresult** cr\_cfg\_safety\_plane\_set(**RobotHandle** robotHandle, int index,

## SafetyLimitsBoundaryPlane safetyLimitsBoundaryPlane)

### 简介

修改安全平面数据。

### 参数

| 参数名称                      | 数据类型                      | 输入/输出 | 描述             |
|---------------------------|---------------------------|-------|----------------|
| robotHandle               | RobotHandle               | 输入    | 机械臂实例句柄        |
| index                     | int                       | 输入    | 安全平面序号（范围：0-7） |
| safetyLimitsBoundaryPlane | SafetyLimitsBoundaryPlane | 输入    | 安全平面信息         |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
SafetyLimitsBoundaryPlane safetyLimitsBoundaryPlane;
safetyLimitsBoundaryPlane.mode = Normal;
safetyLimitsBoundaryPlane.id = 4;
safetyLimitsBoundaryPlane.elbowRestricted = 0;
safetyLimitsBoundaryPlane.sourceGeomFeatureType = baseCoordinate;
safetyLimitsBoundaryPlane.displacement = 11;
char name[20] = { "safeplane" };
strcpy_s(safetyLimitsBoundaryPlane.name, sizeof(name), name);
result = cr_cfg_safety_plane_set(robotHandle, index, safetyLimitsBoundaryPlane);
//修改序号为 1 的安全平面数据
if (result != success) //判断修改安全平面数据是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.10 安全 I/O

### 3.6.10.1 设置可配置输入信号

#### 函数

CRresult cr\_cfg\_safety\_io\_input\_set(RobotHandle robotHandle, int index, SafetyIOInput

safetyinput)

### 简介

设置可配置输入信号。

### 参数

| 参数名称        | 数据类型          | 输入/输出 | 描述          |
|-------------|---------------|-------|-------------|
| robotHandle | RobotHandle   | 输入    | 机械臂实例句柄     |
| index       | int           | 输入    | 索引，输入范围：0-3 |
| safetyinput | SafetyIOInput | 输入    | 可配置输入信号     |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SafetyIOInput safetyinput;
safetyinput = reducedMode;
int index = 1;
result = cr_cfg_safety_io_input_set(robotHandle, index, safetyinput);
//设置序号为 1 的可配置输入信号为缩减模式
if (result != success) //判断设置可配置输入信号是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.10.2 读取可配置输入信号

### 函数

CRresult cr\_cfg\_safety\_io\_input\_get(RobotHandle robotHandle, int index, SafetyIOInput \*safetyinput)

### 简介

读取可配置输入信号。

### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述          |
|-------------|-------------|-------|-------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄     |
| index       | int         | 输入    | 索引，输入范围：0-3 |

|             |                 |    |         |
|-------------|-----------------|----|---------|
| safetyinput | SafetyIOInput * | 输出 | 可配置输入信号 |
|-------------|-----------------|----|---------|

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SafetyIOInput safetyinput;
int index = 1;
result = cr_cfg_safety_io_input_get(robotHandle, index, &safetyinput);
//读取序号为 1 的可配置输入信号
if (result != success) //判断读取可配置输入信号是否成功
    return -1;
printf("%d\n", safetyinput);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.10.3 设置可配置输出信号

#### 函数

**CRresult** cr\_cfg\_safety\_io\_output\_set(**RobotHandle** robotHandle, int index, **SafetyIOOutput** safetyoutput)

#### 简介

设置可配置输出信号。

#### 参数

| 参数名称         | 数据类型          | 输入/输出 | 描述          |
|--------------|---------------|-------|-------------|
| robotHandle  | RobotHandle   | 输入    | 机械臂实例句柄     |
| index        | int           | 输入    | 索引，输入范围：0-3 |
| safetyoutput | SafetyIOInput | 输入    | 可配置输出信号     |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
```

```
int index = 1;
SafetyIOOutput safetyoutput = SafetyIOOutput::robotNotStopping;
result = cr_cfg_safety_io_output_set(robotHandle,index, safetyoutput);
//设置序号为 1 的可配置输出信号为机器人未停止
if (result != success) //判断设置可配置输出信号是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.10.4 读取可配置输出信号

#### 函数

```
CRresult cr_cfg_safety_io_output_get(RobotHandle robotHandle,int index, SafetyIOOutput
*safetyoutput)
```

#### 简介

读取可配置输出信号。

#### 参数

| 参数名称         | 数据类型             | 输入/输出 | 描述          |
|--------------|------------------|-------|-------------|
| robotHandle  | RobotHandle      | 输入    | 机械臂实例句柄     |
| index        | int              | 输入    | 索引，输入范围：0-3 |
| safetyoutput | SafetyIOOutput * | 输出    | 可配置输出信号     |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
SafetyIOOutput safetyoutput;
result = cr_cfg_safety_io_output_get(robotHandle,index, &safetyoutput);
//读取序号为 1 的可配置输出信号
if (result != success) //判断读取可配置输出信号是否成功
    return -1;
printf("%d\n", safetyoutput);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.11 示教器控制

### 3.6.11.1 设置是否启用示教器

#### 函数

**CRresult** cr\_cfg\_safety\_tp\_use\_set(**RobotHandle** robotHandle,**BOOL** isUse)

#### 简介

设置是否启用示教器。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                            |
|-------------|-------------|-------|-------------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                       |
| isUse       | BOOL        | 输入    | 是否启用示教器<br>(FALSE-禁用 TRUE-启用) |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
BOOL isUse = TRUE;
result = cr_cfg_safety_tp_use_set(robotHandle, isUse); //设置启用示教器
if (result != success) //判断设置启用示教器是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.11.2 读取当前是否启用示教器

#### 函数

**CRresult** cr\_cfg\_safety\_tp\_use\_get(**RobotHandle** robotHandle,**BOOL** \*isUse)

#### 简介

读取当前是否启用示教器。

#### 参数

| 参数名称 | 数据类型 | 输入/输出 | 描述 |
|------|------|-------|----|
|------|------|-------|----|



|             |             |    |                        |
|-------------|-------------|----|------------------------|
| robotHandle | RobotHandle | 输入 | 机械臂实例句柄                |
| isUse       | BOOL *      | 输出 | 是否启用示教器<br>(0-禁用 1-启用) |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
BOOL isUse;
result = cr_cfg_safety_tp_use_get(robotHandle, &isUse); //读取当前是否启用示教器
if (result != success) //判断读取当前是否启用示教器是否成功
    return -1;
printf("%d\n", isUse);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.12 IO 变量

### 3.6.12.1 设置数字输入信号配置

#### 函数

```
CRresult cr_cfg_var_di_set(RobotHandle robotHandle, VarDigitalIOType type, int index,
    IOInputConfiguration inputConfiguration)
```

#### 简介

设置数字输入信号配置。

#### 参数

| 参数名称               | 数据类型                 | 输入/输出 | 描述              |
|--------------------|----------------------|-------|-----------------|
| robotHandle        | RobotHandle          | 输入    | 机械臂实例句柄         |
| type               | VarDigitalIOType     | 输入    | IO 类型           |
| index              | int                  | 输入    | IO 索引 (范围: 0-7) |
| inputConfiguration | IOInputConfiguration | 输入    | 数字量输入配置         |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
VarDigitalIOType type = Digital;
int index = 1;
IOInputConfiguration inputConfiguration;
inputConfiguration.inputFilteringTime = 1;
inputConfiguration.inputActions = InputAction_NONE;
char name[NAMELENTH] = { "digital" };
for (int i = 0; i < NAMELENTH; i++)
{
    inputConfiguration.inputName[i] = name[i];
}
result = cr_cfg_var_di_set(robotHandle, type, index, inputConfiguration);
//设置索引号为 1 的数字输入信号配置
if (result != success) //判断设置数字输入信号配置是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.12.2 读取数字输入信号配置

## 函数

```
CRresult cr_cfg_var_di_get(RobotHandle robotHandle, VarDigitalIOType type, int index,
IOInputConfiguration *inputConfiguration)
```

## 简介

读取数字输入信号配置。

## 参数

| 参数名称               | 数据类型                   | 输入/输出 | 描述            |
|--------------------|------------------------|-------|---------------|
| robotHandle        | RobotHandle            | 输入    | 机械臂实例句柄       |
| type               | VarDigitalIOType       | 输入    | IO 类型         |
| index              | int                    | 输入    | IO 索引（范围：0-7） |
| inputConfiguration | IOInputConfiguration * | 输出    | 数字量输入配置       |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
VarDigitalIOType type = Digital;
int index = 1;
IOInputConfiguration inputConfiguration;
result = cr_cfg_var_di_get(robotHandle, type, index, &inputConfiguration);
//读取索引号为 1 的数字输入信号配置
if (result != success) //判断读取数字输入信号配置是否成功
    return -1;
printf("%d\n", type);
printf("%s\n", inputConfiguration.inputName);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.12.3 设置数字输出信号配置

## 函数

**CRresult** cr\_cfg\_var\_do\_set(**RobotHandle** robotHandle, **VarDigitalIOType** type, int index, **IOOutputConfiguration** outputConfiguration)

## 简介

设置数字输出信号配置。

## 参数

| 参数名称                | 数据类型                  | 输入/输出 | 描述            |
|---------------------|-----------------------|-------|---------------|
| robotHandle         | RobotHandle           | 输入    | 机械臂实例句柄       |
| type                | VarDigitalIOType      | 输入    | IO 类型         |
| index               | int                   | 输入    | IO 索引（范围：0-7） |
| outputConfiguration | IOOutputConfiguration | 输入    | 数字量输出配置       |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
VarDigitalIOType type = Digital;
```

```
int index = 1;
IOOutputConfiguration outputConfiguration;
outputConfiguration.outputActions = OutputAction_RobotON_LOW;
char name[NAMELENTH] = { "digital" };
for (int i = 0; i < NAMELENTH; i++)
{
    outputConfiguration.outputName[i] = name[i];
}
outputConfiguration.outputOptions = OutputOptions_Enable;
outputConfiguration.outputRule = OutputRule_Disable;
result = cr_cfg_var_do_set(robotHandle,type, index, outputConfiguration);
//设置索引号为 1 的数字输出信号配置
if (result != success) //判断设置数字输出信号配置是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.12.4 读取数字输出信号配置

#### 函数

```
CRresult cr_cfg_var_do_get(RobotHandle robotHandle,VarDigitalIOType type,int index,
IOOutputConfiguration *outputConfiguration)
```

#### 简介

读取数字输出信号配置。

#### 参数

| 参数名称                | 数据类型                    | 输入/输出 | 描述            |
|---------------------|-------------------------|-------|---------------|
| robotHandle         | RobotHandle             | 输入    | 机械臂实例句柄       |
| type                | VarDigitalIOType        | 输入    | IO 类型         |
| index               | int                     | 输入    | IO 索引（范围：0-7） |
| outputConfiguration | IOOutputConfiguration * | 输出    | 数字量输出配置       |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
VarDigitalIOType type = Digital;
int index = 1;
```

```
IOOutputConfiguration outputConfiguration;
result = cr_cfg_var_do_get(robotHandle, type, index, &outputConfiguration);
//读取索引号为 1 的数字输出信号配置
if (result != success) //判断读取数字输出信号配置是否成功
    return -1;
printf("%s\n", outputConfiguration.outputName);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.12.5 设置工具 IO 类型

#### 函数

**CRresult** cr\_cfg\_var\_tool\_io\_type\_set(**RobotHandle** robotHandle,int index,**ToolIoType** toolIoType)

#### 简介

设置工具 IO 类型。调用一次或多次该接口后，都需要先调用 **cr\_destroy\_robot** 接口释放当前句柄，然后再调用 **cr\_create\_robot** 接口重新创建新句柄。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述                                      |
|-------------|-------------|-------|-----------------------------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄                                 |
| index       | int         | 输入    | IO 索引（4 路 IO 范围：0~3；<br>10 路 IO 范围：0~9） |
| toolIoType  | ToolIoType  | 输入    | 工具 IO 类型                                |

#### 返回值

数据类型为 **CRresult**，若为 **success** 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 0;
ToolIoType toolIoType = IO_OUTPUT;
result = cr_cfg_var_tool_io_type_set(robotHandle, index, toolIoType);
//设置索引号为 0 的工具 IO 类型
if (result != success) //判断设置工具 IO 类型是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
```

```
return -1;
```

### 3.6.12.6 读取工具 IO 类型

#### 函数

```
CRresult cr_cfg_var_tool_io_type_get(RobotHandle robotHandle,int index,ToolIoType
*toolIoType)
```

#### 简介

读取工具 IO 类型。

#### 参数

| 参数名称        | 数据类型         | 输入/输出 | 描述                                      |
|-------------|--------------|-------|-----------------------------------------|
| robotHandle | RobotHandle  | 输入    | 机械臂实例句柄                                 |
| index       | int          | 输入    | IO 索引（4 路 IO 范围：0~3；<br>10 路 IO 范围：0~9） |
| toolIoType  | ToolIoType * | 输出    | 工具 IO 类型                                |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 0;
ToolIoType toolIoType;
result = cr_cfg_var_tool_io_type_get(robotHandle, index, &toolIoType);
//读取索引号为 0 的工具 IO 类型
if (result != success) //判断读取工具 IO 类型是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.12.7 设置模拟量输入信号类型

#### 函数

```
CRresult cr_cfg_var_ai_set(RobotHandle robotHandle,VarDigitalIOType type,int index,
AnalogInputConfiguration analogInputConfiguration)
```

#### 简介

设置模拟量输入信号类型。

## 参数

| 参数名称                     | 数据类型                     | 输入/输出 | 描述                 |
|--------------------------|--------------------------|-------|--------------------|
| robotHandle              | RobotHandle              | 输入    | 机械臂实例句柄            |
| type                     | VarDigitalIOType         | 输入    | 模拟量类型              |
| index                    | int                      | 输入    | 模拟量索引<br>(范围: 0-2) |
| analogInputConfiguration | AnalogInputConfiguration | 输入    | 模拟量输入配置            |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
AnalogInputConfiguration analogInputConfiguration;
VarDigitalIOType type = Analog;
char Name[NAMELENGTH] = { "AI0" };
int index = 1;
for(int i=0;i< NAMELENGTH;i++)
{
    analogInputConfiguration.analogInputName[i] = Name[i];
}
analogInputConfiguration.analogInputType = Voltage;
result = cr_cfg_var_ai_set(robotHandle, type, index, analogInputConfiguration);
//设置索引号为 1 的模拟量输入信号类型
if (result != success) //判断设置模拟量输入信号类型是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.12.8 读取模拟量输入信号类型

## 函数

```
CRresult cr_cfg_var_ai_get(RobotHandle robotHandle, VarDigitalIOType type, int
index, AnalogInputConfiguration *analogInputConfiguration)
```

## 简介

读取模拟量输入信号类型。

## 参数

| 参数名称                     | 数据类型                       | 输入/输出 | 描述                 |
|--------------------------|----------------------------|-------|--------------------|
| robotHandle              | RobotHandle                | 输入    | 机械臂实例句柄            |
| type                     | VarDigitalIOType           | 输入    | 模拟量类型              |
| index                    | int                        | 输入    | 模拟量索引<br>(范围: 0-2) |
| analogInputConfiguration | AnalogInputConfiguration * | 输出    | 模拟量输入配置            |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
AnalogInputConfiguration analogInputConfiguration;
VarDigitalIOType type = Analog;
int index = 1;
result = cr_cfg_var_ai_get(robotHandle, type, index, &analogInputConfiguration);
//读取索引号为 1 的模拟量输入信号类型
if (result != success) //判断读取模拟量输入信号类型是否成功
    return -1;
printf("%s\n", analogInputConfiguration.analogInputName);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.12.9 设置模拟量输出信号类型

## 函数

```
CRresult cr_cfg_var_ao_set(RobotHandle robotHandle, int index, AnalogOutputConfiguration
analogOutputConfiguration)
```

## 简介

设置模拟量输出信号类型。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述               |
|-------------|-------------|-------|------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄          |
| index       | int         | 输入    | 模拟量索引<br>(范围: 0) |



|                           |                           |    |         |
|---------------------------|---------------------------|----|---------|
| analogOutputConfiguration | AnalogOutputConfiguration | 输入 | 模拟量输出配置 |
|---------------------------|---------------------------|----|---------|

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 0;
AnalogOutputConfiguration analogOutputConfiguration;
analogOutputConfiguration.analogOutputType = Current;
char Name[NAMELENTH] = { "AN0" };
for (int i = 0; i < NAMELENTH; i++)
{
    analogOutputConfiguration.analogOutputName[i] = Name[i];
}
result = cr_cfg_var_ao_set(robotHandle, index, analogOutputConfiguration);
//设置索引号为 0 的模拟量输出信号类型
if (result != success) //判断设置模拟量输出信号类型是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.12.10 读取模拟量输出信号类型

### 函数

CRresult cr\_cfg\_var\_ao\_get(RobotHandle robotHandle, int index, AnalogOutputConfiguration \*analogOutputConfiguration)

### 简介

读取模拟量输出信号类型。

### 参数

| 参数名称                      | 数据类型                        | 输入/输出 | 描述               |
|---------------------------|-----------------------------|-------|------------------|
| robotHandle               | RobotHandle                 | 输入    | 机械臂实例句柄          |
| index                     | int                         | 输入    | 模拟量索引<br>(范围: 0) |
| analogOutputConfiguration | AnalogOutputConfiguration * | 输出    | 模拟量输出配置          |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 0;
AnalogOutputConfiguration analogOutputConfiguration;
result = cr_cfg_var_ao_get(robotHandle, index, &analogOutputConfiguration);
//读取索引号为 0 的模拟量输出信号类型
if (result != success) //判断读取模拟量输出信号类型是否成功
    return -1;
printf("%s\n", analogOutputConfiguration.analogOutputName);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.12.11 设置工具电压输出值

## 函数

**CResult** cr\_cfg\_var\_to\_set(**RobotHandle** robotHandle, **ToolPower** toolPower)

## 简介

设置工具电压输出值。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| toolPower   | ToolPower   | 输入    | 电压值     |

## 返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
ToolPower toolPower = Power_off;
result = cr_cfg_var_to_set(robotHandle, toolPower); //设置工具电压输出值为 0V
if (result != success) //判断设置工具电压输出值是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
```

```
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.12.12 读取工具电压输出值

#### 函数

**CRresult** cr\_cfg\_var\_to\_get(**RobotHandle** robotHandle, **ToolPower** \*toolPower)

#### 简介

读取工具电压输出值。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| toolPower   | ToolPower * | 输出    | 电压值     |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
ToolPower toolPower;
result = cr_cfg_var_to_get(robotHandle, &toolPower); //读取工具电压输出值
if (result != success) //判断读取工具电压输出值是否成功
    return -1;
printf("%d\n", toolPower);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.12.13 设置编码器配置

#### 函数

**CRresult** cr\_cfg\_encoder\_config\_set(**RobotHandle** robotHandle, int index, **EncoderConfiguration** encoderConfig)

#### 简介

设置编码器配置。

## 参数

| 参数名称          | 数据类型                 | 输入/输出 | 描述      |
|---------------|----------------------|-------|---------|
| robotHandle   | RobotHandle          | 输入    | 机械臂实例句柄 |
| index         | int                  | 输入    | 编码器索引   |
| encoderConfig | EncoderConfiguration | 输入    | 编码器状态   |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 0;
EncoderConfiguration encoderConfig;
encoderConfig.eqepType = 0;
encoderConfig.eqepCountMode = 0;
encoderConfig.strobeInputMode = 0;
encoderConfig.eqepEnabled = 1;
result = cr_cfg_encoder_config_set(robotHandle, index, encoderConfig); //设置编码器配置
if (result != success) //判断设置编码器配置是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.12.14 读取编码器配置

## 函数

```
CRresult cr_cfg_encoder_config_get(RobotHandle robotHandle, int index, EncoderConfiguration
*encoderConfig)
```

## 简介

读取编码器配置。

## 参数

| 参数名称          | 数据类型                   | 输入/输出 | 描述      |
|---------------|------------------------|-------|---------|
| robotHandle   | RobotHandle            | 输入    | 机械臂实例句柄 |
| index         | int                    | 输入    | 编码器索引   |
| encoderConfig | EncoderConfiguration * | 输出    | 编码器状态   |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 0;
EncoderConfiguration encoderConfig;
result = cr_cfg_encoder_config_get(robotHandle, index, &encoderConfig); //读取编码器配置
if (result != success) //判断读取编码器配置是否成功
    return -1;
printf("%d\n", encoderConfig.eqepType);
printf("%d\n", encoderConfig.eqepCountMode);
printf("%d\n", encoderConfig.strobeInputMode);
printf("%d\n", encoderConfig.eqepEnabled);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.13 通讯变量

### 3.6.13.1 设置位寄存器配置

#### 函数

**CRresult** cr\_cfg\_var\_bit\_reg\_set(**RobotHandle** robotHandle, int index, **BitRegisterConfiguration** bitRegisterConfiguration)

#### 简介

设置位寄存器配置。

#### 参数

| 参数名称                     | 数据类型                     | 输入/输出 | 描述                   |
|--------------------------|--------------------------|-------|----------------------|
| robotHandle              | RobotHandle              | 输入    | 机械臂实例句柄              |
| index                    | int                      | 输入    | 寄存器索引值<br>(范围：0-999) |
| bitRegisterConfiguration | BitRegisterConfiguration | 输入    | 位寄存器配置               |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
BitRegisterConfiguration bitRegisterConfiguration;
bitRegisterConfiguration.GeneralPurposeBooleanRegisterInputActions = InputAction_NONE;
bitRegisterConfiguration.GeneralPurposeBooleanRegisterOutputActions =
OutputAction_RobotON_HIGH;
bitRegisterConfiguration.GeneralPurposeBooleanRegisterOutputRule = OutputRule_Disable;
char Name[NAMELENGTH] = { "WEI" };
strcpy_s(bitRegisterConfiguration.GeneralPurposeBooleanRegisterName, sizeof(Name),
Name);
result = cr_cfg_var_bit_reg_set(robotHandle, index, bitRegisterConfiguration);
//设置索引值为 1 的位寄存器配置
if (result != success) //判断设置位寄存器配置是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.13.2 读取位寄存器配置

### 函数

**CRresult cr\_cfg\_var\_bit\_reg\_get(RobotHandle robotHandle, int index, BitRegisterConfiguration \*bitRegisterConfiguration)**

### 简介

读取位寄存器配置。

### 参数

| 参数名称                     | 数据类型                       | 输入/输出 | 描述                    |
|--------------------------|----------------------------|-------|-----------------------|
| robotHandle              | RobotHandle                | 输入    | 机械臂实例句柄               |
| index                    | int                        | 输入    | 寄存器索引值<br>(范围: 0-999) |
| bitRegisterConfiguration | BitRegisterConfiguration * | 输出    | 位寄存器配置                |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
```

```
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
BitRegisterConfiguration bitRegisterConfiguration;
int index = 1;
result = cr_cfg_var_bit_reg_get(robotHandle, index, &bitRegisterConfiguration);
//读取索引值为 1 的位寄存器配置
if (result != success) //判断读取位寄存器配置是否成功
    return -1;
printf("%s\n", bitRegisterConfiguration.GeneralPurposeBOOLeanRegisterName);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.13.3 读取所有位寄存器配置

#### 函数

```
CRresult cr_cfg_var_all_bit_reg_get(RobotHandle robotHandle, BitRegisterConfiguration
*bitRegisterConfigurationArray, int arraySize)
```

#### 简介

读取所有位寄存器配置。

#### 参数

| 参数名称                          | 数据类型                       | 输入/输出 | 描述         |
|-------------------------------|----------------------------|-------|------------|
| robotHandle                   | RobotHandle                | 输入    | 机械臂实例句柄    |
| bitRegisterConfigurationArray | BitRegisterConfiguration * | 输出    | 存放寄存器配置的地址 |
| arraySize                     | int                        | 输入    | 读取的数量      |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
BitRegisterConfiguration bitRegisterConfigurationArray[1000];
int arraySize = 1000;
result = cr_cfg_var_all_bit_reg_get(robotHandle, bitRegisterConfigurationArray, arraySize);
//读取所有位寄存器配置
if (result != success) //判断读取所有位寄存器配置是否成功
```

```

    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;

```

### 3.6.13.4 设置 16 位整数寄存器变量名

#### 函数

**CRresult** cr\_cfg\_var\_int16\_reg\_name\_set(**RobotHandle** robotHandle, int index, char \*name, int len)

#### 简介

设置 16 位整数寄存器变量名。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述               |
|-------------|-------------|-------|------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄          |
| index       | int         | 输入    | 寄存器索引值（范围：0-999） |
| name        | char *      | 输入    | 寄存器名字            |
| len         | int         | 输入    | 名字长度             |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char Name[NAMELENGTH] = { "WEI16" };
int index = 1;
int len = 6;
result = cr_cfg_var_int16_reg_name_set(robotHandle, index, Name, len);
//设置索引值为 1 的 16 位整数寄存器变量名
if (result != success) //判断设置 16 位整数寄存器变量名是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;

```



### 3.6.13.5 读取 16 位整数寄存器变量名

#### 函数

```
CRresult cr_cfg_var_int16_reg_name_get(RobotHandle robotHandle,int index, char *name,int len)
```

#### 简介

读取 16 位整数寄存器变量名。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述               |
|-------------|-------------|-------|------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄          |
| index       | int         | 输入    | 寄存器索引值（范围：0-999） |
| name        | char *      | 输出    | 寄存器名字            |
| len         | int         | 输入    | 名字长度             |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char name[NAMELENTN];
int index = 1;
int len = 6;
result = cr_cfg_var_int16_reg_name_get(robotHandle, index, name, 6);
//读取索引值为 1 的 16 位整数寄存器变量名
if (result != success) //判断读取 16 位整数寄存器变量名是否成功
    return -1;
printf("%s\n", name);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.13.6 读取所有 16 位整数寄存器变量名

#### 函数

```
CRresult cr_cfg_var_all_int16_reg_name_get(RobotHandle robotHandle, RegisterConfiguration *registerConfigurationArray,int arrSize)
```

## 简介

读取所有 16 位整数寄存器变量名。

## 参数

| 参数名称                       | 数据类型                    | 输入/输出 | 描述         |
|----------------------------|-------------------------|-------|------------|
| robotHandle                | RobotHandle             | 输入    | 机械臂实例句柄    |
| registerConfigurationArray | RegisterConfiguration * | 输出    | 存放寄存器名字的地址 |
| arrSize                    | int                     | 输入    | 读取的名字数量    |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
RegisterConfiguration registerConfigurationArray[1000];
int arrSize = 1000;
result = cr_cfg_var_all_int16_reg_name_get(robotHandle, registerConfigurationArray, arrSize);
//读取所有 16 位整数寄存器变量名
if (result != success) //判断读取所有 16 位整数寄存器变量名是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.13.7 设置 32 位整数寄存器变量名

## 函数

```
CRresult cr_cfg_var_int32_reg_name_set(RobotHandle robotHandle, int index, char *name, int len)
```

## 简介

设置 32 位整数寄存器变量名。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述               |
|-------------|-------------|-------|------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄          |
| index       | int         | 输入    | 寄存器索引值（范围：0-999） |
| name        | char *      | 输入    | 寄存器名字            |
| len         | int         | 输入    | 名字长度             |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char Name[NAMELENGTH] = { "WEI32" };
int index = 1;
int len = 6;
result = cr_cfg_var_int32_reg_name_set(robotHandle, index, Name, len);
//设置索引值为 1 的 32 位整数寄存器变量名
if (result != success) //判断设置 32 位整数寄存器变量名是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.13.8 读取 32 位整数寄存器变量名

## 函数

**CRresult** cr\_cfg\_var\_int32\_reg\_name\_get(**RobotHandle** robotHandle, int index, char \*name, int len)

## 简介

读取 32 位整数寄存器变量名。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述               |
|-------------|-------------|-------|------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄          |
| index       | int         | 输入    | 寄存器索引值（范围：0-999） |
| name        | char *      | 输出    | 寄存器名字            |
| len         | int         | 输入    | 名字长度             |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
```

```
char name[NAMELENTH];
int index = 1;
int len = 6;
result = cr_cfg_var_int32_reg_name_get(robotHandle, index, name, len);
//读取索引值为 1 的 32 位整数寄存器变量名
if (result != success) //判断读取 32 位整数寄存器变量名是否成功
    return -1;
printf("%s\n", name);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.13.9 读取所有 32 位整数寄存器变量名

#### 函数

**CRresult** cr\_cfg\_var\_all\_int32\_reg\_name\_get(**RobotHandle** robotHandle, **RegisterConfiguration** \*registerConfigurationArray, int arrSize)

#### 简介

读取所有 32 位整数寄存器变量名。

#### 参数

| 参数名称                       | 数据类型                    | 输入/输出 | 描述         |
|----------------------------|-------------------------|-------|------------|
| robotHandle                | RobotHandle             | 输入    | 机械臂实例句柄    |
| registerConfigurationArray | RegisterConfiguration * | 输出    | 存放寄存器名字的地址 |
| arrSize                    | int                     | 输入    | 读取的名字数量    |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
RegisterConfiguration registerConfigurationArray[1000];
int arrSize = 1000;
result = cr_cfg_var_all_int32_reg_name_get(robotHandle, registerConfigurationArray, arrSize);
//读取所有 32 位整数寄存器变量名
if (result != success) //判断读取所有 32 位整数寄存器变量名是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
```

```
return -1;
```

### 3.6.13.10 设置浮点寄存器变量名

#### 函数

**CRresult** cr\_cfg\_var\_float\_reg\_name\_set(**RobotHandle** robotHandle,int index, char \*name,int len)

#### 简介

设置浮点寄存器变量名。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述               |
|-------------|-------------|-------|------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄          |
| index       | int         | 输入    | 寄存器索引值（范围：0-999） |
| name        | char *      | 输入    | 寄存器名字            |
| len         | int         | 输入    | 名字长度             |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char Name[NAMELENGTH] = { "FLOAT" };
int index = 1;
int len = 6;
result = cr_cfg_var_float_reg_name_set(robotHandle, index, Name, len);
//设置索引值为 1 的浮点寄存器变量名
if (result != success) //判断设置浮点寄存器变量名是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.13.11 读取浮点寄存器变量名

#### 函数

**CRresult** cr\_cfg\_var\_float\_reg\_name\_get(**RobotHandle** robotHandle,int index, char \*name,int len)

## 简介

读取浮点寄存器变量名。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述               |
|-------------|-------------|-------|------------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄          |
| index       | int         | 输入    | 寄存器索引值（范围：0-999） |
| name        | char *      | 输出    | 寄存器名字            |
| len         | int         | 输入    | 名字长度             |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char name[NAMELENGTH];
int index = 1;
int len = 6;
result = cr_cfg_var_float_reg_name_get(robotHandle, 1, name, 6);
//读取索引值为 1 的浮点寄存器变量名
if (result != success) //判断读取浮点寄存器变量名是否成功
    return -1;
printf("%s\n", name);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.13.12 读取所有浮点寄存器变量名

## 函数

```
CRresult cr_cfg_var_all_float_reg_name_get(RobotHandle robotHandle, RegisterConfiguration
*registerConfigurationArray, int arrSize)
```

## 简介

读取所有浮点寄存器变量名。

## 参数

| 参数名称                       | 数据类型                    | 输入/输出 | 描述         |
|----------------------------|-------------------------|-------|------------|
| robotHandle                | RobotHandle             | 输入    | 机械臂实例句柄    |
| registerConfigurationArray | RegisterConfiguration * | 输出    | 存放寄存器名字的地址 |

|         |     |    |         |
|---------|-----|----|---------|
| arrSize | int | 输入 | 读取的名字数量 |
|---------|-----|----|---------|

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
RegisterConfiguration registerConfigurationArray[1000];
int arrSize = 1000;
result = cr_cfg_var_all_float_reg_name_get(robotHandle, registerConfigurationArray, arrSize);
//读取所有浮点寄存器变量名
if (result != success) //判断读取所有浮点寄存器变量名是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.14 安装变量

### 3.6.14.1 读取安装变量个数

#### 函数

```
CRresult cr_cfg_var_install_count(RobotHandle robotHandle, int *count)
```

#### 简介

读取安装变量个数。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| count       | int *       | 输出    | 安装变量数量  |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
```

```
int count;
result = cr_cfg_var_install_count(robotHandle, &count); //读取安装变量个数
if (result != success) //判断读取安装变量个数是否成功
    return -1;
printf("%d\n", count);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.14.2 读取安装变量数据

#### 函数

**CRresult** cr\_cfg\_var\_install\_get(**RobotHandle** robotHandle, int index, **VariableMsg** \*variableMsg)

#### 简介

读取安装变量数据。

#### 参数

| 参数名称        | 数据类型          | 输入/输出 | 描述              |
|-------------|---------------|-------|-----------------|
| robotHandle | RobotHandle   | 输入    | 机械臂实例句柄         |
| index       | int           | 输入    | 安装变量索引（范围：0-99） |
| variableMsg | VariableMsg * | 输出    | 安装变量数据          |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
VariableMsg variableMsg;
result = cr_cfg_var_install_get(robotHandle, 1, &variableMsg);
//读取索引号为 1 的安装变量数据
if (result != success) //判断读取安装变量数据是否成功
    return -1;
printf("%d\n", variableMsg.boolValue);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```



### 3.6.14.3 增加安装变量

#### 函数

**CRresult** cr\_cfg\_var\_install\_add(**RobotHandle** robotHandle, **VariableMsg** \*variableMsg)

#### 简介

增加安装变量。

#### 参数

| 参数名称        | 数据类型          | 输入/输出 | 描述      |
|-------------|---------------|-------|---------|
| robotHandle | RobotHandle   | 输入    | 机械臂实例句柄 |
| variableMsg | VariableMsg * | 输入    | 安装变量数据  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
char name[CR6_ISTALL_VAR_NAME_LENGTH] = { "install" };
VariableMsg variablemsg;
strcpy_s(variablemsg.variableName, sizeof(name), name);
variablemsg.variableType= LUA_TBOOLEAN;
variablemsg.boolValue = TRUE;
variablemsg.variableID = 3;
result = cr_cfg_var_install_add(robotHandle, &variablemsg); //增加安装变量
if (result != success) //判断增加安装变量是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.14.4 删除安装变量

#### 函数

**CRresult** cr\_cfg\_var\_install\_delete(**RobotHandle** robotHandle, int index)

#### 简介

删除安装变量。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述              |
|-------------|-------------|-------|-----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄         |
| index       | int         | 输入    | 安装变量索引（范围：0-99） |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;  
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接  
if (result != success) //判断机器人连接是否成功  
    return -1;  
int index = 1;  
result = cr_cfg_var_install_delete(robotHandle, index); //删除索引号为 1 的安装变量  
if (result != success) //判断删除安装变量是否成功  
    return -1;  
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄  
if (result != success) //判断机器人断开连接是否成功  
    return -1;
```

### 3.6.14.5 修改安装变量

## 函数

CRresult cr\_cfg\_var\_install\_set(RobotHandle robotHandle, int index, VariableMsg \*variableMsg)

## 简介

修改安装变量。

## 参数

| 参数名称        | 数据类型          | 输入/输出 | 描述              |
|-------------|---------------|-------|-----------------|
| robotHandle | RobotHandle   | 输入    | 机械臂实例句柄         |
| index       | int           | 输入    | 安装变量索引（范围：0-99） |
| variableMsg | VariableMsg * | 输入    | 安装变量数据          |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;  
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接  
if (result != success) //判断机器人连接是否成功
```

```

        return -1;
    int index = 2;
    char name[CR6_INSTALL_VAR_NAME_LENGTH] = { "install" };
    VariableMsg variablemsg;
    strcpy_s(variablemsg.variableName, sizeof(name), name);
    variablemsg.variableType = LUA_TBOOLEAN;
    variablemsg.boolValue = TRUE;
    variablemsg.variableID = 1;
    result = cr_cfg_var_install_set(robotHandle, index, &variablemsg);
    //修改索引号为 2 的安装变量
    if (result != success) //判断修改安装变量是否成功
        return -1;
    result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
    if (result != success) //判断机器人断开连接是否成功
        return -1;

```

## 3.6.15 坐标系

### 3.6.15.1 读取点坐标系个数

#### 函数

**CRresult** cr\_cfg\_cs\_point\_count(**RobotHandle** robotHandle,int \*count)

#### 简介

读取点坐标系个数。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| count       | int *       | 输出    | 点坐标系数量  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int count;
result = cr_cfg_cs_point_count(robotHandle, &count); //读取点坐标系个数
if (result != success) //判断读取点坐标系个数是否成功
    return -1;

```

```
printf("%d\n", count);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.15.2 读取点坐标系数据

#### 函数

**CRresult** cr\_cfg\_cs\_point\_get(**RobotHandle** robotHandle,int index, **PointCSNode** \*pointCSNode)

#### 简介

读取点坐标系数据。

#### 参数

| 参数名称        | 数据类型          | 输入/输出 | 描述             |
|-------------|---------------|-------|----------------|
| robotHandle | RobotHandle   | 输入    | 机械臂实例句柄        |
| index       | int           | 输入    | 点坐标系索引（范围：0-9） |
| pointCSNode | PointCSNode * | 输出    | 点坐标系数据         |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
PointCSNode pointCSNode;
int index = 1;
result = cr_cfg_cs_point_get(robotHandle, index, &pointCSNode);
//读取索引号为 1 的点坐标系数据
if (result != success) //判断读取点坐标系数据是否成功
    return -1;
printf("%s\n", pointCSNode.name);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.15.3 读取点坐标系数据(通过名称)

#### 函数

**CRresult** cr\_cfg\_cs\_point\_accord\_name\_get(**RobotHandle** robotHandle, char\* csName, **PointCSNode** \*pointCSNode)

## 简介

读取点坐标系数据(通过名称)。

## 参数

| 参数名称        | 数据类型          | 输入/输出 | 描述      |
|-------------|---------------|-------|---------|
| robotHandle | RobotHandle   | 输入    | 机械臂实例句柄 |
| csName      | char*         | 输入    | 点坐标系名称  |
| pointCSNode | PointCSNode * | 输出    | 点坐标系数据  |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
PointCSNode pointCSNode;
char csName[10] = "point_1";
result = cr_cfg_cs_point_accord_name_get(robotHandle, csName, &pointCSNode);
//读取名称为 point_1 的点坐标系数据
if (result != success) //判断读取点坐标系数据是否成功
    return -1;
printf("%d\n", pointCSNode.isValid);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.15.4 增加点坐标系

## 函数

**CRresult** cr\_cfg\_cs\_point\_add(**RobotHandle** robotHandle, **PointCSNode** pointCSNode)

## 简介

增加点坐标系。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| pointCSNode | PointCSNode | 输入    | 点坐标系数据  |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
PointCSNode pointCSNode;
char name[32] = { "point3" };
strcpy_s(pointCSNode.name, sizeof(name), name);
double toolAxisAngle[6] = { 30,30,30,30,30,30 };
double toolPosition[6];
for (int i = 0; i < 6; i++)
{
    pointCSNode.point.toolAxisAngle[i] = toolAxisAngle[i];
}
result = cr_kineForward(robotHandle, toolAxisAngle, toolPosition); //正解求点的位姿
if (result != success) //判断正解是否成功
    return -1;
for (int i = 0; i < 6; i++)
{
    pointCSNode.point.toolPosition[i] = toolPosition[i];
}
pointCSNode.isValid = TRUE;
pointCSNode.id = 1;
result = cr_cfg_cs_point_add(robotHandle, pointCSNode); //增加点坐标系
if (result != success) //判断增加点坐标系是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.15.5 删除点坐标系

### 函数

**CRresult** cr\_cfg\_cs\_point\_delete(**RobotHandle** robotHandle, int index)

### 简介

删除点坐标系。

### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述             |
|-------------|-------------|-------|----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄        |
| index       | int         | 输入    | 点坐标系索引（范围：0-9） |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
result = cr_cfg_cs_point_delete(robotHandle, index); //删除索引号为 1 的点坐标系
if (result != success) //判断删除点坐标系是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.15.6 修改点坐标系数据

## 函数

**CRresult** cr\_cfg\_cs\_point\_set(**RobotHandle** robotHandle, int index, **PointCSNode** pointCSNode)

## 简介

修改点坐标系数据。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述             |
|-------------|-------------|-------|----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄        |
| index       | int         | 输入    | 点坐标系索引（范围：0-9） |
| pointCSNode | PointCSNode | 输入    | 点坐标系数据         |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
PointCSNode pointCSNode;
char name[32] = { "point3" };
strcpy_s(pointCSNode.name, sizeof(name), name);
double toolAxisAngle[6] = { 30,30,30,30,30,30 };
```

```
double toolPosition[6];
for (int i = 0; i < 6; i++)
{
    pointCSNode.point.toolAxisAngle[i] = toolAxisAngle[i];
}
result = cr_kineForward(robotHandle, toolAxisAngle, toolPosition); //正解求点的位姿
if (result != success) //判断正解是否成功
    return -1;
for (int i = 0; i < 6; i++)
{
    pointCSNode.point.toolPosition[i] = toolPosition[i];
}
pointCSNode.isValid = TRUE;
pointCSNode.id = 1;
result = cr_cfg_cs_point_set(robotHandle, index, pointCSNode);
//修改索引号为 1 的点坐标系数据
if (result != success) //判断修改点坐标系数据是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.15.7 读取线坐标系个数

#### 函数

**CRresult** cr\_cfg\_cs\_line\_count(**RobotHandle** robotHandle,int \*count)

#### 简介

读取线坐标系个数。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| count       | int *       | 输出    | 线坐标系个数  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int count;
```



```
result = cr_cfg_cs_line_count(robotHandle, &count); //读取线坐标系个数
if (result != success) //判断读取线坐标系个数是否成功
    return -1;
printf("%d\n", count);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.15.8 读取线坐标系数据

#### 函数

**CRresult** cr\_cfg\_cs\_line\_get(**RobotHandle** robotHandle, int index, **LineCSNode** \*lineCSNode)

#### 简介

读取线坐标系数据。

#### 参数

| 参数名称        | 数据类型         | 输入/输出 | 描述             |
|-------------|--------------|-------|----------------|
| robotHandle | RobotHandle  | 输入    | 机械臂实例句柄        |
| index       | int          | 输入    | 线坐标系索引（范围：0-9） |
| lineCSNode  | LineCSNode * | 输出    | 线坐标系数据         |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
LineCSNode lineCSNode;
int index = 1;
result = cr_cfg_cs_line_get(robotHandle, index, &lineCSNode);
//读取索引号为 1 的线坐标系数据
if (result != success) //判断读取线坐标系数据是否成功
    return -1;
printf("%s\n", lineCSNode.name);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.15.9 读取线坐标系数据(通过名称)

#### 函数

```
CRresult cr_cfg_cs_line_accord_name_get(RobotHandle robotHandle, char* csName,  
LineCSNode *lineCSNode)
```

#### 简介

读取线坐标系数据(通过名称)。

#### 参数

| 参数名称        | 数据类型         | 输入/输出 | 描述      |
|-------------|--------------|-------|---------|
| robotHandle | RobotHandle  | 输入    | 机械臂实例句柄 |
| csName      | char*        | 输入    | 线坐标系名称  |
| lineCSNode  | LineCSNode * | 输出    | 线坐标系数据  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;  
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接  
if (result != success) //判断机器人连接是否成功  
    return -1;  
LineCSNode lineCSNode;  
char csName[10] = "line_1";  
result = cr_cfg_cs_line_accord_name_get(robotHandle, csName, &lineCSNode);  
//读取名称为 line_1 的线坐标系数据  
if (result != success) //判断读取线坐标系数据是否成功  
    return -1;  
printf("%d\n", lineCSNode.isValid);  
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄  
if (result != success) //判断机器人断开连接是否成功  
    return -1;
```

### 3.6.15.10 增加线坐标系

#### 函数

```
CRresult cr_cfg_cs_line_add(RobotHandle robotHandle, LineCSNode lineCSNode)
```

#### 简介

增加线坐标系。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| lineCSNode  | LineCSNode  | 输入    | 线坐标系数据  |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
LineCSNode lineCSNode;
double toolAxisAngle1[6] = { 0,0,90,0,-90,0 }; //第一个点关节角度
double toolAxisAngle2[6] = { 9.29,3.39,86.49,0.08,-89.91,9.29 }; //第二个点关节角度
double toolPosition1[6];
double toolPosition2[6];
double coordinatePose[6];
double coordinateJointPos[6] = { 0,0,0,0,0,0 };
result = cr_kineForward(robotHandle, toolAxisAngle1, toolPosition1); //正解求第一个点位姿
if (result != success) //判断正解是否成功
    return -1;
result = cr_kineForward(robotHandle, toolAxisAngle2, toolPosition2); //正解求第二个点位姿
if (result != success) //判断正解是否成功
    return -1;
result = cr_compute_cs_line(toolPosition1, 6, toolPosition2, 6, coordinatePose, 6);
//计算线坐标系位姿
if (result != success) //判断计算线坐标系位姿是否成功
    return -1;
CRresult res = cr_kineInverse(robotHandle, coordinatePose, coordinateJointPos,
coordinateJointPos); //逆解求线坐标系关节角度
if (res == 0) //判断逆解是否存在，即坐标系是否有效
{
    lineCSNode.isValid = 1;
}
else
{
    lineCSNode.isValid = 0;
}
char name[32] = { "LINE" };
char point_name1[32] = { "p1" };
char point_name2[32] = { "p2" };
```

```
strcpy_s(lineCSNode.firstPoint.name, sizeof(point_name1), point_name1);
strcpy_s(lineCSNode.secondPoint.name, sizeof(point_name2), point_name2);
strcpy_s(lineCSNode.name, sizeof(name), name);
for (int i = 0; i < 6; i++)
{
    lineCSNode.firstPoint.point.toolAxisAngle[i] = toolAxisAngle1[i];
    lineCSNode.secondPoint.point.toolAxisAngle[i] = toolAxisAngle2[i];
    lineCSNode.firstPoint.point.toolPosition[i] = toolPosition1[i];
    lineCSNode.secondPoint.point.toolPosition[i] = toolPosition2[i];
    lineCSNode.coordinateJointPos[i] = coordinateJointPos[i];
    lineCSNode.coordinatePose[i] = coordinatePose[i];
}
lineCSNode.firstPoint.id = 1;
lineCSNode.firstPoint.isValid = 1;
lineCSNode.secondPoint.id = 2;
lineCSNode.secondPoint.isValid = 1;
lineCSNode.id = 2;
result = cr_cfg_cs_line_add(robotHandle, lineCSNode); //增加线坐标系
if (result != success) //判断增加线坐标系是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.15.11 删除线坐标系

#### 函数

**CRresult** cr\_cfg\_cs\_line\_delete(**RobotHandle** robotHandle,int index)

#### 简介

删除线坐标系。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述             |
|-------------|-------------|-------|----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄        |
| index       | int         | 输入    | 线坐标系索引（范围：0-9） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
```

```

        return -1;
    int index = 1;
    result = cr_cfg_cs_line_delete(robotHandle, index); //删除索引号为 1 的线坐标系
    if (result != success) //判断删除线坐标系是否成功
        return -1;
    result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
    if (result != success) //判断机器人断开连接是否成功
        return -1;

```

### 3.6.15.12 修改线坐标系数据

#### 函数

**CRresult** cr\_cfg\_cs\_line\_set(**RobotHandle** robotHandle,int index, **LineCSNode** lineCSNode)

#### 简介

修改线坐标系数据。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述             |
|-------------|-------------|-------|----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄        |
| index       | int         | 输入    | 线坐标系索引（范围：0-9） |
| lineCSNode  | LineCSNode  | 输入    | 线坐标系数据         |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
LineCSNode lineCSNode;
double toolAxisAngle1[6] = { 0,0,90,0,-90,0 }; //第一个点关节角度
double toolAxisAngle2[6] = { 9.29,3.39,86.49,0.08,-89.91,9.29 }; //第二个点关节角度
double toolPosition1[6];
double toolPosition2[6];
double coordinatePose[6];
double coordinateJointPos[6] = { 0,0,0,0,0,0 };
result = cr_kineForward(robotHandle, toolAxisAngle1, toolPosition1); //正解求第一个点位姿
if (result != success) //判断正解是否成功
    return -1;
result = cr_kineForward(robotHandle, toolAxisAngle2, toolPosition2); //正解求第二个点位姿
if (result != success) //判断正解是否成功

```

```
    return -1;
result = cr_compute_cs_line(toolPosition1, 6, toolPosition2, 6, coordinatePose, 6);
//计算线坐标系位姿
if (result != success) //判断计算线坐标系位姿是否成功
    return -1;
CResult res = cr_kineInverse(robotHandle, coordinatePose, coordinateJointPos,
coordinateJointPos); //逆解求线坐标系关节角度
if (res == 0) //判断逆解是否存在，即坐标系是否有效
{
    lineCSNode.isValid = 1;
}
else
{
    lineCSNode.isValid = 0;
}
char name[32] = { "LINE" };
char point_name1[32] = { "p1" };
char point_name2[32] = { "p2" };
strcpy_s(lineCSNode.firstPoint.name, sizeof(point_name1), point_name1);
strcpy_s(lineCSNode.secondPoint.name, sizeof(point_name2), point_name2);
strcpy_s(lineCSNode.name, sizeof(name), name);
for (int i = 0; i < 6; i++)
{
    lineCSNode.firstPoint.point.toolAxisAngle[i] = toolAxisAngle1[i];
    lineCSNode.secondPoint.point.toolAxisAngle[i] = toolAxisAngle2[i];
    lineCSNode.firstPoint.point.toolPosition[i] = toolPosition1[i];
    lineCSNode.secondPoint.point.toolPosition[i] = toolPosition2[i];
    lineCSNode.coordinateJointPos[i] = coordinateJointPos[i];
    lineCSNode.coordinatePose[i] = coordinatePose[i];
}
lineCSNode.firstPoint.id = 1;
lineCSNode.firstPoint.isValid = 1;
lineCSNode.secondPoint.id = 2;
lineCSNode.secondPoint.isValid = 1;
lineCSNode.id = 2;
result = cr_cfg_cs_line_set(robotHandle, 1, lineCSNode); //修改索引号为 1 的线坐标系数据
if (result != success) //判断修改线坐标系数据是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.15.13 读取面坐标系个数

#### 函数

`CRresult cr_cfg_cs_plane_count(RobotHandle robotHandle,int *count)`

#### 简介

读取面坐标系个数。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| count       | int *       | 输出    | 面坐标系数量  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int count;
result = cr_cfg_cs_plane_count(robotHandle, &count); //读取面坐标系个数
if (result != success) //判断读取面坐标系个数是否成功
    return -1;
printf("%d\n", count);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.15.14 读取面坐标系数据

#### 函数

`CRresult cr_cfg_cs_plane_get(RobotHandle robotHandle,int index, PlaneCSNode *planeCSNode)`

#### 简介

读取面坐标系数据。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述             |
|-------------|-------------|-------|----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄        |
| index       | int         | 输入    | 面坐标系索引（范围：0-9） |

|             |               |    |        |
|-------------|---------------|----|--------|
| planeCSNode | PlaneCSNode * | 输出 | 面坐标系数据 |
|-------------|---------------|----|--------|

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
PlaneCSNode planeCSNode;
result = cr_cfg_cs_plane_get(robotHandle, index, &planeCSNode);
//读取索引号为 1 的面坐标系数据
if (result != success) //判断读取面坐标系数据是否成功
    return -1;
printf("%s\n", planeCSNode.name);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.15.15 读取面坐标系数据(通过名称)

### 函数

```
CRresult cr_cfg_cs_plane_accord_name_get(RobotHandle robotHandle, char* csName,
PlaneCSNode *planeCSNode)
```

### 简介

读取面坐标系数据(通过名称)。

### 参数

| 参数名称        | 数据类型          | 输入/输出 | 描述      |
|-------------|---------------|-------|---------|
| robotHandle | RobotHandle   | 输入    | 机械臂实例句柄 |
| csName      | char*         | 输入    | 面坐标系名称  |
| planeCSNode | PlaneCSNode * | 输出    | 面坐标系数据  |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess) //判断机器人连接是否成功
```



```
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
string planeName = "plane_1";
PlaneCSNode planeCSNode = new PlaneCSNode();
result = CS.cr_cfg_cs_plane_accord_name_get(robotHandle, planeName, ref planeCSNode);
if (result != CRresult.sucess)//判断读取面坐标系数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string PlaneName = new string(planeCSNode.name);
Console.WriteLine(PlaneName);
Console.WriteLine(planeCSNode.isValid);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

### 3.6.15.16 增加面坐标系

#### 函数

`CRresult cr_cfg_cs_plane_add(RobotHandle robotHandle, PlaneCSNode planeCSNode)`

#### 简介

增加面坐标系。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| planeCSNode | PlaneCSNode | 输入    | 面坐标系数据  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
PlaneCSNode planeCSNode;
double toolAxisAngle1[6] = { 0, 0, 90, 0, -90, 0 }; //第一个点关节角度
```

```

double toolAxisAngle2[6] = { 0, 4.84, 84.91, 0.25, -89.99, 0 }; //第二个点关节角度
double toolAxisAngle3[6] = { -8.1, -1.5, 91.48, 0.04, -90.07, -8.1 }; //第三个点关节角度
double toolPosition1[6];
double toolPosition2[6];
double toolPosition3[6];
double coordinatePose[6];
double coordinateJointPos[6] = { 0,0,0,0,0,0 };
result = cr_kineForward(robotHandle, toolAxisAngle1, toolPosition1); //正解求第一个点位姿
if (result != success) //判断正解是否成功
    return -1;
result = cr_kineForward(robotHandle, toolAxisAngle2, toolPosition2); //正解求第二个点位姿
if (result != success) //判断正解是否成功
    return -1;
result = cr_kineForward(robotHandle, toolAxisAngle3, toolPosition3); //正解求第三个点位姿
if (result != success) //判断正解是否成功
    return -1;
result = cr_compute_cs_plane(toolPosition1, 6, toolPosition2, 6, toolPosition3, 6, coordinatePose,
6); //计算面坐标系位姿
if (result != success) //判断计算面坐标系位姿是否成功
    return -1;
CResult res = cr_kineInverse(robotHandle, coordinatePose, coordinateJointPos,
coordinateJointPos); //逆解求面坐标系关节角度
if (res == 0) //判断逆解是否存在，即坐标系是否有效
{
    planeCSNode.isValid = 1;
}
else
{
    planeCSNode.isValid = 0;
}
char name[32] = { "plane_1" };
char point_name1[32] = { "firstpoint" };
char point_name2[32] = { "secondpoint" };
char point_name3[32] = { "thirdpoint" };
strcpy_s(planeCSNode.firstPoint.name, sizeof(point_name1), point_name1);
strcpy_s(planeCSNode.secondPoint.name, sizeof(point_name2), point_name2);
strcpy_s(planeCSNode.thirdPoint.name, sizeof(point_name3), point_name3);
strcpy_s(planeCSNode.name, sizeof(name), name);
for (int i = 0; i < 6; i++)
{
    planeCSNode.firstPoint.point.toolAxisAngle[i] = toolAxisAngle1[i];
    planeCSNode.secondPoint.point.toolAxisAngle[i] = toolAxisAngle2[i];
    planeCSNode.thirdPoint.point.toolAxisAngle[i] = toolAxisAngle3[i];
    planeCSNode.firstPoint.point.toolPosition[i] = toolPosition1[i];
}

```

```

    planeCSNode.secondPoint.point.toolPosition[i] = toolPosition2[i];
    planeCSNode.thirdPoint.point.toolPosition[i] = toolPosition3[i];
    planeCSNode.coordinatePose[i] = coordinatePose[i];
    planeCSNode.coordinateJointPos[i] = coordinateJointPos[i];
}
planeCSNode.firstPoint.id = 1;
planeCSNode.firstPoint.isValid = 1;
planeCSNode.secondPoint.id = 2;
planeCSNode.secondPoint.isValid = 1;
planeCSNode.thirdPoint.id = 3;
planeCSNode.thirdPoint.isValid = 1;
planeCSNode.id = 2;
result = cr_cfg_cs_plane_add(robotHandle, planeCSNode); //增加面坐标系
if (result != success) //判断增加面坐标系是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;

```

### 3.6.15.17 删除面坐标系

#### 函数

**CRresult** cr\_cfg\_cs\_plane\_delete(**RobotHandle** robotHandle,int index)

#### 简介

删除面坐标系。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述             |
|-------------|-------------|-------|----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄        |
| index       | int         | 输入    | 面坐标系索引（范围：0-9） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int index = 1;
result = cr_cfg_cs_plane_delete(robotHandle, index); //删除索引号为 1 的面坐标系
if (result != success) //判断删除面坐标系是否成功
    return -1;

```

```
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.15.18 修改面坐标系数据

#### 函数

**CRresult** cr\_cfg\_cs\_plane\_set(**RobotHandle** robotHandle,int index, **PlaneCSNode** planeCSNode)

#### 简介

修改面坐标系数据。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述             |
|-------------|-------------|-------|----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄        |
| index       | int         | 输入    | 面坐标系索引（范围：0-9） |
| planeCSNode | PlaneCSNode | 输入    | 面坐标系数据         |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
PlaneCSNode planeCSNode;
double toolAxisAngle1[6] = { 0, 0, 90, 0, -90, 0 }; //第一个点关节角度
double toolAxisAngle2[6] = { 0, 4.84, 84.91, 0.25, -89.99, 0 }; //第二个点关节角度
double toolAxisAngle3[6] = { -8.1, -1.5, 91.48, 0.04, -90.07, -8.1 }; //第三个点关节角度
double toolPosition1[6];
double toolPosition2[6];
double toolPosition3[6];
double coordinatePose[6];
double coordinateJointPos[6] = { 0,0,0,0,0,0 };
result = cr_kineForward(robotHandle, toolAxisAngle1, toolPosition1); //正解求第一个点位姿
if (result != success) //判断正解是否成功
    return -1;
result = cr_kineForward(robotHandle, toolAxisAngle2, toolPosition2); //正解求第二个点位姿
if (result != success) //判断正解是否成功
    return -1;
result = cr_kineForward(robotHandle, toolAxisAngle3, toolPosition3); //正解求第三个点位姿
if (result != success) //判断正解是否成功
```

```
    return -1;
result = cr_compute_cs_plane(toolPosition1, 6, toolPosition2, 6, toolPosition3, 6, coordinatePose,
6); //计算面坐标系位姿
if (result != success) //判断计算面坐标系位姿是否成功
    return -1;
CResult res = cr_kineInverse(robotHandle, coordinatePose, coordinateJointPos,
coordinateJointPos); //逆解求面坐标系关节角度
if (res == 0) //判断逆解是否存在，即坐标系是否有效
{
    planeCSNode.isValid = 1;
}
else
{
    planeCSNode.isValid = 0;
}
char name[32] = { "plane_1" };
char point_name1[32] = { "firstpoint" };
char point_name2[32] = { "secondpoint" };
char point_name3[32] = { "thirdpoint" };
strcpy_s(planeCSNode.firstPoint.name, sizeof(point_name1), point_name1);
strcpy_s(planeCSNode.secondPoint.name, sizeof(point_name2), point_name2);
strcpy_s(planeCSNode.thirdPoint.name, sizeof(point_name3), point_name3);
strcpy_s(planeCSNode.name, sizeof(name), name);
for (int i = 0; i < 6; i++)
{
    planeCSNode.firstPoint.point.toolAxisAngle[i] = toolAxisAngle1[i];
    planeCSNode.secondPoint.point.toolAxisAngle[i] = toolAxisAngle2[i];
    planeCSNode.thirdPoint.point.toolAxisAngle[i] = toolAxisAngle3[i];
    planeCSNode.firstPoint.point.toolPosition[i] = toolPosition1[i];
    planeCSNode.secondPoint.point.toolPosition[i] = toolPosition2[i];
    planeCSNode.thirdPoint.point.toolPosition[i] = toolPosition3[i];
    planeCSNode.coordinatePose[i] = coordinatePose[i];
    planeCSNode.coordinateJointPos[i] = coordinateJointPos[i];
}
planeCSNode.firstPoint.id = 1;
planeCSNode.firstPoint.isValid = 1;
planeCSNode.secondPoint.id = 2;
planeCSNode.secondPoint.isValid = 1;
planeCSNode.thirdPoint.id = 3;
planeCSNode.thirdPoint.isValid = 1;
planeCSNode.id = 2;
result = cr_cfg_cs_plane_set(robotHandle, 1, planeCSNode); //修改索引号为 1 的面坐标系数据
if (result != success) //判断修改面坐标系数据是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
```

```
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.15.19 读取基坐标系数据

#### 函数

**CRresult** cr\_cfg\_cs\_base\_get(**RobotHandle** robotHandle, double \*csPose, int len)

#### 简介

读取基坐标系数据。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述         |
|-------------|-------------|-------|------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄    |
| csPose      | double*     | 输出    | 基坐标系数据     |
| len         | int         | 输入    | 数据长度，长度为 6 |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int len = 6;
double csPose[6];
result = cr_cfg_cs_base_get(robotHandle, csPose, 6); //读取基坐标系数据
if (result != success) //判断读取基坐标系数据是否成功
    return -1;
for (int i = 0; i < 6; i++)
{
    printf("%f\n", csPose[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.15.20 读取工具坐标系数据

#### 函数

**CRresult** cr\_cfg\_cs\_tool\_get(**RobotHandle** robotHandle, double \*csPose, int len)

## 简介

读取工具坐标系数据。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述         |
|-------------|-------------|-------|------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄    |
| csPose      | double *    | 输出    | 工具坐标系数据    |
| len         | int         | 输入    | 数据长度，长度为 6 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int len = 6;
double csPose[6];
result = cr_cfg_cs_tool_get(robotHandle, csPose, len); //读取工具坐标系数据
if (result != success) //判断读取工具坐标系数据是否成功
    return -1;
for (int i = 0; i < 6; i++)
{
    printf("%f\n", csPose[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.15.21 计算点坐标系

## 函数

**CRresult** cr\_compute\_cs\_point(double \*pointPose, int pLen, double \*pointCS, int csLen)

## 简介

计算点坐标系。

## 参数

| 参数名称      | 数据类型     | 输入/输出 | 描述                                    |
|-----------|----------|-------|---------------------------------------|
| pointPose | double * | 输入    | 计算点坐标系的位姿：x，y，z（单位：mm）；Rx，Ry，Rz（单位：°） |

|         |          |    |                                           |
|---------|----------|----|-------------------------------------------|
| pLen    | int      | 输入 | 位姿的长度，至少为 6                               |
| pointCS | double * | 输出 | 点坐标系的位姿：x, y, z (单位：mm)；Rx, Ry, Rz (单位：°) |
| csLen   | int      | 输入 | 点坐标系的长度，至少为 6                             |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
int pLen = 6;
int csLen = 6;
double pointPose[6] = { -66, -438, 887, -57, 0, -148 };
double pointCS[6];
CRresult result = cr_compute_cs_point(pointPose, pLen, pointCS, csLen); //计算点坐标系
if (result != success) //判断计算点坐标系是否成功
    return -1;
for (int i = 0; i < 6; i++)
{
    printf("%f\n", pointCS[i]);
}
```

## 3.6.15.22 计算线坐标系

### 函数

**CRresult** cr\_compute\_cs\_line(double \*point1Pose,int p1Len,double \*point2Pose,int p2Len,double \*lineCS,int csLen)

### 简介

计算线坐标系。

### 参数

| 参数名称       | 数据类型     | 输入/输出 | 描述                                            |
|------------|----------|-------|-----------------------------------------------|
| point1Pose | double * | 输入    | 计算线坐标系的位姿 1：x, y, z (单位：mm)；Rx, Ry, Rz (单位：°) |
| p1Len      | int      | 输入    | 位姿的长度，至少为 6                                   |
| point2Pose | double * | 输入    | 计算线坐标系的位姿 2：x, y, z (单位：mm)；Rx, Ry, Rz (单位：°) |
| p2Len      | int      | 输入    | 位姿的长度，至少为 6                                   |
| lineCS     | double * | 输出    | 线坐标系的位姿：x, y, z (单位：mm)；Rx, Ry, Rz (单位：°)     |
| csLen      | int      | 输入    | 线坐标系的长度，至少为 6                                 |



## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
int p1Len = 6;
int p2Len = 6;
int csLen = 6;
double point1Pose[6] = { -66, -438, 887, -57, 0, -148 };
double point2Pose[6] = { -76, -438, 887, -57, 0, -148 };
double lineCS[6];
CRresult result = cr_compute_cs_line(point1Pose, p1Len, point2Pose, p2Len, lineCS, csLen);
//计算线坐标系
if (result != success) //判断计算线坐标系是否成功
    return -1;
for (int i = 0; i < 6; i++)
{
    printf("%f\n", lineCS[i]);
}
```

### 3.6.15.23 计算面坐标系

## 函数

**CRresult** cr\_compute\_cs\_plane(double \*point1Pose,int p1Len,double \*point2Pose,int p2Len,double \*point3Pose,int p3Len,double \*planeCS,int csLen)

## 简介

计算面坐标系。

## 参数

| 参数名称       | 数据类型     | 输入/输出 | 描述                                                |
|------------|----------|-------|---------------------------------------------------|
| point1Pose | double * | 输入    | 计算面坐标系的位姿 1: x, y, z (单位: mm); Rx, Ry, Rz (单位: °) |
| p1Len      | int      | 输入    | 位姿的长度, 至少为 6                                      |
| point2Pose | double * | 输入    | 计算面坐标系的位姿 2: x, y, z (单位: mm); Rx, Ry, Rz (单位: °) |
| p2Len      | int      | 输入    | 位姿的长度, 至少为 6                                      |
| point3Pose | double * | 输入    | 计算面坐标系的位姿 3: x, y, z (单位: mm); Rx, Ry, Rz (单位: °) |
| p3Len      | int      | 输入    | 位姿的长度, 至少为 6                                      |
| planeCS    | double * | 输出    | 面坐标系的位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)     |
| csLen      | int      | 输入    | 面坐标系的长度, 至少为 6                                    |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
int p1Len = 6;
int p2Len = 6;
int p3Len = 6;
int csLen = 6;
double point1Pose[6] = { -70, -436.75, 893, 27.15, 22, 150.76 };
double point2Pose[6] = { -75, -436.75, 893, 27.15, 22, 150.76 };
double point3Pose[6] = { -75, -440, 893, 27.15, 22, 150.76 };
double planeCS[6];
CRresult result = cr_compute_cs_plane(point1Pose, p1Len, point2Pose, p2Len, point3Pose,
p3Len, planeCS, csLen); //计算面坐标系
if (result != success) //判断计算面坐标系是否成功
    return -1;
for (int i = 0; i < 6; i++)
{
    printf("%f\n", planeCS[i]);
}
```

## 3.6.16 串口通讯

### 3.6.16.1 设置串口配置

## 函数

CRresult cr\_cfg\_comm\_serial\_setting\_set(RobotHandle robotHandle, SerialType serialType, SerialCommSettings serialCommSettings)

## 简介

设置串口配置。

## 参数

| 参数名称               | 数据类型               | 输入/输出 | 描述      |
|--------------------|--------------------|-------|---------|
| robotHandle        | RobotHandle        | 输入    | 机械臂实例句柄 |
| serialType         | SerialType         | 输入    | 串口类型    |
| serialCommSettings | SerialCommSettings | 输入    | 串口配置数据  |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
```

```
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SerialType serialType;
serialType = rs485_1_CommSettings;
SerialCommSettings serialCommSettings;
serialCommSettings.parity = odd;
serialCommSettings.serialCommType = Modbus_ASCII;
serialCommSettings.baudRate = BaudRate1200000;
serialCommSettings.dataBits = 7;
serialCommSettings.modbusSlaveNo = 1;
serialCommSettings.retryNumber = 3;
serialCommSettings.stopBits = 1;
serialCommSettings.timeout = 300;
result = cr_cfg_comm_serial_setting_set(robotHandle, serialType, serialCommSettings);
//设置串口配置
if (result != success) //判断设置串口配置是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.16.2 读取串口配置

#### 函数

**CRresult** cr\_cfg\_comm\_serial\_setting\_get(**RobotHandle** robotHandle, **SerialType** serialType, **SerialCommSettings** \*serialCommSettings)

#### 简介

读取串口配置。

#### 参数

| 参数名称               | 数据类型                 | 输入/输出 | 描述      |
|--------------------|----------------------|-------|---------|
| robotHandle        | RobotHandle          | 输入    | 机械臂实例句柄 |
| serialType         | SerialType           | 输入    | 串口类型    |
| serialCommSettings | SerialCommSettings * | 输出    | 串口配置数据  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
```

```

        return -1;
SerialType serialType;
serialType = rs485_1_CommSettings;
SerialCommSettings serialCommSettings;
result = cr_cfg_comm_serial_setting_get(robotHandle, serialType, &serialCommSettings);
//读取串口配置
if (result != success) //判断读取串口配置是否成功
    return -1;
printf("%d\n", serialCommSettings.baudRate);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;

```

## 3.6.17 网络

### 3.6.17.1 设置控制柜网络配置

#### 函数

**CRresult** cr\_cfg\_comm\_ethernet\_ip\_set(**RobotHandle** robotHandle, **IPConfig** ipconfig)

#### 简介

设置控制柜网络配置。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| ipconfig    | IPConfig    | 输入    | 网络地址配置  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
IPConfig ipconfig;
ipconfig.ipAddr[0] = 192;
ipconfig.ipAddr[1] = 168;
ipconfig.ipAddr[2] = 6;
ipconfig.ipAddr[3] = 6;
ipconfig.subnetMask[0] = 255;
ipconfig.subnetMask[1] = 255;

```

```
ipconfig.subnetMask[2] = 255;
ipconfig.subnetMask[3] = 0;
result = cr_cfg_comm_ethernet_ip_set(robotHandle, ipconfig); //设置控制柜网络配置
if (result != success) //判断设置控制柜网络配置是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.17.2 读取控制柜网络配置

#### 函数

**CRresult** cr\_cfg\_comm\_ethernet\_ip\_get(**RobotHandle** robotHandle, **IPConfig** \*ipconfig)

#### 简介

读取控制柜网络配置。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| ipconfig    | IPConfig *  | 输出    | 网络地址配置  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
IPConfig ipconfig;
result = cr_cfg_comm_ethernet_ip_get(robotHandle, &ipconfig); //读取控制柜网络配置
if (result != success) //判断读取控制柜网络配置是否成功
    return -1;
for (int i = 0; i < 4; i++)
{
    printf("%d\n", ipconfig.ipAddr[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.17.3 设置控制柜从站配置

#### 函数

```
CRresult cr_cfg_comm_ethernet_modbus_slave_num_set(RobotHandle robotHandle,int  
modbusSlaveNo)
```

#### 简介

设置控制柜从站配置。

#### 参数

| 参数名称          | 数据类型        | 输入/输出 | 描述      |
|---------------|-------------|-------|---------|
| robotHandle   | RobotHandle | 输入    | 机械臂实例句柄 |
| modbusSlaveNo | int         | 输入    | 从站号     |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;  
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接  
if (result != success) //判断机器人连接是否成功  
    return -1;  
int modbusSlaveNo = 1;  
result = cr_cfg_comm_ethernet_modbus_slave_num_set(robotHandle, modbusSlaveNo);  
//将控制柜从站站号设置为 1  
if (result != success) //判断设置控制柜从站配置是否成功  
    return -1;  
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄  
if (result != success) //判断机器人断开连接是否成功  
    return -1;
```

### 3.6.17.4 读取控制柜从站配置

#### 函数

```
CRresult cr_cfg_comm_ethernet_modbus_slave_num_get(RobotHandle robotHandle,int  
*modbusSlaveNo)
```

#### 简介

读取控制柜从站配置。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

|               |       |    |     |
|---------------|-------|----|-----|
| modbusSlaveNo | int * | 输出 | 从站号 |
|---------------|-------|----|-----|

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int modbusSlaveNo;
result = cr_cfg_comm_ethernet_modbus_slave_num_get(robotHandle, &modbusSlaveNo);
//读取控制柜从站配置
if (result != success) //判断读取控制柜从站配置是否成功
    return -1;
printf("%d\n", modbusSlaveNo);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.18 modbus 主站

### 3.6.18.1 读取从站个数

#### 函数

```
CRresult cr_cfg_comm_modbus_slave_count(RobotHandle robotHandle, SerialType type, int
*count)
```

#### 简介

读取从站个数。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| type        | SerialType  | 输入    | 从站类型    |
| count       | int *       | 输出    | 从站数量    |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
```

```
if (result != success) //判断机器人连接是否成功
    return -1;
SerialType type = rs485_1_CommSettings;
int count;
result = cr_cfg_comm_modbus_slave_count(robotHandle, type, &count);
//读取 rs485_1 的从站个数
if (result != success) //判断读取从站个数是否成功
    return -1;
printf("%d\n", count);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.18.2 读取从站数据

#### 函数

`CRresult cr_cfg_comm_modbus_slave_get(RobotHandle robotHandle, SerialType type, int index, ModbusMasterConfig *modbusConfig)`

#### 简介

读取从站数据。

#### 参数

| 参数名称         | 数据类型                 | 输入/输出 | 描述            |
|--------------|----------------------|-------|---------------|
| robotHandle  | RobotHandle          | 输入    | 机械臂实例句柄       |
| type         | SerialType           | 输入    | 从站类型          |
| index        | int                  | 输入    | 从站索引（范围：0-19） |
| modbusConfig | ModbusMasterConfig * | 输出    | 从站配置信息        |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SerialType type = rs485_1_CommSettings;
ModbusMasterConfig modbusConfig;
result = cr_cfg_comm_modbus_slave_get(robotHandle, type, 1, &modbusConfig);
//读取 rs485_1 从站索引号为 1 的从站数据
if (result != success) //判断读取从站数据是否成功
    return -1;
```



```
printf("%d\n", modbusConfig.slaveNo);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.18.3 添加从站

#### 函数

`CRresult cr_cfg_comm_modbus_slave_add(RobotHandle robotHandle, SerialType type, ModbusMasterConfig *modbusConfig)`

#### 简介

添加从站。

#### 参数

| 参数名称         | 数据类型                 | 输入/输出 | 描述      |
|--------------|----------------------|-------|---------|
| robotHandle  | RobotHandle          | 输入    | 机械臂实例句柄 |
| type         | SerialType           | 输入    | 从站类型    |
| modbusConfig | ModbusMasterConfig * | 输入    | 从站配置信息  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SerialType type = rs485_1_CommSettings;
ModbusMasterConfig modbusConfig;
modbusConfig.id = 3;
modbusConfig.slaveNo = 4;
modbusConfig.scanTime = 5;
modbusConfig.autoConnect = 0;
modbusConfig.modbusMasterOperate = Stop;
result = cr_cfg_comm_modbus_slave_add(robotHandle, type, &modbusConfig);
//添加 rs485_1 的从站
if (result != success) //判断添加从站是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.18.4 删除从站

#### 函数

```
CRresult cr_cfg_comm_modbus_slave_delete(RobotHandle robotHandle, SerialType type, int index)
```

#### 简介

删除从站。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述            |
|-------------|-------------|-------|---------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄       |
| type        | SerialType  | 输入    | 从站类型          |
| index       | int         | 输入    | 从站索引（范围：0-19） |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;  
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接  
if (result != success) //判断机器人连接是否成功  
    return -1;  
SerialType type = rs485_1_CommSettings;  
result = cr_cfg_comm_modbus_slave_delete(robotHandle, type, 1);  
//删除 rs485_1 索引号为 1 的从站  
if (result != success) //判断删除从站是否成功  
    return -1;  
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄  
if (result != success) //判断机器人断开连接是否成功  
    return -1;
```

### 3.6.18.5 修改从站数据

#### 函数

```
CRresult cr_cfg_comm_modbus_slave_set(RobotHandle robotHandle, SerialType type, int index, ModbusMasterConfig modbusConfig)
```

#### 简介

修改从站数据。

#### 参数

| 参数名称 | 数据类型 | 输入/输出 | 描述 |
|------|------|-------|----|
|------|------|-------|----|

|              |                    |    |               |
|--------------|--------------------|----|---------------|
| robotHandle  | RobotHandle        | 输入 | 机械臂实例句柄       |
| type         | SerialType         | 输入 | 从站类型          |
| index        | int                | 输入 | 从站索引（范围：0-19） |
| modbusConfig | ModbusMasterConfig | 输入 | 从站配置信息        |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SerialType type = rs485_1_CommSettings;
ModbusMasterConfig modbusConfig;
modbusConfig.id = 3;
modbusConfig.slaveNo = 4;
modbusConfig.scanTime = 5;
modbusConfig.autoConnect = 0;
modbusConfig.modbusMasterOperate = Stop;
result = cr_cfg_comm_modbus_slave_set(robotHandle, type, 1, modbusConfig);
//修改 rs485_1 索引号为 1 的从站数据
if (result != success) //判断修改从站数据是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.18.6 从站连接/断开

### 函数

**CRresult** cr\_cfg\_comm\_modbus\_slave\_operate(**RobotHandle** robotHandle, **SerialType** type, int index, **ModbusMasterOperate** modbusOperate)

### 简介

从站连接/断开。

### 参数

| 参数名称          | 数据类型                | 输入/输出 | 描述            |
|---------------|---------------------|-------|---------------|
| robotHandle   | RobotHandle         | 输入    | 机械臂实例句柄       |
| type          | SerialType          | 输入    | 从站类型          |
| index         | int                 | 输入    | 从站索引（范围：0-19） |
| modbusOperate | ModbusMasterOperate | 输入    | 从站操作          |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
ModbusMasterOperate modbusOperate = Start;
SerialType type = rs485_1_CommSettings;
result = cr_cfg_comm_modbus_slave_operate(robotHandle, type, 1, modbusOperate);
//连接 rs485_1 从站索引号为 1 的从站
if (result != success) //判断从站连接是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.18.7 获取从站连接状态

## 函数

[CRresult](#) cr\_cfg\_comm\_modbus\_slave\_connect\_get([RobotHandle](#) robotHandle, [SerialType](#) type, int index, int\* status)

## 简介

获取从站连接状态。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述            |
|-------------|-------------|-------|---------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄       |
| type        | SerialType  | 输入    | 从站类型          |
| index       | int         | 输入    | 从站索引（范围：0-19） |
| status      | int*        | 输出    | 从站连接状态        |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int status;
```

```
SerialType type = rs485_1_CommSettings;
result = cr_cfg_comm_modbus_slave_connect_get(robotHandle, type, 0, &status);
//读取 rs485_1 从站索引号为 0 的从站连接状态
if (result != success) //判断从站连接状态是否成功
    return -1;
printf("%d\n", status);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.18.8 读取地址映射数量

#### 函数

**CRresult** cr\_cfg\_comm\_modbus\_addr\_map\_count(**RobotHandle** robotHandle, **SerialType** type, int serialIndex, int \*count)

#### 简介

读取地址映射数量。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述            |
|-------------|-------------|-------|---------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄       |
| type        | SerialType  | 输入    | 从站类型          |
| serialIndex | int         | 输入    | 从站索引（范围：0-19） |
| count       | int *       | 输出    | 地址映射数量        |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
int count;
SerialType type = rs485_1_CommSettings;
result = cr_cfg_comm_modbus_addr_map_count(robotHandle, type, 1, &count);
//读取 rs485_1 从站索引号为 1 的从站的地址映射数量
if (result != success) //判断读取地址映射数量是否成功
    return -1;
printf("%d\n", count);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
```

```
return -1;
```

### 3.6.18.9 读取地址映射数据

#### 函数

```
CRresult cr_cfg_comm_modbus_addr_map_get(RobotHandle robotHandle, SerialType type,int
serialIndex,int index,ModbusAddrMap *addrMap)
```

#### 简介

读取地址映射数据。

#### 参数

| 参数名称        | 数据类型           | 输入/输出 | 描述              |
|-------------|----------------|-------|-----------------|
| robotHandle | RobotHandle    | 输入    | 机械臂实例句柄         |
| type        | SerialType     | 输入    | 从站类型            |
| serialIndex | int            | 输入    | 从站索引（范围：0-19）   |
| index       | int            | 输入    | 地址映射索引（范围：0-99） |
| addrMap     | ModbusAddrMap* | 输出    | 地址映射数据          |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323,"123"); //机器人连接
if(result != success) //判断机器人连接是否成功
    return -1;
SerialType type = rs485_1_CommSettings;
ModbusAddrMap addrMap;
result = cr_cfg_comm_modbus_addr_map_get(robotHandle, type, 1, 1, &addrMap);
//读取 rs485_1 从站索引号为 1 的从站的地址映射索引号为 1 的地址映射数据
if(result != success) //判断读取地址映射数量是否成功
    return -1;
printf("%d\n", addrMap.masterStartAddr);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if(result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.18.10 添加地址映射

#### 函数

```
CRresult cr_cfg_comm_modbus_addr_map_add(RobotHandle robotHandle, SerialType type,int
serialIndex,ModbusAddrMap addrMap)
```

## 简介

添加地址映射。

## 参数

| 参数名称        | 数据类型          | 输入/输出 | 描述            |
|-------------|---------------|-------|---------------|
| robotHandle | RobotHandle   | 输入    | 机械臂实例句柄       |
| type        | SerialType    | 输入    | 从站类型          |
| serialIndex | int           | 输入    | 从站索引（范围：0-19） |
| addrMap     | ModbusAddrMap | 输入    | 地址映射数据        |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SerialType type = rs232_CommSettings;
ModbusAddrMap addrMap;
int serialIndex = 2;
addrMap.masterOffsetNo = 1;
addrMap.functionNo = WriteBool;
addrMap.masterStartAddr = 2;
addrMap.slaveStartAddr = 1;
result = cr_cfg_comm_modbus_addr_map_add(robotHandle, type, serialIndex, addrMap);
//添加 rs232 从站索引号为 2 的地址映射
if (result != success) //判断添加地址映射是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.18.11 删除地址映射

## 函数

```
CRresult cr_cfg_comm_modbus_addr_map_delete(RobotHandle robotHandle, SerialType type,
int serialIndex, int index)
```

## 简介

删除地址映射。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述              |
|-------------|-------------|-------|-----------------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄         |
| type        | SerialType  | 输入    | 从站类型            |
| serialIndex | int         | 输入    | 从站索引（范围：0-19）   |
| index       | int         | 输入    | 地址映射索引（范围：0-99） |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SerialType type = rs485_1_CommSettings;
result = cr_cfg_comm_modbus_addr_map_delete(robotHandle, type, 1, 1);
//删除 rs485_1 从站索引号为 1 的从站的地址映射索引号为 1 的地址映射
if (result != success) //判断删除地址映射是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.18.12 修改地址映射数据

## 函数

**CRresult** cr\_cfg\_comm\_modbus\_addr\_map\_set(**RobotHandle** robotHandle, **SerialType** type, int serialIndex, int index, **ModbusAddrMap** addrMap)

## 简介

修改地址映射数据。

## 参数

| 参数名称        | 数据类型          | 输入/输出 | 描述              |
|-------------|---------------|-------|-----------------|
| robotHandle | RobotHandle   | 输入    | 机械臂实例句柄         |
| type        | SerialType    | 输入    | 从站类型            |
| serialIndex | int           | 输入    | 从站索引（范围：0-19）   |
| index       | int           | 输入    | 地址映射索引（范围：0-99） |
| addrMap     | ModbusAddrMap | 输入    | 地址映射数据          |



## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
SerialType type = rs232_CommSettings;
ModbusAddrMap addrMap;
int serialIndex = 2;
addrMap.masterOffsetNo = 1;
addrMap.functionNo = WriteBool;
addrMap.masterStartAddr = 2;
addrMap.slaveStartAddr = 1;
result = cr_cfg_comm_modbus_addr_map_set(robotHandle, type, serialIndex, 1, addrMap);
//修改 rs232 从站索引号为 2 的从站的地址映射索引号为 1 的地址映射数据
if (result != success) //判断修改地址映射数据是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.19 力控

### 3.6.19.1 设置力控配置

## 函数

CRresult cr\_force\_cfg\_set(RobotHandle robotHandle, ForceConfig forceConfig)

## 简介

设置力控配置。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| forceConfig | ForceConfig | 输入    | 力控配置参数  |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
```

```
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
ForceConfig forceConfig = { 0 }; //结构体初始化
char activeForceTCP_name[20] = "TCP_4"; //激活的力控传感器 TCP 的名称
memcpy(forceConfig.forceCtrlTcpImpl.activeForceTCP_name, activeForceTCP_name,
sizeof(activeForceTCP_name));
char activePayload_name[20] = "ForcePayload7"; //激活的力控传感器负载的名称
memcpy(forceConfig.forceCtrlTcpImpl.activeForcePayload_name, activePayload_name,
sizeof(activePayload_name));
ForceTCPMsg availTcp[20] = { { "TCP_4", 0, { 0, 0, 0, 0, -110 } },
{ "ForceTcp4", 1, { 0, 0, 0, 180, 0, 0 } }, { "ForceTcp5", 2, { 0, 0, 0, 180, 0, 0 } } };
//当前待选的力控传感器 TCP 的配置信息
memcpy(forceConfig.forceCtrlTcpImpl.availableForceTCP, availTcp, sizeof(ForceTCPMsg) *
20);
ForcePayLoad availPayload[20] = { { "ForcePayload7", 0, 1.3, { 1, 2, 3 } },
{ "ForcePayload5", 1, 1.3, { 1, 2, 3 } }, { "ForcePayload6", 2, 0.5, { 0, 0, 0 } } };
//当前待选的力控传感器负载的配置信息
memcpy(forceConfig.forceCtrlTcpImpl.availableForcePayLoad, availPayload,
sizeof(ForcePayLoad) * 20);
forceConfig.forceCtrlTcpImpl.availableForceTCPLen = 3;
//当前待选的力控传感器 TCP 的配置信息的个数
forceConfig.forceCtrlTcpImpl.availableForcePayLoadLen = 3;
// 当前待选的力控传感器负载的配置信息的个数

forceConfig.commuConfig.autoConnect = 1;
//自动与传感器通信，传感器开始接收数据
forceConfig.commuConfig.mannaulOperate = 0;
//自动与传感器通信，传感器开始接收数据
forceConfig.commuConfig.paraBuf = 0; //预留参数

forceConfig.sensorMessage.venderNo = 0; //传感器品牌
forceConfig.sensorMessage.productNo = 1; //传感器产品号，预留参数
forceConfig.sensorMessage.sequenceNo = 0; //传感器序列号，预留参数
result = cr_force_cfg_set(robotHandle, forceConfig); //设置力控配置
if (result != success) //判断设置力控配置是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.19.2 读力控配置

#### 函数

`CRresult cr_force_cfg_get(RobotHandle robotHandle, ForceConfig *forceConfig)`

#### 简介

读力控配置。

#### 参数

| 参数名称        | 数据类型          | 输入/输出 | 描述      |
|-------------|---------------|-------|---------|
| robotHandle | RobotHandle   | 输入    | 机械臂实例句柄 |
| forceConfig | ForceConfig * | 输出    | 力控配置参数  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
ForceConfig forceConfig = { 0 };
result = cr_force_cfg_get(robotHandle, &forceConfig); //读力控配置
if (result != success) //判断读力控配置是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.19.3 开启力控并下发相关设置

#### 函数

`CRresult cr_force_open(RobotHandle robotHandle, ForceSetting forceSetting)`

#### 简介

开启力控并下发相关设置。

#### 参数

| 参数名称         | 数据类型         | 输入/输出 | 描述      |
|--------------|--------------|-------|---------|
| robotHandle  | RobotHandle  | 输入    | 机械臂实例句柄 |
| forceSetting | ForceSetting | 输入    | 力控设置参数  |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
ForceSetting forceSetting={0};
forceSetting.forceBaseType = 0;
//力控类型: 0-基于 TCP 力传感器,1-基于动力学,2-基于底座力传感器
forceSetting.forceCtrlType = 1; //力控用法: 0-运动力控,1-力控拖动
int flexibleAxis[6] = { 0,0,1,0,0,0 }; //z 轴可拖动
memcpy(forceSetting.flexibleAxis, flexibleAxis, sizeof(flexibleAxis));
result = cr_force_open(robotHandle, forceSetting); //开启力控并下发相关配置
if (result != success) //判断开启力控并下发相关配置是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.19.4 关闭力控

## 函数

**CRresult** cr\_force\_close(**RobotHandle** robotHandle)

## 简介

关闭力控。

## 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

## 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_force_close(robotHandle); //关闭力控
```

```

if (result != success) //判断关闭力控是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;

```

### 3.6.19.5 设置力控控制参数

#### 函数

**CRresult** cr\_force\_para\_set(**RobotHandle** robotHandle, **ForceCtlPara** forceCtlPara)

#### 简介

设置力控控制参数。

#### 参数

| 参数名称         | 数据类型         | 输入/输出 | 描述      |
|--------------|--------------|-------|---------|
| robotHandle  | RobotHandle  | 输入    | 机械臂实例句柄 |
| forceCtlPara | ForceCtlPara | 输入    | 力控控制参数  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
ForceCtlPara forceCtlPara = { 0 };
double damping[6] = { 1,1,0.1,1,1,1 }; //x,y,Rx,Ry,Rz 阻尼系数均为 1,z 阻尼系数为 0.1
double stiffness[6] = { 1,1,0.1,1,1,1 }; //x,y,Rx,Ry,Rz 刚度均为 1,z 刚度为 0.1
double mass[6] = { 1,1,0.5,1,1,1 }; //x,y,Rx,Ry,Rz 质量系数均为 1,z 质量系数为 0.5
double limits[6] = { 0.1,0.1,0.005,0.002,0.002,0.002 };
//x,y 方向速度保护为 0.1m/s, z 方向速度保护为 0.005m/s,Rx,Ry,Rz 偏差保护为 0.002rad
memcpy(forceCtlPara.damping, damping, sizeof(double) * 6);
memcpy(forceCtlPara.stiffness, stiffness, sizeof(double) * 6);
memcpy(forceCtlPara.mass, mass, sizeof(double) * 6);
memcpy(forceCtlPara.limits, limits, sizeof(double) * 6);

double taskframe[6] = { 0,0,0,0,0,0 }; //力控坐标系为基坐标系
double wrench[6] = { 0,0,10,0,0,0 }; //z 轴正向施加 10N 的力
memcpy(forceCtlPara.taskFrame, taskframe, sizeof(double) * 6);
memcpy(forceCtlPara.wrench, wrench, sizeof(double) * 6);

```

```
forceCtlPara.dampingLen = 6; //0、1 或 6
forceCtlPara.limitsLen = 6; //0 或 6
forceCtlPara.massLen = 6; //0、1 或 6
forceCtlPara.stiffnessLen = 6; //0、1 或 6

result = cr_force_para_set(robotHandle, forceCtlPara); //设置力控控制参数
if (result != success) //判断设置力控控制参数是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.19.6 读力控控制参数

#### 函数

```
CRresult cr_force_para_get(RobotHandle robotHandle, ForceCtlPara *forceCtlPara)
```

#### 简介

读力控控制参数。

#### 参数

| 参数名称         | 数据类型           | 输入/输出 | 描述      |
|--------------|----------------|-------|---------|
| robotHandle  | RobotHandle    | 输入    | 机械臂实例句柄 |
| forceCtlPara | ForceCtlPara * | 输出    | 力控控制参数  |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
ForceCtlPara forceCtlPara = { 0 };
result = cr_force_para_get(robotHandle, &forceCtlPara); //读力控控制参数
if (result != success) //判断读力控控制参数是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.19.7 读力控数据

#### 函数

**CRresult** cr\_force\_data\_get(**RobotHandle** robotHandle, **ForceData** \*forceData)

#### 简介

读力控数据。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| forceData   | ForceData * | 输出    | 力控数据    |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
ForceData forceData = { 0 };
result = cr_force_data_get(robotHandle, &forceData); //读力控数据
if (result != success) //判断读力控数据是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.20 扩展轴

### 3.6.20.1 设置扩展轴使能状态

#### 函数

**CRresult** cr\_set\_exj\_enableStatus(**RobotHandle** robotHandle, int index, **BOOL** status)

#### 简介

设置扩展轴使能状态。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

|        |      |    |       |
|--------|------|----|-------|
| index  | int  | 输入 | 扩展轴索引 |
| status | BOOL | 输入 | 扩展轴状态 |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
BOOL status = TRUE;
result = cr_set_exj_enableStatus(robotHandle, 0, status); //设置扩展轴使能状态
if (result != success) //判断设置扩展轴使能状态是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.20.2 读取扩展轴使能状态

### 函数

```
CRresult cr_get_exj_enableStatus(RobotHandle robotHandle, int index, BOOL* status)
```

### 简介

读取扩展轴使能状态。

### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| index       | int         | 输入    | 扩展轴索引   |
| status      | BOOL*       | 输出    | 扩展轴状态   |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
BOOL status;
result = cr_get_exj_enableStatus(robotHandle, 0, &status); //读取扩展轴使能状态
```



```
if (result != success) //判断读取扩展轴使能状态是否成功
    return -1;
printf("%d\n", status);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.20.3 读取机械臂和扩展轴数据

#### 函数

**CRresult** cr\_get\_robot\_exj\_data(**RobotHandle** robotHandle, **RobotExjData\*** robotExjData)

#### 简介

读取机械臂和扩展轴数据。

#### 参数

| 参数名称         | 数据类型          | 输入/输出 | 描述      |
|--------------|---------------|-------|---------|
| robotHandle  | RobotHandle   | 输入    | 机械臂实例句柄 |
| robotExjData | RobotExjData* | 输出    | 待读取的值   |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
RobotExjData robotExjData;
result = cr_get_robot_exj_data(robotHandle, &robotExjData); //读取机械臂和扩展轴数据
if (result != success) //判断读取机械臂和扩展轴数据是否成功
    return -1;
printf("%f\n", robotExjData.actualExjointPos[0]);
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", robotExjData.actualJointPos[i]);
}
for (int i = 0; i < ROB_AXIS_NUM; i++)
{
    printf("%f\n", robotExjData.actualTcpVector[i]);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
```

```
return -1;
```

### 3.6.20.4 设置扩展轴配置

#### 函数

`CRresult cr_cfg_exj_set(RobotHandle robotHandle, const char* exjName, ExjConfig exjConfig)`

#### 简介

设置扩展轴配置。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| exjName     | const char* | 输入    | 扩展轴名称   |
| exjConfig   | ExjConfig   | 输入    | 扩展轴配置   |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
ExjConfig exjConfig;
strcpy_s(exjConfig.name, "Exj_3");
exjConfig.type = line_externalJoint;
exjConfig.ratio = 100;
exjConfig.minlimit = -99999;
exjConfig.maxlimit = 99999;
exjConfig.zeroPosition = 2;
memset(exjConfig.exjToRef, 0, sizeof(double)*ROB_AXIS_NUM);
strcpy_s(exjConfig.masterExjName, "Exj_2");
exjConfig.masterSlaveRate = 1;
const char* exjName = "Exj_0";
result = cr_cfg_exj_set(robotHandle, exjName, exjConfig); //设置扩展轴配置
if (result != success) //判断设置扩展轴配置是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.20.5 获取扩展轴配置

#### 函数

`CRresult cr_cfg_exj_get(RobotHandle robotHandle, const char* exjName, ExjConfig* exjConfig)`

#### 简介

获取扩展轴配置。

#### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |
| exjName     | const char* | 输入    | 扩展轴名称   |
| exjConfig   | ExjConfig*  | 输出    | 扩展轴配置   |

#### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

#### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
ExjConfig exjConfig;
const char* exjName = "Exj_0";
result = cr_cfg_exj_get(robotHandle, exjName, &exjConfig); //获取扩展轴配置
if (result != success) //判断获取扩展轴配置是否成功
    return -1;
printf("%f\n", exjConfig.ratio);
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

### 3.6.20.6 增加扩展轴

#### 函数

`CRresult cr_cfg_exj_add(RobotHandle robotHandle, ExjConfig exjConfig)`

#### 简介

增加扩展轴。

#### 参数

| 参数名称 | 数据类型 | 输入/输出 | 描述 |
|------|------|-------|----|
|------|------|-------|----|

|             |             |    |         |
|-------------|-------------|----|---------|
| robotHandle | RobotHandle | 输入 | 机械臂实例句柄 |
| exjConfig   | ExjConfig   | 输入 | 扩展轴配置   |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
ExjConfig exjConfig;
strcpy_s(exjConfig.name, "Exj_2");
exjConfig.type = line_externalJoint;
exjConfig.ratio = 100;
exjConfig.minlimit = -99999;
exjConfig.maxlimit = 99999;
exjConfig.zeroPosition = 2;
memset(exjConfig.exjToRef, 0, sizeof(double)*ROB_AXIS_NUM);
strcpy_s(exjConfig.masterExjName, "Exj_1");
exjConfig.masterSlaveRate = 1;
result = cr_cfg_exj_add(robotHandle, exjConfig); //增加扩展轴
if (result != success) //判断增加扩展轴是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

## 3.6.20.7 删除扩展轴

### 函数

**CRresult** cr\_cfg\_exj\_delete(**RobotHandle** robotHandle)

### 简介

删除扩展轴（只能删除最后一个）。

### 参数

| 参数名称        | 数据类型        | 输入/输出 | 描述      |
|-------------|-------------|-------|---------|
| robotHandle | RobotHandle | 输入    | 机械臂实例句柄 |

### 返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

## 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); //机器人连接
if (result != success) //判断机器人连接是否成功
    return -1;
result = cr_cfg_exj_delete(robotHandle); //删除扩展轴
if (result != success) //判断删除扩展轴是否成功
    return -1;
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != success) //判断机器人断开连接是否成功
    return -1;
```

# 四、综合示例

## 4.1 机械臂使能

### 简介

对机械臂进行操作需要使机械臂处于使能状态。本示例的功能为：在机械臂处于未上电状态时，通过 sdk 接口使机械臂上电使能完成。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); // 机械臂连接
if (result == success)
{
    enum RobotModes robotMode;
    result = cr_get_robotMode(robotHandle, &robotMode); //读机械臂当前状态
    if (result == success && robotMode == 6)
        //判断状态，robotMode 为 6 时是本体未上电状态才可以进行上电
        {
            result = cr_poweron(robotHandle); //机械臂上电
            if (result == success)
            {
                while (result == success)
                {
                    result = cr_get_robotMode(robotHandle, &robotMode);
                    Sleep(50);
                    if (robotMode == 8)
                        //判断状态，robotMode 为 8 时是机器人上电完成后再进行使能
                        break;
                }
                result = cr_enable(robotHandle); //机械臂使能
            }
        }
}
```

```

        if (result == success )
        {
            while (result == success)
            {
                result = cr_get_robotMode(robotHandle, &robotMode);
                Sleep(50);
                if (robotMode == 103)
                    //判断状态,robotMode 为 103 时机器人使能完成
                    break;
            }
            printf("%d:%s\n", __LINE__, "使能完成");
        }
        else
        {
            printf("%d:%d\n", __LINE__, result);
        }
    }
    else
    {
        printf("%d:%d\n", __LINE__, result);
    }
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result == success)
{
    printf("%d:机器人已成功断开连接", __LINE__);
}
else
{
    printf("%d:%d\n", __LINE__, result);
    printf("%d:机器人断开连接失败", __LINE__);
}
system("pause");

```

## 4.2 工程控制

### 简介

本示例介绍了对工程进行控制的方法，包括下载、上传、运行工程。

注：本示例中下载的工程会将机械臂的 4、5、6 关节各运动  $5^{\circ}$ ，故使用本示例前，需注意机械臂运动过程中是否会和其他物体发生碰撞，以及是否会超出关节角度范围限值。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); // 机械臂连接
if (result == success)
{
    Lua_ScriptStatus scriptstatus;
    result = cr_get_lua_scriptstatus(robotHandle, &scriptstatus); //读脚本运行状态
    if (result == success && scriptstatus == lua_Script_stop)
        //当脚本运行状态处于脚本程序停止时才可进行程序下载
    {
        char crpFilepathname[100] = "../program/demo.crp"; //crp 文件路径
        char crscriptFilepathname[100] = "../program/demo.crscript"; //脚本程序路径
        result = cr_downloadProject(robotHandle, crpFilepathname, crscriptFilepathname);
        //下载工程
        if (result == success)
        {
            result = cr_play(robotHandle); //运行工程
            if (result == success)
            {
                while (result == success)
                {
                    enum RobotModes robotMode;
                    result = cr_get_robotMode(robotHandle, &robotMode);
                    if (robotMode == 103)
                        //读机械臂当前状态,程序停止状态下才可以上传程序
                    break;
                }
                if (result == success) //判断状态
                {
                    char FilePath[100] = "../program/"; //文件保存路径
                    char filename[100] = "demo"; //crp 和脚本程序的文件名
                    result = cr_uploadProject(robotHandle, FilePath, filename);
                    //工程文件上传
                    if (result == success)
                    {
                        printf("%d:%s\n", __LINE__, "已成功上传工程文件");
                    }
                }
            }
        }
    }
}
```

```

        }
        else
        {
            printf("%d:%d\n", __LINE__, result);
        }
    }
    else
    {
        printf("%d:%d\n", __LINE__, result);
    }
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
}
else
{
    printf("%d:%d\n", __LINE__, scriptstatus);
    printf("%d:%d\n", __LINE__, result);
}
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result == success)
{
    printf("%d:机器人已成功断开连接", __LINE__);
}
else
{
    printf("%d:%d\n", __LINE__, result);
    printf("%d:机器人断开连接失败", __LINE__);
}
system("pause");

```



## 4.3 移动控制

### 简介

本示例介绍了 moveJ、moveL、cr\_move\_joint、cr\_move\_line 四个接口的使用方法，moveJ 和 moveL 是非阻塞移动，cr\_move\_joint 和 cr\_move\_line 可以设置是否需要阻塞移动。

注：使用本示例前，需注意机械臂运动过程中是否会和其他物体发生碰撞。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); // 机械臂连接
if (result == success)
{
    double jointpos[6] = { 0, 0, 90, 0, -90, 0 }; //目标点示例数据
    double pose[6] = { 0, 0, 0, 0, 0, 0 }; //在 MoveJ 中无影响
    PointControlPara pointControlPara;
    for (int index = 0; index < ROB_AXIS_NUM; index++)
    {
        pointControlPara.pose[index] = pose[index];
        pointControlPara.jointpos[index] = jointpos[index];
        pointControlPara.tcpOffset[index] = 0;
        pointControlPara.coordinatePose[index] = 0;
        pointControlPara.speed[index] = 30;
        pointControlPara.acc[index] = 30;
        pointControlPara.jerk[index] = 60;
    }
    pointControlPara.tcpID = -1;
    pointControlPara.coordinateType = jointCoordinate;
    pointControlPara.pointTransType = pointTransStop;
    pointControlPara.pointTransRadius = 0;
    pointControlPara.poseTranType = poseTranMoveToTargetPose;
    pointControlPara.motiontriggerMode = MovetriggerbyOnlyRpc;
    result = cr_moveJ(robotHandle, pointControlPara); //轴空间运动
    if (result == success)
    {
        Sleep(50);
        BOOL isRobotMoving;
        while (result == success)
        {
            result = cr_get_robotMoveStatus(robotHandle, &isRobotMoving);
            //读机械臂运动状态,如果运动已经停止,代表上一次运动已经结束,则跳出
            //循环进行下一次运动
            if (!isRobotMoving)
            {
                break;
            }
        }
    }
}
```

```

    }
}
result = cr_get_tcpActualPose(robotHandle, pose); //读实际 TCP 位置
pose[2] = pose[2] - 150; //Z 方向向下运动 150mm
if (result == success)
{
    pointControlPara.coordinateType = baseCoordinate;
    for (int index = 0; index < ROB_AXIS_NUM; index++)
    {
        pointControlPara.pose[index] = pose[index];
        pointControlPara.jointpos[index] = jointpos[index];
    }
    result = cr_moveL(robotHandle, pointControlPara); //直线运动
    if (result == success)
    {
        Sleep(50);
        while (result == success)
        {
            result = cr_get_robotMoveStatus(robotHandle, &isRobotMoving);
            //读机械臂运动状态,如果运动已经停止,代表上一次运动已经结束,则跳出循环进行下一次运动
            if (!isRobotMoving)
            {
                break;
            }
        }
        pose[2] = pose[2] + 150;
        PointControlParaSimple pointControlParaSimple;
        for (int index = 0; index < ROB_AXIS_NUM; index++)
        {
            pointControlParaSimple.pose[index] = pose[index];
            pointControlParaSimple.jointpos[index] = jointpos[index];
            pointControlParaSimple.tcpOffset[index] = 0;
            pointControlParaSimple.coordinatePose[index] = 0;
            pointControlParaSimple.speed[index] = 30;
            pointControlParaSimple.acc[index] = 30;
        }
        pointControlParaSimple.coordinateType = baseCoordinate;
        pointControlParaSimple.pointTransType = pointTransStop;
        pointControlParaSimple.motiontriggerMode = MovetriggerbyOnlyRpc;
        result = cr_move_pointControlPara_transfer(robotHandle,
pointControlParaSimple, &pointControlPara);
        //PointControlParaSimple 数据转换为 PointControlPara
        result = cr_move_line(robotHandle, pointControlPara, TRUE);
        //阻塞式的直线运动,由于是阻塞的直线运动,则运动会在运动到位后

```

在进行后续的程序运行，因此不用像之前做判断

```

        if (result == success)
        {
            double jointpos2[6] = { 90, 0, 90, 0, -90, 0 };
            pointControlParaSimple.coordinateType = jointCoordinate;
            for (int index = 0; index < ROB_AXIS_NUM; index++)
            {
                pointControlParaSimple.pose[index] = pose[index];
                pointControlParaSimple.jointpos[index] = jointpos2[index];
            }
            result = cr_move_pointControlPara_transfer(robotHandle,
pointControlParaSimple, &pointControlPara);
            //PointControlParaSimple 数据转换为 PointControlPara
            result = cr_move_joint(robotHandle, pointControlPara, TRUE);
            //阻塞式的轴空间运动，由于是阻塞的轴空间运动，则运动会在运
运动到位后在进行后续的程序运行，因此不用像之前做判断
            if (result == success)
            {
                pointControlPara.coordinateType = baseCoordinate;
                result = cr_get_tcpActualPose(robotHandle, pose);
                //读实际 TCP 位置
                pose[2] = pose[2] - 150;
                for (int index = 0; index < ROB_AXIS_NUM; index++)
                {
                    pointControlParaSimple.pose[index] = pose[index];
                    pointControlParaSimple.jointpos[index] = jointpos2[index];
                }
                result = cr_move_pointControlPara_transfer(robotHandle,
pointControlParaSimple, &pointControlPara);
                //PointControlParaSimple 数据转换为 PointControlPara
                result = cr_moveL(robotHandle, pointControlPara); //直线运动
                if (result == success)
                {
                    Sleep(50);
                    while (result == success)
                    {
                        result = cr_get_robotMoveStatus(robotHandle,
&isRobotMoving);
                        //读机械臂运动状态,如果运动已经停止，代表上一次
运动已经结束，则跳出循环进行下一次运动
                        if (!isRobotMoving)
                        {
                            break;
                        }
                    }
                }
            }
        }
    }

```



```
    }  
    }  
    else  
    {  
        printf("%d:%d\n", __LINE__, result);  
    }  
}  
else  
{  
    printf("%d:%d\n", __LINE__, result);  
}  
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄  
if (result == success)  
{  
    printf("%d:机器人已成功断开连接", __LINE__);  
}  
else  
{  
    printf("%d:%d\n", __LINE__, result);  
    printf("%d:机器人断开连接失败", __LINE__);  
}  
system("pause");
```

## 4.4 轨迹再现

### 简介

本示例介绍了记录一段轨迹并把轨迹下载到控制柜中的方法。

注：本示例将轨迹下载到索引 2 中，如果控制器中已存在索引为 2 的轨迹，将新记录的轨迹下载到索引 2 后，原轨迹将会被覆盖。使用本示例前，需注意机械臂运动过程中是否会和其他物体发生碰撞。

### 示例

```
RobotHandle robotHandle;  
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); // 机械臂连接  
if (result == success)  
{  
    double jointpos[6] = { 0, 0, 90, 0, -90, 0 }; //目标点示例数据  
    double pose[6] = { 0, 0, 0, 0, 0, 0 }; //在 MoveJ 中无影响  
    PointControlParaSimple pointControlParaSimple;  
    for (int index = 0; index < ROB_AXIS_NUM; index++)  
    {  
        pointControlParaSimple.pose[index] = pose[index];  
        pointControlParaSimple.jointpos[index] = jointpos[index];  
        pointControlParaSimple.tcpOffset[index] = 0;
```

```

        pointControlParaSimple.coordinatePose[index] = 0;
        pointControlParaSimple.speed[index] = 30;
        pointControlParaSimple.acc[index] = 30;
    }
    pointControlParaSimple.coordinateType = jointCoordinate;
    pointControlParaSimple.pointTransType = pointTransStop;
    pointControlParaSimple.motiontriggerMode = MovetriggerbyOnlyRpc;
    PointControlPara pointControlPara;
    result = cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple,
    &pointControlPara); //PointControlParaSimple 数据转换为 PointControlPara
    result = cr_move_joint(robotHandle, pointControlPara, TRUE); //轴空间运动
    RecordPathPara recordPathPara;
    recordPathPara.sampleTime = 2;
    recordPathPara.recordControl = 1;
    result = cr_path_recordPara_set(robotHandle, recordPathPara);
    //设置轨迹记录参数，开启轨迹记录
    Sleep(500);
    double jointpos2[6] = { 90, 0, 90, 0, -90, 0 }; //目标点示例数据
    for (int index = 0; index < ROB_AXIS_NUM; index++)
    {
        pointControlParaSimple.jointpos[index] = jointpos2[index];
    }
    result = cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple,
    &pointControlPara); //PointControlParaSimple 数据转换为 PointControlPara
    if (result == success)
    {
        result = cr_move_joint(robotHandle, pointControlPara, TRUE); //阻塞式轴空间运动
        if (result == success)
        {
            recordPathPara.recordControl = 0;
            result = cr_path_recordPara_set(robotHandle, recordPathPara);
            //设置轨迹记录参数，结束轨迹记录
            if (result == success)
            {
                int index = -1;
                PathData pathData;
                pathData.pathPoints = new PathPoint[10000];
                result = cr_path_upload(robotHandle, index, &pathData);
                if (result == success)
                {
                    int allPathIndex[11];
                    int pathIndexLen;
                    result = cr_path_all_index_get(robotHandle, allPathIndex,
    &pathIndexLen);
                    //读取控制柜当前存在的轨迹索引

```

```

for (int i = 0; i < pathIndexLen; i++)
{
    if (allPathIndex[i] == 2)
        //将下载的轨迹索引设置为 2，故需要判断其是否已经存在
        {
            printf("%d:%s\n", __LINE__, "存在轨迹索引为 2 的轨迹");
            PathDownloadData pathDownloadData;
            pathDownloadData.pathData = pathData;
            pathDownloadData.pathPara.index = 2;
            pathDownloadData.pathPara.moveType = 1;
            result = cr_path_download(robotHandle, pathDownloadData);
            //下载轨迹到索引 2
            if (result == success)
            {
                printf("%d:%s\n", __LINE__, "已成功下载索引为 2 的
轨迹");
            }
            else
            {
                printf("%d:%d\n", __LINE__, result);
            }
            break;
        }
    if (i == (pathIndexLen - 1))
    {
        PathDownloadData pathDownloadData;
        pathDownloadData.pathData = pathData;
        pathDownloadData.pathPara.index = 2;
        pathDownloadData.pathPara.moveType = 1;
        result = cr_path_download(robotHandle, pathDownloadData);
        //下载轨迹到索引 2
        if (result == success)
        {
            printf("%d:%s\n", __LINE__, "已成功下载索引为 2 的
轨迹");
        }
        else
        {
            printf("%d:%d\n", __LINE__, result);
        }
    }
}
else
{

```

```

        printf("%d:%d\n", __LINE__, result);
    }
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result == success)
{
    printf("%d:机器人已成功断开连接", __LINE__);
}
else
{
    printf("%d:%d\n", __LINE__, result);
    printf("%d:机器人断开连接失败", __LINE__);
}
system("pause");

```

## 4.5 安装变量操作

### 简介

本示例介绍了与安装变量相关的接口的使用方法，包括读取安装变量个数、读取安装变量、修改安装变量、增加安装变量。

### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); // 机械臂连接
char name[CR6_INSTALL_VAR_NAME_LENGTH] = { "i_var_1" };

```



```
VariableMsg variablemsg;
strcpy_s(variablemsg.variableName, sizeof(name), name);
variablemsg.variableType = LUA_TBOOLEAN;
variablemsg.boolValue = TRUE;
variablemsg.variableID = 3;
if (result == success)
{
    int count = -1;
    result = cr_cfg_var_install_count(robotHandle, &count); //读取安装变量个数
    if (result == success)
    {
        if (count > 0)
        {
            for (int i = 0; i < count; i++)
            {
                VariableMsg variableMsg;
                result = cr_cfg_var_install_get(robotHandle, i, &variableMsg)
                if (strcmp(name, variableMsg.variableName) == 0)
                //判断是否已存在名为 i_var_1 的安装变量，如果有就修改，没有就增加
                {
                    result = cr_cfg_var_install_set(robotHandle, i, &variablemsg);
                    if (result == success)
                    {
                        printf("%s\n", "已成功修改安装变量");
                    }
                    else
                    {
                        printf("%d:%d\n", __LINE__, result);
                    }
                    break;
                }
            }
            if (i == (count - 1))
            {
                result = cr_cfg_var_install_add(robotHandle, &variablemsg);
                //增加安装变量
                if (result == success)
                {
                    printf("%s\n", "已成功增加安装变量");
                }
                else
                {
                    printf("%d:%d\n", __LINE__, result);
                }
            }
        }
    }
}
```

```

    }
    else //如果不存在安装变量，则添加安装变量
    {
        result = cr_cfg_var_install_add(robotHandle, &variablemsg); //增加安装变量
        if (result == success)
        {
            printf("%d:%s\n", __LINE__, "已成功增加安装变量");
        }
        else
        {
            printf("%d:%d\n", __LINE__, result);
        }
    }
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
}
result = cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result == success)
{
    printf("%d:机器人已成功断开连接", __LINE__);
}
else
{
    printf("%d:%d\n", __LINE__, result);
    printf("%d:机器人断开连接失败", __LINE__);
}
}
system("pause");

```

## 4.6 点坐标系操作

### 简介

本示例介绍了与点坐标系操作相关的接口的使用方法，包括读取点坐标系个数、读取点坐标系、修改安装变量、增加安装变量、删除点坐标系。

### 示例

```

RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); // 机械臂连接

```

```

PointCSNode pointCSNode;
char name[32] = { "point3" };
strcpy_s(pointCSNode.name, sizeof(name), name);
double toolAxisAngle[6] = { 30,30,30,30,30,30 };
double toolPosition[6];
if (result == success)
{
    int count;
    result = cr_cfg_cs_point_count(robotHandle, &count); //读取点坐标系个数
    for (int i = 0; i < 6; i++)
    {
        pointCSNode.point.toolAxisAngle[i] = toolAxisAngle[i];
    }
    if (result == success)
    {
        if (count > 0)
        {
            for (int i = 0; i < count; i++)
            {
                PointCSNode pointCSNode;
                result = cr_cfg_cs_point_get(robotHandle, i, &pointCSNode);
                if (name == pointCSNode.name)
                {
                    result = cr_kineForward(robotHandle, toolAxisAngle, toolPosition);
                    //正解求点的位姿
                    if (result == success)
                    {
                        pointCSNode.isValid = TRUE;
                        pointCSNode.id = 1;
                        result = cr_cfg_cs_point_set(robotHandle, i, pointCSNode);
                    }
                    else
                    {
                        printf("%d:%d\n", __LINE__, result);
                    }
                    break;
                }
            }
            if (i == (count - 1))
            {
                result = cr_kineForward(robotHandle, toolAxisAngle, toolPosition);
                //正解求点的位姿
                if (result == success)
                {
                    pointCSNode.isValid = TRUE;
                    pointCSNode.id = 1;
                }
            }
        }
    }
}

```

```

        result = cr_cfg_cs_point_add(robotHandle, pointCSNode);
        //增加点坐标系数据
    }
    else
    {
        printf("%d:%d\n", __LINE__, result);
    }
}
}
else
{
    pointCSNode.isValid = TRUE;
    pointCSNode.id = 1;
    result = cr_cfg_cs_point_add(robotHandle, pointCSNode); //增加点坐标系数据
}
int index = 1;
result = cr_cfg_cs_point_get(robotHandle, index, &pointCSNode);
//读取索引号为 1 的点坐标系数据,可能不存在该坐标系
if (result == success)
{
    printf("%s\n", pointCSNode.name);
    index = 0;
    result = cr_cfg_cs_point_delete(robotHandle, index);
    //删除索引号为 0 的点坐标系
    if (result == success)
    {
        printf("%s\n", "已成功删除点坐标系");
    }
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
}
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
}
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result == success)
{
    printf("%d:机器人已成功断开连接", __LINE__);
}

```

```
else
{
    printf("%d:%d\n", __LINE__, result);
    printf("%d:机器人断开连接失败", __LINE__);
}
system("pause");
```

## 4.7 输出弹窗

### 简介

本示例介绍了读取输出弹窗的方法，首先通过加载运行程序来创建一个输出弹窗，再判断弹窗是否存在，若存在则读取输出弹窗信息。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); // 机械臂连接
if (result == success)
{
    const char *programfile = "function mainFuncProgram()\n"
        "--start robotConfig\n"
        "TCP_1 = { 0,0,0,0,0,0 }\n"
        "Payload_1= { 0,{0,0,0} }\n"
        "set_tcp_payload(Payload_1[1], Payload_1[2])\n"
        "--end robotConfig\n"
        "function RobotProgram()\n"
        "while (true)\n"
        "do\n"
        "--popup_out:text\n"
        "popup_message(\"hello\", \"message\", false, false, true)\n"
        "wait(1) --sync\n"
        "end\n"
        "end\n"
        "RobotProgram_Result = task_create(RobotProgram)\n"
        "function PauseFuncProgram()\n"
        "while (true)\n"
        "do\n"
        "wait(100) --sync\n"
        "end\n"
        "end\n"
        "PauseFuncProgram_Result = task_create(PauseFuncProgram)\n"
        "end\n"
        "mainFuncProgram_Result = task_create(mainFuncProgram)";
    result = cr_downloadProgram(robotHandle, programfile); //加载程序
    if (result == success)
```

```

{
    result = cr_play(robotHandle); //运行程序
    if (result == success)
    {
        BOOL hasPopup;
        result = cr_script_popup_exist(robotHandle, &hasPopup);
        //是否存在弹窗：1-是，0-否
        if (result == success)
        {
            if (hasPopup == TRUE)
            {
                PopUpMsg popupMsg;
                result = cr_script_popup_msg_get(robotHandle, &popupMsg);
                //读取弹窗信息
                if (result == success)
                {
                    printf("%d:%d\n", __LINE__, popupMsg.popupType);
                    result = cr_stop(robotHandle); //停止程序运行
                }
                else
                {
                    printf("%d:%d\n", __LINE__, result);
                }
            }
            else
            {
                printf("%d:%s\n", __LINE__, "没有弹窗信息");
            }
        }
        else
        {
            printf("%d:%d\n", __LINE__, result);
        }
    }
    else
    {
        printf("%d:%d\n", __LINE__, result);
    }
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
}
else

```

```
{
    printf("%d:%d\n", __LINE__, result);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result == success)
{
    printf("%d:机器人已成功断开连接", __LINE__);
}
else
{
    printf("%d:%d\n", __LINE__, result);
    printf("%d:机器人断开连接失败", __LINE__);
}
system("pause");
```

## 4.8 数学变量

### 简介

本示例介绍了将当前基坐标系下的 TCP 位姿转换到点坐标系下的过程。

### 示例

```
RobotHandle robotHandle;
CRresult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); // 机械臂连接
if (result == success)
{
    PointCSNode pointCSNode;
    char name[32] = { "point3" };
    strcpy_s(pointCSNode.name, sizeof(name), name);
    double toolAxisAngle[6] = { 30,30,30,30,30,30 };
    double toolPosition[6];
    for (int i = 0; i < 6; i++)
    {
        pointCSNode.point.toolAxisAngle[i] = toolAxisAngle[i];
    }
    result = cr_kineForward(robotHandle, toolAxisAngle, toolPosition); //正解求点的位姿
    if (result == success)
    {
        for (int i = 0; i < 6; i++)
        {
            pointCSNode.point.toolPosition[i] = toolPosition[i];
        }
        pointCSNode.isValid = TRUE;
        pointCSNode.id = 1;
        result = cr_cfg_cs_point_add(robotHandle, pointCSNode); //增加点坐标系
    }
}
```

```

Sleep(500);
if (result == success)
{
    double pose[ROB_AXIS_NUM];
    result = cr_get_tcpActualPose(robotHandle, pose); //读实际 TCP 位置
    if (result == success)
    {
        int basePoseLen = 6;
        int userPoseLen = 6;
        int poseInUserLen = 6;
        double basePose[6];
        double userPose[6];
        for (int i = 0; i < ROB_AXIS_NUM; i++)
        {
            basePose[i] = pose[i];
            userPose[i] = pointCSNode.point.toolPosition[i];
            //将点坐标系设置为用户坐标系
        }
        double poseInUser[6];
        result = cr_compute_pose_base_to_user(basePose, basePoseLen, userPose,
userPoseLen, poseInUser, poseInUserLen); //基坐标系下位姿转移到用户坐标系下
        if (result == success)
        {
            for (int i = 0; i < 6; i++)
            {
                printf("%d:%f\n", __LINE__, poseInUser[i]);
            }
        }
        else
        {
            printf("%d:%d\n", __LINE__, result);
        }
    }
    else
    {
        printf("%d:%d\n", __LINE__, result);
    }
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
}
else
{

```



```

        printf("%d:%d\n", __LINE__, result);
    }
}
else
{
    printf("%d:%d\n", __LINE__, result);
}
result = cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result == success)
{
    printf("%d:机器人已成功断开连接", __LINE__);
}
else
{
    printf("%d:%d\n", __LINE__, result);
    printf("%d:机器人断开连接失败", __LINE__);
}
system("pause");

```

## 4.9 TCP 操作

### 简介

本示例介绍了与 TCP 操作相关的接口的使用方法, 包括读取 TCP、修改 TCP、增加 TCP、删除 TCP、激活 TCP、读取当前激活的 TCP 索引。

### 示例

```

RobotHandle robotHandle;
CResult result = cr_create_robot(&robotHandle, "192.168.6.6", 2323, "123"); // 机械臂连接
if (result == success)
{
    int index;
    result = cr_cfg_tcp_active_get(robotHandle, &index); //读取当前激活的 TCP 索引
    if (result == success)
    {
        TCPMsg tcpMsg;
        index = 0;
        result = cr_cfg_tcp_get(robotHandle, index, &tcpMsg);
        //需要先存在索引号为 0 的 TCP
        if (result == success)
        {
            double offset[6] = { 3,10,9,90,80,70 };
            for (int i = 0; i < 6; i++)
            {
                tcpMsg.tcpOffset[i] = offset[i];
            }
        }
    }
}

```

```

    }
    result = cr_cfg_tcp_set(robotHandle, index, tcpMsg);
    //编辑索引号为 0 的 TCP 数据
    if (result == success)
    {
        TCPMsg tcpMsg2;
        char name[20] = { "TCP" };
        strcpy_s(tcpMsg2.tcpName, sizeof(name), name);
        double offset2[6] = { 5,10,15,10,20,30 };
        for (int i = 0; i < 6; i++)
        {
            tcpMsg2.tcpOffset[i] = offset2[i];
        }
        result = cr_cfg_tcp_add(robotHandle, tcpMsg2); //增加 TCP
        if (result == success)
        {
            index = 1;
            result = cr_cfg_tcp_active_set(robotHandle, index);
            //激活索引号为 1 的 TCP
            if (result == success)
            {
                result = cr_cfg_tcp_active_get(robotHandle, &index);
                if (result == success)
                {
                    printf("%d:当前激活的 TCP 为%d\n", __LINE__, index);
                    index = 0;
                    result = cr_cfg_tcp_delete(robotHandle, index);
                    //删除索引号为 0 的 TCP
                    if (result == success)
                    {
                        printf("%d:已成功删除索引号为 0 的 TCP", __LINE__);
                    }
                    else
                    {
                        printf("%d:%d\n", __LINE__, result);
                    }
                }
            }
            else
            {
                printf("%d:%d\n", __LINE__, result);
            }
        }
        else
        {
            printf("%d:%d\n", __LINE__, result);
        }
    }

```



## 附录 A 速度和加速度范围

### 各机型对应速度范围

| 机型          | 关节速度范围限制( $^{\circ}/s$ ) | TCP 速度范围限制(mm/s) |
|-------------|--------------------------|------------------|
| C3          | 1-180                    | 0.01-1000        |
| C4/C6/C6-DK | 1-180                    | 0.01-2000        |
| G3          | 1-360                    | 0.01-2000        |
| G3a         | 1-270                    | 0.01-1500        |
| G4/G6/G7    | 1-360                    | 0.01-3000        |
| G4a/G6a     | 1-270                    | 0.01-2000        |
| G9/G10/G12  | 1-240                    | 0.01-3000        |
| G9a/G12a    | 1-180                    | 0.01-2000        |
| G18         | 1-240                    | 0.01-2000        |
| G18a        | 1-180                    | 0.01-1500        |
| G20         | 1-270                    | 0.01-4000        |
| G30         | 1-270                    | 0.01-3000        |
| X6          | 1-360                    | 0.01-3000        |
| X9/X12/X18  | 1-240                    | 0.01-3000        |

### 各机型对应加速度范围

关节加速度限值统一为  $0.01\sim 2500\text{ }^{\circ}/s^2$ ;

TCP 加速度限值统一为  $0.01\sim 20000\text{ mm}/s^2$ 。

## 各机型对应点动速度范围

关节点动速度限值统一为  $0.1\sim15\text{ }^{\circ}/\text{s}$ ;

关节点动加速度限值统一为  $0.01\sim90\text{ }^{\circ}/\text{s}^2$ 。

TCP 点动速度限值统一为  $0.1\sim120\text{ mm/s}$ ;

TCP 点动加速度限值统一为  $0.01\sim720\text{ mm/s}^2$ 。